

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



Lê Quốc Việt

**NGHIÊN CỨU THỰC HIỆN MỘT ROBOT DI ĐỘNG
CHO NHIỆM VỤ ĐỊNH VỊ VÀ VẼ BẢN ĐỒ ĐỒNG
THỜI (SLAM)**

ĐỒ ÁN TỐT NGHIỆP ĐẠI HỌC HỆ CHÍNH QUY

Ngành: Kỹ thuật máy tính

HÀ NỘI – 2026

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

Lê Quốc Việt

NGHIÊN CỨU THỰC HIỆN MỘT ROBOT DI ĐỘNG
CHO NHIỆM VỤ ĐỊNH VỊ VÀ VẼ BẢN ĐỒ ĐỒNG
THỜI (SLAM)

ĐỒ ÁN TỐT NGHIỆP ĐẠI HỌC HỆ CHÍNH QUY

Ngành: Kỹ thuật máy tính

Cán bộ hướng dẫn: TS. Hoàng Gia Hưng

Cán bộ đồng hướng dẫn: TS. Phạm Duy Hưng

HÀ NỘI – 2026

TÓM TẮT

Tóm tắt: Đồ án tập trung nghiên cứu và thực hiện thuật toán Định vị và lập bản đồ đồng thời (SLAM) cho mô hình robot di động hai bánh vi sai trong môi trường trong nhà. Để khắc phục hạn chế về sai số tuyến tính hóa của phương pháp EKF-SLAM truyền thống, nghiên cứu triển khai một cấu trúc hai tầng xử lý dựa trên Bộ lọc Kalman Unscented (UKF). Tầng thứ nhất thực hiện hợp nhất dữ liệu phi đồng bộ (Sensor Fusion) từ encoder, IMU và từ kế bằng thuật toán Adaptive UKF để ước lượng chính xác tư thế robot. Tầng thứ hai triển khai thuật toán UKF-SLAM kết hợp dữ liệu quét từ cảm biến 2D LiDAR, sử dụng phương pháp Phân tích thành phần chính (PCA) để trích xuất đặc trưng hình học và khoảng cách Mahalanobis để liên kết dữ liệu mốc bản đồ (landmark).

Toàn bộ hệ thống được đóng gói thành các node xử lý độc lập, phát triển trên nền tảng ROS 2 Jazzy và vận hành theo thời gian thực trên máy tính nhúng Raspberry Pi 5. Kết quả thực nghiệm thực tế chứng minh robot có khả năng định vị ổn định với sai số thấp, xây dựng bản đồ chính xác và đáp ứng tốt các kịch bản điều hướng tự động, tránh vật cản trực tuyến.

Từ khóa: *SLAM; Adaptive UKF; Sensor Fusion; robot vi sai; 2D LiDAR; ROS 2 Jazzy; Raspberry Pi 5.*

LỜI CAM ĐOAN

Tôi xin cam đoan Đồ án “Nghiên cứu thực hiện một robot di động cho nhiệm vụ định vị và vẽ bản đồ đồng thời (SLAM)” được thực hiện bởi bản thân dưới sự hướng dẫn của TS. Hoàng Gia Hưng.

Mọi tham khảo, trích dẫn từ các nghiên cứu và tài liệu liên quan được sử dụng trong khóa luận đều được nêu nguồn gốc một cách rõ ràng tại danh mục tài liệu tham khảo. Trong khóa luận hoàn toàn không có sự sao chép tài liệu cũng như công trình nghiên cứu của người khác.

Hà Nội, ngày 23 tháng 05 năm 2026

Tác giả

Lê Quốc Việt

LỜI CẢM ƠN

Đầu tiên, em xin gửi lời cảm ơn chân thành đến toàn thể quý thầy, cô trong Trường Đại học Công nghệ – Đại học Quốc gia Hà Nội đã tạo rất nhiều điều kiện và giúp đỡ tận tình cho sinh viên chúng em.

Đặc biệt, em xin gửi lời cảm ơn sâu sắc nhất đến TS. Hoàng Gia Hưng và TS. Phạm Duy Hưng vì đã tận tình hướng dẫn, giúp đỡ, tạo động lực và cho em những bài học quý giá trong suốt quá trình học tập tại Trường Đại học Công nghệ – Đại học Quốc gia Hà Nội nói chung và trong quá trình nghiên cứu, hoàn thành đồ án nói riêng.

Cuối cùng, em xin gửi lời cảm ơn đến gia đình, bạn bè và tập thể lớp K67-E-CE1 vì đã luôn đồng hành, giúp đỡ cũng như động viên em trong những quãng thời gian khó khăn. Rất nhiều điều em học từ mọi người đã giúp em nhiều trong quá trình nghiên cứu. Mọi người luôn là nguồn động lực để em phấn đấu học tập, hoàn thiện và phát triển bản thân.

Lê Quốc Việt

Mục lục

Chương 1. Tổng quan về đề tài	1
1.1. SLAM trong hệ thống robot di động	1
1.2. Mục tiêu nghiên cứu	1
1.3. Đối tượng và phạm vi nghiên cứu	2
1.4. Bố cục đồ án	2
Chương 2. Cơ sở lý thuyết và tổng quan nghiên cứu	4
2.1. Bài toán SLAM và các nghiên cứu liên quan	4
2.1.1. Bài toán SLAM (Simultaneous Localization and Mapping) trong robot di động	4
2.1.2. Các phương pháp tiếp cận, hạn chế và hướng nghiên cứu	6
2.1.3. Các công trình liên quan	7
2.2. Cơ sở lý thuyết	8
2.2.1. Động học robot di động bánh xe vi sai	8
2.2.2. Các loại cảm biến sử dụng trong hệ thống	11
2.2.3. Kalman Filter và các mở rộng cho hệ phi tuyến	13
2.2.4. UKF và Unscented Transform	16
2.2.5. Biểu diễn bản đồ trong SLAM	19
2.2.6. UKF-SLAM, PCA và khoảng cách Mahalanobis	20
Chương 3. Ước lượng trạng thái với Adaptive UKF và UKF-SLAM	25
3.1. Thiết kế Adaptive UKF cho sensor fusion	25
3.1.1. Mô hình trạng thái (State Model)	25
3.1.2. Mô hình quan sát (Measurement Model)	26
3.1.3. Cơ chế thích nghi (Adaptive Mechanism)	28
3.1.4. Sensor Fusion và Unscented Kalman Filter	30
3.1.5. Pseudo-code Adaptive UKF Sensor Fusion	34
3.2. Thuật toán UKF-SLAM	35
3.2.1. Biểu diễn trạng thái và tham số UKF	35

3.2.2. Prediction step từ gia số odometry	36
3.2.3. Mô hình quan sát landmark	38
3.2.4. Feature extraction từ LiDAR	40
3.2.5. Data association và update	43
3.2.6. Khởi tạo, loại bỏ và hiển thị landmark	45
3.2.7. Pseudo-code UKF-SLAM	47
Chương 4. Thiết kế hệ thống và thực nghiệm đánh giá	49
4.1. Hệ thống robot và giao diện API dữ liệu	49
4.1.1. Tổng quan hệ thống robot thực nghiệm	49
4.1.2. Giao diện API và luồng dữ liệu ROS 2	50
4.1.3. IMU – Vận tốc góc và đặc tính nhiễu gyroscope	52
4.1.4. Encoder – Wheel odometry và đặc tính nhiễu	55
4.1.5. Từ kế – Tham chiếu góc hướng tuyệt đối	56
4.1.6. Kết quả so sánh các nguồn cảm biến trong chuyển động	58
4.2. Thiết kế phần mềm	61
4.2.1. Kiến trúc tổng thể	61
4.2.2. Node hợp nhất cảm biến state_estimate_UKF	62
4.2.3. Node UKF-SLAM ukf_slam	64
4.2.4. Các node phụ trợ	66
4.3. Kết quả thực nghiệm	66
4.3.1. Chất lượng định vị	66
4.3.2. Chất lượng bản đồ	68
4.3.3. Kết quả điều hướng tự động	71
Chương 5. Kết luận và hướng phát triển	73
5.1. Kết quả đạt được	73
5.2. Hạn chế	74
5.3. Hướng phát triển	74
Phụ lục A. Mã nguồn và kết quả thực nghiệm	76
Phụ lục B. Bảng so sánh đặc tính cảm biến và bộ lọc	78

Danh sách hình vẽ

2.1. Bài toán SLAM với trạng thái robot, bản đồ, điều khiển và quan sát cảm biến	5
2.2. Cơ cấu di chuyển robot hai bánh vi sai	9
2.3. Mô hình động học	9
2.4. Mô hình robot di chuyển quanh ICC	10
2.5. Cấu tạo encoder	12
2.6. Cảm biến IMU	12
2.7. Minh họa ước lượng Kalman Filter trong định vị một chiều	14
2.8. So sánh tuyến tính hóa EKF và Unscented Transform với sigma points	18
2.9. Minh họa thuật toán Principal Component Analysis.	21
2.10. Minh họa hệ trục mới sau biến đổi PCA với hai thành phần chính PC1 và PC2.	22
2.11. So sánh giữa khoảng cách Euclid và khoảng cách Mahalanobis.	23
2.12. Khoảng cách Mahalanobis trong bài toán so khớp quan sát mới với cụm tham chiếu và vùng ngưỡng chấp nhận.	24
3.1. Trích xuất đặc trưng từ lidar	42
4.1. Robot thực nghiệm dùng để thu thập dữ liệu cảm biến.	49
4.2. Sơ đồ giao tiếp API với phần cứng robot.	50
4.3. Quá trình lấy mẫu lại các tín hiệu cảm biến không đồng bộ để tạo vector quan sát tại thời điểm xử lý chung t_k	51
4.4. So sánh phương pháp giữ mẫu bậc không và nội suy tuyến tính khi khôi phục dữ liệu cảm biến không đồng bộ.	52
4.5. Tín hiệu vận tốc góc yaw của IMU khi robot đứng yên.	53
4.6. Tín hiệu vận tốc góc yaw của IMU khi robot quay đều.	53
4.7. So sánh tín hiệu gyroscope thô và sau lọc thông thấp trong đoạn robot gần trạng thái đứng yên.	54
4.8. So sánh tín hiệu gyroscope thô và sau lọc thông thấp trong đoạn có biến thiên vận tốc góc lớn.	54
4.9. Vận tốc góc của robot suy ra từ encoder bánh xe khi robot quay đều.	55

4.10. Vận tốc tuyến tính của robot suy ra từ encoder bánh xe khi robot chuyển động đều.	56
4.11. Dữ liệu góc yaw từ từ kế trong quá trình robot hoạt động.	57
4.12. So sánh các nguồn cảm biến khi robot đứng yên, vận tốc tuyến tính xấp xỉ 0 và góc hướng dao động nhỏ.	58
4.13. So sánh các nguồn cảm biến trong đoạn chuyển động có thay đổi góc quay lớn.	59
4.14. So sánh các nguồn cảm biến trong pha quay liên tục, thể hiện sai khác giữa góc tích phân từ từng nguồn đo.	59
4.15. So sánh các nguồn cảm biến trong pha chuyển trạng thái, khi vận tốc góc thay đổi rõ rệt theo thời gian.	60
4.16. Sơ đồ các node phần mềm và luồng dữ liệu chính trong hệ thống ROS 2.	62
4.17. Pipeline xử lý của node <code>ukf_slam</code> trong một chu kỳ cập nhật.	65
4.18. So sánh quỹ đạo khép vòng qua sáu vòng chạy hình chữ nhật của encoder, Adaptive UKF và UKF-SLAM.	67
4.19. Bản đồ landmark chồng lên occupancy grid sau khi robot hoàn thành hành trình khám phá môi trường.	69
4.20. Tiến trình xây dựng bản đồ qua sáu giai đoạn khám phá.	70
4.21. Đường kế hoạch A* (xanh lá) trong quá trình robot điều hướng tự động.	72

Danh sách bảng

4.1. Các topic dữ liệu chính trong hệ thống ROS 2.	50
4.2. Tóm tắt đặc tính các nguồn cảm biến và bộ lọc theo kịch bản chuyển động.	61
4.3. Sai số khép vòng.	68
A.1. Đường dẫn mã nguồn và kết quả thực nghiệm.	76
B.1. So sánh hành vi của các nguồn thông tin trong các kịch bản chuyển động.	78
B.2. Sai số diễn hình và chỉ số định lượng của các nguồn thông tin.	78
B.3. Độ tin cậy, vai trò trong fusion và hướng hiệu chuẩn.	79
B.4. Thống kê nhiều thực nghiệm của các nguồn cảm biến và Adaptive UKF.	79

Danh mục từ viết tắt

Từ viết tắt	Diễn giải
API	Application Programming Interface – giao diện lập trình ứng dụng.
ATE	Absolute Trajectory Error – sai số quỹ đạo tuyệt đối.
CPU	Central Processing Unit – bộ xử lý trung tâm.
EKF	Extended Kalman Filter – bộ lọc Kalman mở rộng.
GNN	Global Nearest Neighbor – liên kết dữ liệu láng giềng gần nhất toàn cục.
GPS	Global Positioning System – hệ thống định vị toàn cầu.
ICC	Instantaneous Center of Curvature – tâm cong tức thời của chuyển động vi sai.
IMU	Inertial Measurement Unit – đơn vị đo quán tính.
IIR	Infinite Impulse Response – bộ lọc đáp ứng xung vô hạn.
KF	Kalman Filter – bộ lọc Kalman.
LiDAR	Light Detection and Ranging – cảm biến đo khoảng cách bằng laser.
LPF	Low-Pass Filter – bộ lọc thông thấp.
NEES	Normalized Estimation Error Squared – sai số ước lượng chuẩn hóa bình phương.
PCA	Principal Component Analysis – phân tích thành phần chính.
QoS	Quality of Service – cấu hình chất lượng dịch vụ truyền thông ROS 2.
ROS	Robot Operating System – hệ điều hành/khung phần mềm cho robot.
RPE	Relative Pose Error – sai số pose tương đối.
SLAM	Simultaneous Localization and Mapping – định vị và xây dựng bản đồ đồng thời.
TF	Transform – cây biến đổi hệ tọa độ trong ROS.
UKF	Unscented Kalman Filter – bộ lọc Kalman Unscented.
VFH	Vector Field Histogram

Danh mục ký hiệu

Ký hiệu	Ý nghĩa
x, y, θ	Vị trí và góc hướng của robot trong hệ tọa độ phẳng.
v, ω	Vận tốc tuyến tính và vận tốc góc của robot.
$\mathbf{x}_k$	Vector trạng thái tại thời điểm k .
$\mathbf{x}^a$	Vector trạng thái mở rộng, gồm trạng thái robot và nhiều quá trình.
$\mathbf{z}_k$	Vector quan sát tại thời điểm k .
$\hat{\mathbf{x}}_{k k-1}$	Ước lượng trạng thái dự đoán trước khi cập nhật phép đo.
$\hat{\mathbf{x}}_{k k}$	Ước lượng trạng thái sau khi cập nhật phép đo.
P	Ma trận hiệp phương sai trạng thái.
Q	Ma trận hiệp phương sai nhiễu quá trình.
R	Ma trận hiệp phương sai nhiễu đo lường.
S	Ma trận hiệp phương sai innovation.
K	Kalman gain.
ν	Vector innovation giữa phép đo thực tế và phép đo dự đoán.
$\mathcal{X}$	Ma trận sigma points trong Unscented Transform/UKF.
$\mathcal{X}'$	Ma trận sigma points sau bước lan truyền dự đoán.
$W_i^{(m)}, W_i^{(c)}$	Trọng số trung bình và trọng số hiệp phương sai của sigma point thứ i .
α, β, κ	Các tham số điều chỉnh phân bố sigma points của UKF.
λ	Tham số tỉ lệ trong phép biến đổi Unscented.

Danh mục ký hiệu (tiếp)

Ký hiệu	Ý nghĩa
$m_j = [m_{x,j}, m_{y,j}]^T$	Tọa độ landmark thứ j trong bản đồ.
M	Số lượng landmark đang được duy trì trong trạng thái SLAM.
r, ϕ	Khoảng cách và góc phương vị của phép đo range-bearing.
$\Delta x_R, \Delta y_R, \Delta \theta$	Gia số chuyển động của robot giữa hai thời điểm liên tiếp.
Q_R	Ma trận nhiễu quá trình thích nghi của pose robot trong UKF-SLAM.
χ_{th}^2	Ngưỡng kiểm định Chi-square dùng trong data association.
μ, σ^2	Giá trị trung bình và phương sai của tín hiệu đo thực nghiệm.
$t_k, \Delta t$	Thời điểm lấy mẫu thứ k và khoảng thời gian giữa hai mẫu.
η_v, η_ω	Thành phần nhiễu quá trình ứng với vận tốc tuyến tính và vận tốc góc.
ε	Ngưỡng nhỏ dùng để phân biệt chuyển động thẳng và chuyển động theo ICC.

Chương 1.

Tổng quan về đề tài

1.1. SLAM trong hệ thống robot di động

Bài toán Định vị và lập bản đồ đồng thời (SLAM) là nền tảng cho các hệ robot di động tự hành trong môi trường trong nhà không có GPS. Một robot muốn hoạt động độc lập cần đồng thời ước lượng chính xác vị trí của chính nó và xây dựng bản đồ môi trường xung quanh theo thời gian thực. Đây là một bài toán cốt lõi vì mọi chức năng ở mức cao hơn như lập kế hoạch đường đi, tránh vật cản, điều hướng tới mục tiêu hay quản lý đội robot đều phụ thuộc trực tiếp vào chất lượng của lời giải SLAM.

Trong các phương pháp cổ điển, EKF-SLAM là hướng tiếp cận phổ biến nhờ cơ sở toán học rõ ràng và khả năng kết hợp nhiều loại cảm biến. Tuy nhiên, EKF-SLAM dựa vào phép tuyến tính hóa cục bộ thông qua ma trận Jacobian, do đó dễ gặp sai lệch khi mô hình có tính phi tuyến mạnh hoặc khi nhiều thay đổi theo thời gian. Sai số tuyến tính hóa tích lũy có thể dẫn đến hiện tượng mất tính nhất quán của bộ lọc.

UKF được đề xuất nhằm khắc phục một phần các hạn chế của EKF thông qua Unscented Transform, cho phép lan truyền phân bố xác suất qua hàm phi tuyến mà không cần tính Jacobian. Cách tiếp cận này thường cho kết quả chính xác hơn trong các mô hình phi tuyến và giảm phụ thuộc vào phép khai triển tuyến tính bậc nhất. Tuy vậy, UKF-SLAM vẫn gặp các vấn đề quan trọng như độ phức tạp tính toán bậc ba theo kích thước trạng thái khi số lượng landmark lớn, cũng như nguy cơ mất tính nhất quán do sai lệch về observability và do mô hình hóa nhiễu chưa thật sự phù hợp.

Đề tài đề xuất xây dựng một hệ thống Adaptive UKF cho sensor fusion và UKF-SLAM cải tiến, kết hợp phương pháp trích xuất đặc trưng LiDAR mạnh mẽ dựa trên PCA và khoảng cách Mahalanobis. Mục tiêu là nâng cao độ chính xác định vị, tăng độ bền vững trước nhiễu và ngoại lai, đồng thời có khả năng hoạt động thời gian thực trên nền tảng Raspberry Pi 5.

1.2. Mục tiêu nghiên cứu

Nghiên cứu và thực hiện thuật toán SLAM trên robot di động hai bánh vi sai hoạt động trong môi trường trong nhà, hướng tới khả năng định vị và xây dựng bản đồ đồng thời theo thời gian thực với độ chính xác và độ ổn định cao.

Cụ thể:

- Thiết kế bộ lọc Adaptive UKF cho bài toán hợp nhất dữ liệu từ encoder, IMU và từ kế nhằm ước lượng tư thế robot ổn định theo thời gian thực.
- Triển khai thuật toán UKF-SLAM với cơ chế state augmentation và cơ chế quản lý landmark phù hợp. Tích hợp phương pháp trích xuất đặc trưng LiDAR dựa trên PCA và liên kết dữ liệu bằng khoảng cách Mahalanobis.
- Triển khai thuật toán hoạt động ổn định trên nền tảng máy tính nhưng có tài nguyên hạn chế của robot hoạt động thời gian thực.
- Đánh giá thực nghiệm trên robot thật thông qua các chỉ số về quỹ đạo, chất lượng bản đồ và thời gian tính toán.

1.3. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu là robot di động bánh xe vi sai với hai bánh chủ động, sử dụng các cảm biến encoder, IMU và LiDAR để phục vụ bài toán định vị và lập bản đồ.

Đề án được giới hạn trong môi trường trong nhà có mặt sàn phẳng, tập trung vào bài toán SLAM hai chiều thay vì mô hình ba chiều đầy đủ. Hệ thống chưa đi sâu vào các kỹ thuật loop closure phức tạp; tuy nhiên, đây là một hướng mở có thể được tích hợp trong giai đoạn phát triển tiếp theo nếu thời gian cho phép. Nền tảng phần mềm sử dụng là ROS 2 Jazzy trên Raspberry Pi 5.

1.4. Bố cục đề án

Nội dung của đề án tốt nghiệp được tổ chức thành 5 chương chính như sau:

Chương 1: Mở đầu. Trình bày tổng quan về đề tài, lý do lựa chọn đề tài, mục tiêu, đối tượng, phạm vi nghiên cứu và bố cục chung của đề án.

Chương 2: Cơ sở lý thuyết. Trình bày tổng quan về bài toán SLAM, các hướng tiếp cận và nghiên cứu liên quan; đồng thời hệ thống hóa cơ sở toán học về mô hình động học robot vi sai, nguyên lý cảm biến và các bộ lọc ước lượng trạng thái như EKF và UKF.

Chương 3: Thiết kế và triển khai thuật toán. Trình bày mô hình Adaptive UKF cho bài toán sensor fusion từ encoder, IMU và từ kế; đồng thời mô tả thuật toán UKF-SLAM, cơ chế quản lý landmark, trích xuất đặc trưng LiDAR bằng PCA và liên kết dữ liệu bằng khoảng cách Mahalanobis.

Chương 4: Thiết kế hệ thống, kết quả thực nghiệm và thảo luận. Mô tả kiến trúc phần cứng robot, giao diện API dữ liệu, cấu trúc các node xử lý phần mềm trên ROS 2; đồng thời trình bày kết quả thực nghiệm thực tế, phân tích sai số quỹ đạo, chất lượng bản đồ thu được, khả năng điều hướng và hiệu năng tính toán của hệ thống.

Chương 5: Kết luận và hướng phát triển. Tổng kết lại các kết quả đạt được của đồ án, chỉ rõ những điểm hạn chế hiện tại và đề xuất các hướng nghiên cứu, cải tiến mở rộng trong tương lai.

Chương 2.

Cơ sở lý thuyết và tổng quan nghiên cứu

2.1. Bài toán SLAM và các nghiên cứu liên quan

2.1.1. Bài toán SLAM (Simultaneous Localization and Mapping) trong robot di động

a. Bài toán SLAM

SLAM (Simultaneous Localization and Mapping) là bài toán cốt lõi trong lĩnh vực robot di động, yêu cầu robot phải đồng thời ước lượng trạng thái, tức tư thế của chính nó, và xây dựng bản đồ của môi trường chưa biết dựa trên dữ liệu cảm biến thu nhận theo thời gian [5]. Về bản chất, đây là bài toán kết hợp chặt chẽ giữa định vị và lập bản đồ: để xây dựng được bản đồ tốt, robot cần biết khá chính xác mình đang ở đâu; ngược lại, để định vị tốt, robot cần một biểu diễn môi trường đủ tin cậy.

Về mặt xác suất, bài toán SLAM thường được mô tả thông qua việc ước lượng phân phối hậu nghiệm của quỹ đạo robot và bản đồ khi biết toàn bộ chuỗi điều khiển và chuỗi quan sát. Nếu ký hiệu $x_{1:t}$ là chuỗi trạng thái robot, m là bản đồ, $z_{1:t}$ là chuỗi phép đo cảm biến và $u_{1:t}$ là chuỗi lệnh điều khiển, mục tiêu của SLAM có thể được biểu diễn dưới dạng:

$$p(x_{1:t}, m \mid z_{1:t}, u_{1:t}) \quad (2.1)$$

Trong đó, $x_{1:t}$ thường bao gồm vị trí và góc hướng của robot theo thời gian; m là bản đồ môi trường, có thể được biểu diễn dưới dạng tập landmark, lưới chiếm chỗ hoặc các đặc trưng hình học; còn $z_{1:t}$ và $u_{1:t}$ lần lượt là thông tin đo lường và điều khiển thu được trong quá trình robot hoạt động.

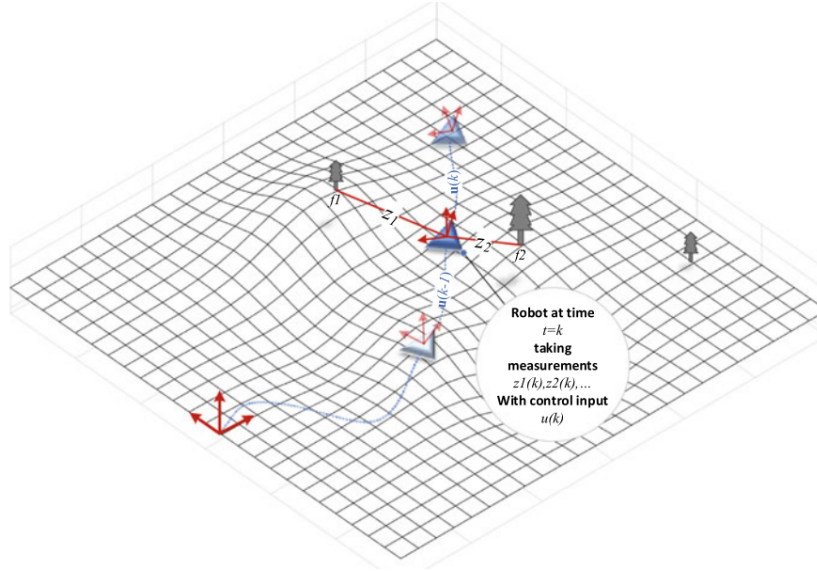
Đặt trong khuôn khổ lý thuyết ước lượng, bài toán định vị, lập bản đồ và SLAM đều có thể được xem như bài toán ước lượng trạng thái đa biến trong điều kiện đo lường có nhiễu [5]. Nếu ký hiệu x là vector trạng thái cần ước lượng và z là vector đo lường, nghiệm theo nghĩa cực đại hậu nghiệm được viết dưới dạng:

$$\hat{x}_{MAP} = \underset{x}{\operatorname{argmax}} p(x \mid z) \quad (2.2)$$

Áp dụng định lý Bayes, biểu thức trên có thể được viết lại thành:

$$\hat{x}_{MAP} = \underset{x}{\operatorname{argmax}} [p(z \mid x) p(x)] \quad (2.3)$$

trong đó $p(z | x)$ là hàm khả năng của phép đo và $p(x)$ là phân bố tiên nghiệm của trạng thái. Cách phát biểu này cho phép xây dựng bộ ước lượng theo dạng đệ quy, nghĩa là trạng thái không cần được giải lại từ đầu sau mỗi lần có quan sát mới, mà được cập nhật liên tục qua từng bước thời gian dựa trên thông tin dự đoán và thông tin đo hiện thời.



Hình 2.1. Bài toán SLAM với trạng thái robot, bản đồ, điều khiển và quan sát cảm biến

Đối với robot di động hoạt động trong không gian hai chiều, trạng thái của robot thường được mô tả bởi vector tư thế $x_r = [x_r, y_r, \phi_r]^T$, trong khi bản đồ đặc trưng gồm n landmark được biểu diễn bởi vector trạng thái bản đồ x_m . Khi đó, trạng thái tổng quát của bài toán SLAM tại thời điểm k được viết dưới dạng:

$$x(k) = \begin{bmatrix} x_r(k) \\ x_m(k) \end{bmatrix} \quad (2.4)$$

Với cách tổ chức này, bài toán SLAM không còn chỉ là định vị hay lập bản đồ riêng rẽ, mà trở thành bài toán ước lượng đồng thời hai thành phần có quan hệ phụ thuộc lẫn nhau: robot cần bản đồ để định vị, trong khi bản đồ chỉ được cập nhật chính xác khi tư thế robot được ước lượng đủ tin cậy.

Bài toán SLAM thường được phân chia thành hai hướng chính. *Full SLAM* hướng tới việc ước lượng toàn bộ quỹ đạo robot cùng với bản đồ sau khi đã có toàn bộ dữ liệu, nên thường cho kết quả chính xác và nhất quán cao hơn nhưng chi phí tính toán lớn. Trong khi đó, *Online SLAM* chỉ tập trung ước lượng trạng thái hiện tại và cập nhật bản đồ theo thời gian thực, phù hợp hơn với triển khai trên robot nhúng. Trong phạm vi đề án này, hướng tiếp cận được lựa chọn là Online SLAM để đáp ứng yêu cầu vận hành thời gian thực trên nền tảng Raspberry Pi 5.

2.1.2. Các phương pháp tiếp cận, hạn chế và hướng nghiên cứu

Các phương pháp giải quyết SLAM có thể chia thành hai nhóm lớn là phương pháp dựa trên bộ lọc (*filter-based SLAM*) và phương pháp dựa trên tối ưu hóa đồ thị (*graph-based SLAM*). Trong nhóm thứ nhất, các kỹ thuật như EKF-SLAM và UKF-SLAM trực tiếp ước lượng trạng thái robot và bản đồ bằng cách lan truyền trung bình và hiệp phương sai của trạng thái. Trong nhóm thứ hai, bài toán được mô hình hóa dưới dạng tối ưu hóa trên đồ thị ràng buộc, cho phép xử lý *loop closure* và tối ưu toàn cục hiệu quả hơn [1, 2].

Đối với nhóm phương pháp dựa trên bộ lọc, quá trình suy ra trạng thái tại mỗi thời điểm thường diễn ra qua ba bước cơ bản: dự đoán, quan sát và cập nhật. Ở bước dự đoán, bộ lọc sử dụng mô hình chuyển động và tín hiệu điều khiển để suy ra trạng thái kế tiếp. Ở bước quan sát, robot đo các đặc trưng môi trường và với các phương pháp SLAM dựa trên landmark, cần thực hiện thêm liên kết dữ liệu để xác định quan sát mới tương ứng với landmark nào đã tồn tại trong bản đồ hoặc có phải là landmark mới hay không. Cuối cùng, ở bước cập nhật, thông tin dự đoán và thông tin đo được kết hợp để tạo ra ước lượng cải thiện hơn. EKF-SLAM và UKF-SLAM đều tuân theo khung đệ quy chung này, nhưng khác nhau ở cách lan truyền phân bố qua mô hình phi tuyến [5].

EKF-SLAM, được khởi xướng bởi Smith, Self và Cheeseman, là phương pháp tiên phong trong việc mô hình hóa SLAM dưới dạng bài toán ước lượng trạng thái xác suất [9]. Đóng góp cốt lõi của hướng tiếp cận này là khái niệm *stochastic map*, trong đó trạng thái robot và tọa độ các landmark được gộp chung vào một vector trạng thái mở rộng duy nhất, còn ma trận hiệp phương sai đầy đủ mã hóa không chỉ độ bất định riêng của từng thành phần mà còn cả tương quan chéo giữa chúng. Về cơ chế, chính các khối hiệp phương sai ngoài đường chéo giữa robot và các landmark, cũng như giữa từng cặp landmark, đã mã hóa sự phụ thuộc thống kê giữa các ước lượng. Vì vậy, khi robot quan sát lại một landmark đã biết và thực hiện hiệu chỉnh, thay đổi ở một thành phần trạng thái sẽ được lan truyền qua các khối tương quan chéo này sang các thành phần liên quan, qua đó đồng thời cải thiện cả vị trí robot lẫn các landmark khác trên bản đồ [9]. Sau đó, Dissanayake và các cộng sự đã chỉ ra các tính chất hội tụ quan trọng của EKF-SLAM, qua đó củng cố nền tảng lý thuyết cho hướng tiếp cận này [3].

UKF-SLAM khắc phục một phần hạn chế của EKF-SLAM bằng cách sử dụng Unscented Transform để lan truyền trực tiếp các sigma points qua hàm phi tuyến, từ đó thường cho kết quả ước lượng tốt hơn trong các bài toán phi tuyến vừa và mạnh [6, 7]. Tuy nhiên, UKF-SLAM vẫn tồn tại một số hạn chế quan trọng khi triển khai trên robot di động thực tế. Trước hết, chi phí tính toán tăng nhanh theo kích thước trạng thái, đặc biệt khi số lượng landmark lớn; trong nhiều cấu hình, đây trở thành nút thắt đáng kể đối với khả năng vận hành thời gian thực [4]. Bên cạnh đó, hiệu năng của UKF còn phụ thuộc

vào việc lựa chọn các tham số điều chỉnh, cũng như vào giả thiết về nhiễu quá trình và nhiễu đo [6, 7].

Một vấn đề thực tế khác là đặc tính nhiễu của encoder, IMU và LiDAR thường thay đổi theo trạng thái chuyển động của robot. Khi robot tăng tốc, giảm tốc, quay đột ngột hoặc di chuyển trên bề mặt không đồng nhất, các ma trận hiệp phương sai cố định khó phản ánh đúng điều kiện vận hành. Điều này có thể làm giảm độ chính xác ước lượng và ảnh hưởng đến tính nhất quán của bộ lọc [12, 13].

Để khắc phục các hạn chế nêu trên, đồ án lựa chọn hướng tiếp cận dựa trên Adaptive UKF cho sensor fusion kết hợp với UKF-SLAM. Ý tưởng chính là điều chỉnh động các ma trận hiệp phương sai dựa trên chuỗi innovation nhằm tăng khả năng thích nghi của bộ lọc trước sự thay đổi của điều kiện nhiễu [12, 13]. Song song với đó, dữ liệu LiDAR được xử lý theo hướng trích xuất đặc trưng ổn định bằng PCA và liên kết dữ liệu bằng khoảng cách Mahalanobis nhằm nâng cao độ tin cậy trong quá trình cập nhật trạng thái. Hướng tiếp cận này nhằm tận dụng ưu điểm của UKF trong xử lý phi tuyến, đồng thời tăng khả năng thích nghi của bộ lọc trước sự thay đổi của điều kiện nhiễu và giảm nguy cơ liên kết dữ liệu sai trong quá trình SLAM.

2.1.3. Các công trình liên quan

Nền tảng lý thuyết của bài toán SLAM được đặt ra từ rất sớm khi Smith, Self và Cheeseman công bố khái niệm *stochastic map*, trong đó tư thế robot và vị trí các landmark được ước lượng đồng thời trong một vector trạng thái mở rộng đi kèm ma trận hiệp phương sai đầy đủ [9]. Điểm quan trọng trong công trình này là các khối hiệp phương sai ngoài đường chéo không chỉ lưu thông tin về độ bất định riêng lẻ, mà còn mã hóa tương quan giữa robot với landmark và giữa các landmark với nhau; chính các tương quan đó tạo cơ chế để thông tin hiệu chỉnh được lan truyền xuyên suốt toàn bộ bản đồ khi một quan sát mới xuất hiện [9]. Đây được xem là một trong những công trình mở đầu cho hướng tiếp cận *filter-based SLAM* và có ảnh hưởng sâu rộng đến các phương pháp EKF-SLAM và UKF-SLAM về sau.

Tiếp theo, Dissanayake và các cộng sự đã cung cấp phân tích toán học quan trọng về tính chất hội tụ của EKF-SLAM, cho thấy độ bất định của bản đồ có thể được giới hạn theo thời gian dưới các giả thiết phù hợp [3]. Về mặt tổng quan học thuật, Durrant-Whyte và Bailey đã hệ thống hóa tương đối đầy đủ các vấn đề cốt lõi của SLAM trong hai bài survey kinh điển, bao gồm mô hình xác suất, liên kết dữ liệu, *loop closure* và các đánh đổi giữa những hướng tiếp cận khác nhau [1, 2].

Đối với các bộ lọc phi tuyến, hướng phát triển từ Unscented Transform đến Unscented Kalman Filter do Julier, Uhlmann, Wan và van der Merwe đề xuất đã mở ra khả năng xử lý tốt hơn các bài toán có tính phi tuyến mạnh, trong đó có SLAM [6, 7].

Tuy nhiên, khi áp dụng vào SLAM, chi phí tính toán và các vấn đề liên quan đến tính nhất quán vẫn là những thách thức đáng kể, đặc biệt khi số lượng landmark tăng lớn [4].

Trong hướng nghiên cứu gắn với dữ liệu LiDAR, nhiều công trình cụ thể đã cho thấy hiệu quả rõ rệt của việc khai thác đặc trưng hình học thay vì chỉ sử dụng trực tiếp toàn bộ scan thô. Chẳng hạn, Zhang và Singh trong LOAM đã sử dụng các đặc trưng cạnh và phẳng để nâng cao độ chính xác của LiDAR odometry và mapping thời gian thực [10]. Ở hướng mở rộng hơn, Pan và các cộng sự trong MULS khai thác nhiều loại đặc trưng hình học từ point cloud nhằm tăng độ ổn định của quá trình định vị và lập bản đồ [11]. Đối với bài toán thích nghi nhiều trong sensor fusion, các cơ chế điều chỉnh động hiệp phương sai dựa trên innovation của Mohamed và Schwarz, cũng như Adaptive UKF đa cảm biến của Zhu và các cộng sự, cung cấp nền tảng trực tiếp cho hướng tiếp cận adaptive filtering trong đề án này [12, 13]. Trong bối cảnh đó, PCA và khoảng cách Mahalanobis là hai công cụ phù hợp để xây dựng một quy trình trích xuất đặc trưng và so khớp đủ gọn cho nền tảng nhúng nhưng vẫn giữ được cơ sở toán học rõ ràng.

2.2. Cơ sở lý thuyết

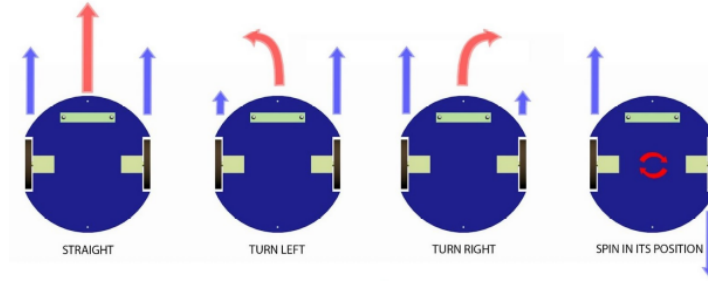
2.2.1. Động học robot di động bánh xe vi sai

Robot bánh xe vi sai (*Differential Drive Robot*) gồm hai bánh chủ động đặt song song và một hoặc hai bánh tự do (*caster*) để duy trì thăng bằng. Nhờ cấu trúc cơ khí này, robot có khả năng di chuyển thẳng về phía trước, lùi về phía sau, quay tại chỗ và thực hiện các quỹ đạo cong với bán kính tùy ý. Tuy nhiên, robot chịu ràng buộc *non-holonomic*, nghĩa là không thể dịch chuyển tức thời theo phương ngang, tức hướng vuông góc với thân robot.

a. Cơ chế chuyển động của robot bánh xe vi sai

Cơ chế chuyển động của robot bánh xe vi sai hoàn toàn dựa vào sự khác biệt vận tốc giữa hai bánh chủ động. Cụ thể:

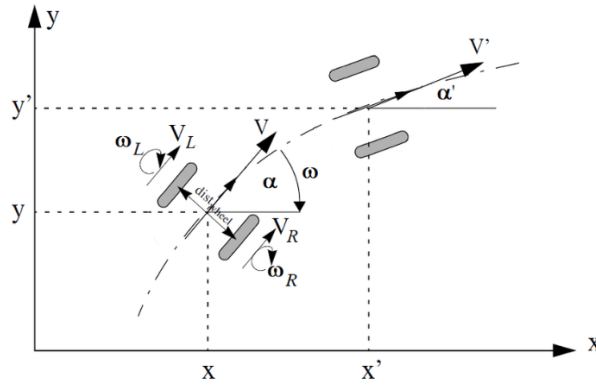
- Khi hai bánh quay với cùng vận tốc và cùng chiều, robot di chuyển thẳng.
- Khi hai bánh quay với vận tốc khác nhau, robot di chuyển theo quỹ đạo cong.
- Khi hai bánh quay ngược chiều với cùng tốc độ, robot quay tại chỗ quanh trục thẳng đứng đi qua tâm robot.



Hình 2.2. Cơ cấu di chuyển robot hai bánh vi sai

Sự kết hợp linh hoạt giữa hai bánh chủ động mang lại khả năng cơ động cao, nhưng cũng dẫn đến tính chất *non-holonomic* của hệ thống [5].

b. Động học thuận (Forward Kinematics)



Hình 2.3. Mô hình động học

Động học thuận mô tả mối quan hệ giữa vận tốc hai bánh xe với vận tốc tổng thể của robot. Gọi v_r và v_l lần lượt là vận tốc tuyến tính của bánh phải và bánh trái, còn d là khoảng cách giữa hai bánh chủ động. Các quan hệ này là mô hình động học cơ bản của robot vi sai. Khi đó:

$$v = \frac{v_r + v_l}{2} \quad (2.5)$$

$$\omega = \frac{v_r - v_l}{d} \quad (2.6)$$

Từ vận tốc tịnh tiến v và vận tốc góc ω , trạng thái của robot trong mặt phẳng có thể được cập nhật theo mô hình odometry. Với trạng thái $x_k = [x_k, y_k, \theta_k]^T$ và khoảng thời gian lấy mẫu Δt , công thức cập nhật vị trí được xấp xỉ như sau:

$$\begin{cases} x_{k+1} = x_k + v_k \cos(\theta_k) \Delta t, \\ y_{k+1} = y_k + v_k \sin(\theta_k) \Delta t, \\ \theta_{k+1} = \theta_k + \omega_k \Delta t. \end{cases} \quad (2.7)$$

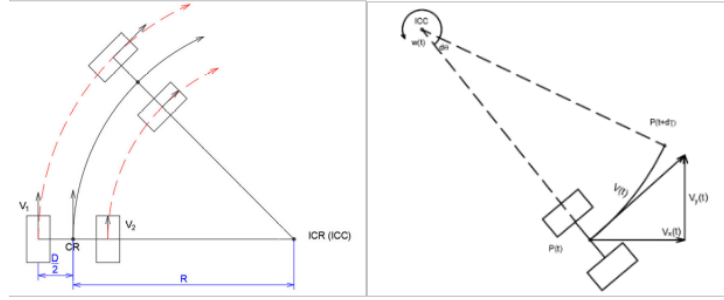
c. Động học nghịch (Inverse Kinematics)

Động học nghịch cho phép tính toán vận tốc cần thiết của hai bánh xe khi biết trước vận tốc mong muốn của robot (v, ω) :

$$v_r = v + \frac{\omega d}{2}, \quad v_l = v - \frac{\omega d}{2} \quad (2.8)$$

Mô hình này được sử dụng chủ yếu trong tầng điều khiển vận tốc để chuyển đổi lệnh điều khiển thành tín hiệu điều khiển hai động cơ.

d. Động học trên tâm quay tức thời ICC



Hình 2.4. Mô hình robot di chuyển quanh ICC

Khi hai bánh quay với vận tốc khác nhau, robot sẽ chuyển động quanh một điểm gọi là tâm quay tức thời (*Instantaneous Center of Curvature* – ICC). Khoảng cách từ tâm robot đến ICC được xác định bởi công thức:

$$R = \frac{v}{\omega} = \frac{d(v_r + v_l)}{2(v_r - v_l)} \quad (2.9)$$

Với trạng thái robot tại thời điểm k là (x_k, y_k, θ_k) , sau khoảng thời gian lấy mẫu Δt , vị trí và góc hướng tiếp theo được tính theo:

$$\begin{cases} x_{k+1} = x_k + R [\sin(\theta_k + \omega \Delta t) - \sin \theta_k], \\ y_{k+1} = y_k - R [\cos(\theta_k + \omega \Delta t) - \cos \theta_k], \\ \theta_{k+1} = \theta_k + \omega \Delta t. \end{cases} \quad (2.10)$$

Mô hình ICC cho phép mô tả chính xác hơn chuyển động cong của robot trong khoảng thời gian ngắn. Đây là công cụ hữu ích để phân tích quỹ đạo chuyển động của robot bánh xe vi sai.

e. Odometry

Odometry là phương pháp ước lượng trạng thái chuyển động của robot, bao gồm vị trí và hướng, bằng cách tích lũy các thay đổi chuyển động theo thời gian dựa trên dữ

liệu từ các cảm biến đo lường. Các cảm biến này có thể bao gồm encoder bánh xe, cảm biến quán tính (IMU) hoặc các nguồn đo chuyển động khác.

Về bản chất, odometry là dạng đơn giản nhất của bài toán định vị, còn được gọi là *dead-reckoning*. Phương pháp này thực hiện việc theo vết chuyển động của robot bằng cách cập nhật liên tục trạng thái hiện tại dựa trên một trạng thái ban đầu đã biết. Nói cách khác, odometry không xác định vị trí tuyệt đối trong môi trường, mà chỉ ước lượng vị trí tương đối thông qua quá trình tích lũy chuyển động:

$$x_{t+1} = x_t + \Delta x_t \quad (2.11)$$

Trong đó:

- $x_t = (x_t, y_t, \theta_t)$ là trạng thái của robot tại thời điểm t .
- Δx_t là sự thay đổi trạng thái trong khoảng thời gian Δt .

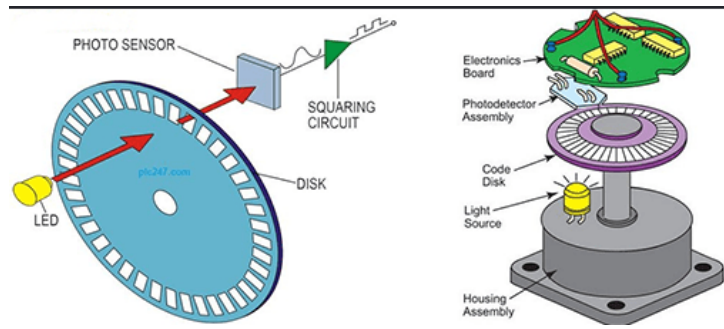
Trong robot bánh xe vi sai, odometry thường được xây dựng dựa trên mô hình động học thuận kết hợp với dữ liệu đo vận tốc hoặc quãng đường của từng bánh xe [5]. Odometry có ưu điểm là đơn giản, chi phí thấp và cung cấp thông tin trạng thái với tần số cao, phù hợp cho việc ước lượng chuyển động ngắn hạn. Tuy nhiên, do phụ thuộc vào quá trình tích lũy theo thời gian, phương pháp này dễ bị ảnh hưởng bởi sai số cảm biến, sai số mô hình và hiện tượng trượt bánh, dẫn đến sai số tích lũy ngày càng tăng [5].

2.2.2. Các loại cảm biến sử dụng trong hệ thống

a. Encoder

Encoder là cảm biến dùng để đo lường chuyển động của cơ cấu cơ khí và chuyển đổi chuyển động đó thành tín hiệu điện. Tùy theo dạng chuyển động cần đo, encoder thường được chia thành ba loại chính gồm encoder tuyến tính (*linear encoder*), encoder quay (*rotary encoder*) và encoder góc (*angle encoder*) [5]. Trong đó, encoder quay là loại được sử dụng phổ biến nhất trong robot di động.

Trong các hệ robot bánh xe, encoder thường được gắn trực tiếp vào trục động cơ hoặc trục bánh xe để đo chuyển động quay. Đối với encoder gia tăng (*incremental encoder*), một đĩa quay chứa các khe hoặc vạch chia được chiếu sáng bởi nguồn phát quang. Khi đĩa quay, tín hiệu ánh sáng đi qua các khe được cảm biến quang thu nhận và chuyển thành chuỗi xung điện, từ đó cho phép xác định góc quay của trục [5].



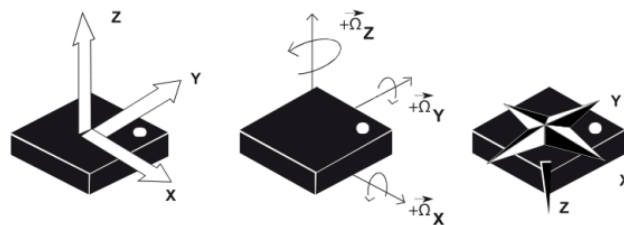
Hình 2.5. Cấu tạo encoder

Trong robot di động, encoder đóng vai trò là nguồn dữ liệu quan trọng cho việc xây dựng odometry. Thông qua việc đo chuyển động của từng bánh xe, encoder cung cấp thông tin đầu vào để ước lượng vận tốc và cập nhật trạng thái của robot theo các quan hệ động học thuận ở các công thức (2.5) và (2.6), mô hình cập nhật odometry, cũng như quan hệ hình học với tâm quay tức thời ICC ở công thức (2.9). Phương pháp này còn được gọi là *wheel odometry*. Tuy nhiên, do encoder không đo trực tiếp vị trí tuyệt đối, kết quả odometry vẫn có thể bị ảnh hưởng bởi hiện tượng trượt bánh, sai số cơ khí và sai số tích lũy theo thời gian.

b. Cảm biến đo lường quán tính (IMU)

Cảm biến đo lường quán tính (*Inertial Measurement Unit – IMU*) là thiết bị đo chuyển động và định hướng của robot thông qua sự kết hợp của gia tốc kế (*accelerometer*) và con quay hồi chuyển (*gyroscope*); ngoài ra, một số cấu hình còn tích hợp thêm từ kế (*magnetometer*) [5].

- *Gia tốc kế*: đo gia tốc tuyến tính dọc thân robot, hỗ trợ theo dõi các biến thiên ngắn hạn của vận tốc tịnh tiến v khi robot tăng tốc hoặc giảm tốc.
- *Con quay hồi chuyển*: đo vận tốc góc quanh trục vuông góc với mặt phẳng chuyển động, cung cấp thông tin về vận tốc góc ω giúp cập nhật và ổn định góc hướng θ của robot.



Hình 2.6. Cảm biến IMU

Dữ liệu tần số cao từ IMU cho phép phản ánh nhanh chóng các thay đổi tức thời về tư thế, đặc biệt hiệu quả trong các pha chuyển động nhanh hoặc khi xảy ra trượt bánh. Tuy nhiên, do chịu ảnh hưởng lớn bởi sai lệch cấu trúc (*bias*), nhiễu ngẫu nhiên (*noise*) và hiện tượng trôi (*drift*), sai số ước lượng từ IMU sẽ tích lũy và tăng nhanh theo thời gian nếu chỉ tích phân độc lập [5].

Trong đồ án này, IMU không đóng vai trò định vị độc lập mà được sử dụng như một nguồn thông tin bổ trợ động học cho encoder. Dữ liệu vận tốc góc ω và gia tốc từ IMU được hợp nhất thông qua bộ lọc Adaptive UKF nhằm giảm thiểu sai số tích lũy, từ đó nâng cao độ chính xác và tính ổn định cho góc hướng θ của robot vì sai trước khi đưa vào tầng xử lý SLAM.

c. LiDAR

Cảm biến laser là một loại cảm biến đo khoảng cách sử dụng nguyên lý phản xạ ánh sáng. Thiết bị phát ra một chùm tia laser hướng về phía vật thể và phân tích tín hiệu phản xạ thu được để xác định khoảng cách đến mục tiêu [5]. Các hệ thống đo khoảng cách bằng laser thường sử dụng hai phương pháp chính là đo thời gian bay (*Time-of-Flight – TOF*) và đo dịch pha (*phase-shift*). Trong phương pháp TOF, một xung laser được phát ra và thời gian để tín hiệu phản xạ quay trở lại được đo nhằm suy ra khoảng cách đến vật thể.

LiDAR (*Light Detection And Ranging*) là một dạng mở rộng của cảm biến laser, cho phép quét môi trường xung quanh và thu thập dữ liệu khoảng cách theo nhiều hướng. Nhờ khả năng tạo ra bản đồ khoảng cách chi tiết, LiDAR được sử dụng rộng rãi trong robot di động cho các bài toán như phát hiện vật cản, tránh va chạm, xây dựng bản đồ và theo dõi chuyển động trong không gian [5].

Trong robot di động, LiDAR hai chiều (2D LiDAR) là loại được sử dụng phổ biến nhất. Thiết bị này thực hiện quét trong một mặt phẳng ngang bằng cách quay tia laser xung quanh trục thẳng đứng, từ đó thu được tập hợp các điểm khoảng cách theo góc quét. Dữ liệu thu được thường ở dạng các cặp (r_i, θ_i) , trong đó r_i là khoảng cách đo được và θ_i là góc quét tương ứng. Từ dữ liệu này, có thể chuyển đổi sang hệ tọa độ Descartes theo công thức:

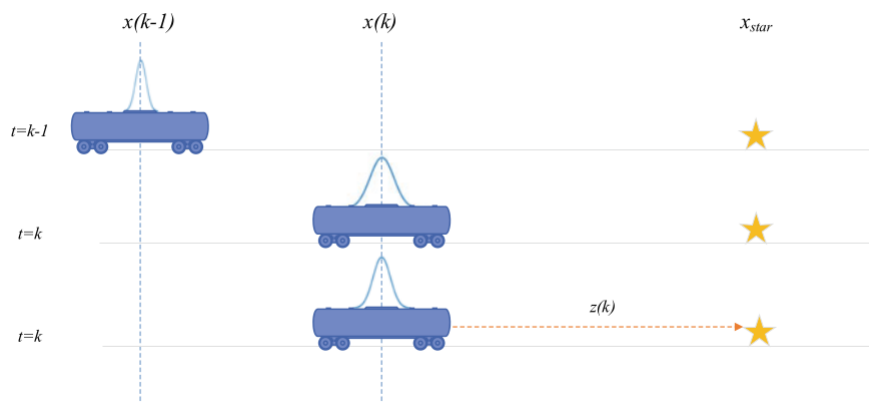
$$x_i = r_i \cos(\theta_i), \quad y_i = r_i \sin(\theta_i) \quad (2.12)$$

2.2.3. Kalman Filter và các mở rộng cho hệ phi tuyến

Kalman Filter (KF) là một thuật toán ước lượng trạng thái đệ quy dùng để suy ra trạng thái bên trong của một hệ động từ các phép đo bị nhiễu và mô hình động học của hệ. Ý tưởng cốt lõi của bộ lọc là kết hợp thông tin dự đoán từ mô hình với thông tin quan sát từ cảm biến để thu được ước lượng tốt hơn so với khi chỉ sử dụng riêng lẻ từng

nguồn thông tin [5]. Trong trường hợp hệ tuyến tính với nhiễu Gaussian, KF cho phép lan truyền đồng thời giá trị trung bình và hiệp phương sai của trạng thái theo thời gian, từ đó tạo ra một cơ chế ước lượng hiệu quả và nhất quán.

Một cách trực quan, có thể hình dung Kalman Filter qua bài toán robot chỉ chuyển động trên một trục x như minh họa hình 2.2. Tại thời điểm $t = k - 1$, robot ở vị trí thực $x(k - 1)$ nhưng không biết chính xác giá trị này; thay vào đó, robot chỉ có ước lượng $\hat{x}(k - 1 | k - 1)$ cùng với một độ bất định tương ứng. Khi robot thực hiện lệnh điều khiển $u(k)$ để tiến tới thời điểm $t = k$, mô hình chuyển động cho phép dự đoán vị trí mới dưới dạng $\hat{x}(k | k - 1)$. Tuy nhiên, do ảnh hưởng của sai số mô hình và nhiễu quá trình, độ bất định của ước lượng dự đoán thường tăng lên.



Hình 2.7. Minh họa ước lượng Kalman Filter trong định vị một chiều

Ở bước tiếp theo, robot quan sát một mốc chuẩn có vị trí đã biết trên bản đồ, ký hiệu là x_{star} , và cảm biến đo được khoảng cách $z(k)$ từ robot đến mốc này. Phép đo mới cung cấp thêm thông tin về vị trí hiện tại của robot tại thời điểm $t = k$. Khi kết hợp ước lượng dự đoán với phép đo cảm biến, Kalman Filter sẽ tự động xác định mức độ tin cậy tương đối của từng nguồn thông tin thông qua các ma trận hiệp phương sai. Nếu cảm biến đáng tin cậy hơn, ước lượng sau cập nhật sẽ nghiêng nhiều hơn về phía thông tin đo; ngược lại, nếu phép đo nhiễu lớn, bộ lọc sẽ ưu tiên mô hình dự đoán.

Sau bước hiệu chỉnh, robot thu được ước lượng cải thiện $\hat{x}(k | k)$ với độ bất định nhỏ hơn so với trước khi cập nhật. Chính cơ chế lặp lại liên tục giữa hai bước dự đoán và hiệu chỉnh này làm cho Kalman Filter trở thành công cụ đặc biệt phù hợp cho các bài toán định vị robot theo thời gian thực: ước lượng tại thời điểm hiện tại vừa tận dụng được xu hướng động học của hệ, vừa liên tục được sửa sai nhờ dữ liệu cảm biến mới [5].

a. Nguyên lý lọc Kalman

Đối với hệ rời rạc, bài toán lọc thường được mô tả dưới dạng:

$$x_k = A_{k-1}x_{k-1} + B_{k-1}u_{k-1} + w_{k-1} \quad (2.13)$$

$$z_k = H_k x_k + v_k \quad (2.14)$$

trong đó w_{k-1} và v_k lần lượt là nhiễu quá trình và nhiễu đo, thường được giả sử có phân phối Gaussian với kỳ vọng bằng 0 và hiệp phương sai tương ứng là Q_k và R_k .

KF hoạt động theo cấu trúc dự đoán – hiệu chỉnh. Trong bước dự đoán, trạng thái và hiệp phương sai được lan truyền theo mô hình hệ:

$$\hat{x}_k^- = A_{k-1}\hat{x}_{k-1} + B_{k-1}u_{k-1} \quad (2.15)$$

$$P_k^- = A_{k-1}P_{k-1}A_{k-1}^T + Q_k \quad (2.16)$$

Trong bước cập nhật, khi có phép đo mới, gain Kalman được xác định bởi:

$$K_k = P_k^- H_k^T (H_k P_k^- H_k^T + R_k)^{-1} \quad (2.17)$$

Trạng thái và hiệp phương sai sau hiệu chỉnh được cập nhật theo:

$$\hat{x}_k = \hat{x}_k^- + K_k (z_k - H_k \hat{x}_k^-) \quad (2.18)$$

$$P_k = (I - K_k H_k) P_k^- \quad (2.19)$$

Trong đó, K_k là ma trận gain Kalman, đóng vai trò cân bằng giữa độ tin cậy của mô hình dự đoán và dữ liệu đo. Kalman Filter có thể được chứng minh là bộ ước lượng tuyến tính tối ưu theo tiêu chuẩn *minimum mean square error* (MMSE), tức là cực tiểu hóa sai số bình phương trung bình của ước lượng trạng thái [8].

b. Hạn chế của Kalman Filter

KF chỉ áp dụng trực tiếp cho hệ tuyến tính. Tuy nhiên, trong robotics, đặc biệt là bài toán định vị và SLAM, mô hình động học và mô hình đo thường có dạng phi tuyến:

$$x_k = f(x_{k-1}, u_{k-1}) + w_{k-1} \quad (2.20)$$

$$z_k = h(x_k) + v_k \quad (2.21)$$

Do đó, cần mở rộng KF để xử lý các hệ phi tuyến.

c. Extended Kalman Filter (EKF)

Extended Kalman Filter (EKF) là mở rộng của KF cho hệ phi tuyến bằng cách xấp xỉ tuyến tính hóa bậc nhất quanh điểm làm việc hiện tại. Ý tưởng cốt lõi là thay các hàm

phi tuyến bởi khai triển Taylor bậc nhất thông qua các ma trận Jacobian. Khi đó, Jacobian của mô hình trạng thái và mô hình đo được xác định bởi:

$$F_{k-1} = \left. \frac{\partial f}{\partial x} \right|_{\hat{x}_{k-1}, u_{k-1}}, \quad H_k = \left. \frac{\partial h}{\partial x} \right|_{\hat{x}_k} \quad (2.22)$$

Dựa trên các ma trận Jacobian này, bước dự đoán của EKF được viết như sau:

$$\hat{x}_k^- = f(\hat{x}_{k-1}, u_{k-1}) \quad (2.23)$$

$$P_k^- = F_{k-1} P_{k-1} F_{k-1}^T + Q_k \quad (2.24)$$

Bước cập nhật của EKF vẫn giữ cấu trúc tương tự KF, công thức (21-23):

EKF là phương pháp rất phổ biến trong robotics nhờ dễ triển khai, chi phí tính toán thấp và phù hợp với các hệ thời gian thực. Tuy nhiên, EKF cũng tồn tại một số hạn chế quan trọng: chỉ dựa trên xấp xỉ Taylor bậc nhất nên sai số có thể lớn khi hệ phi tuyến mạnh; phụ thuộc vào việc xây dựng Jacobian nên khó áp dụng cho các mô hình phức tạp; và có thể mất ổn định hoặc phân kỳ nếu mô hình không chính xác hoặc nhiễu lớn. Theo Wan và van der Merwe, EKF chỉ đạt độ chính xác bậc nhất trong việc lan truyền phân bố Gaussian qua hệ phi tuyến.

2.2.4. UKF và Unscented Transform

Mặc dù EKF là mở rộng phổ biến của Kalman Filter cho hệ phi tuyến, phương pháp này vẫn bị giới hạn bởi bước tuyến tính hóa bậc nhất quanh điểm làm việc hiện tại. Khi tính phi tuyến của hệ mạnh hoặc mô hình quan sát có độ cong lớn, sai số tuyến tính hóa có thể làm suy giảm chất lượng ước lượng. Từ hạn chế đó, Unscented Transform và Unscented Kalman Filter được đề xuất như một hướng tiếp cận thay thế nhằm lan truyền phân bố xác suất qua mô hình phi tuyến chính xác hơn mà không cần tính Jacobian.

Để khắc phục hạn chế của EKF trong việc xấp xỉ tuyến tính hóa, Unscented Kalman Filter (UKF) được đề xuất dựa trên ý tưởng lan truyền trực tiếp phân phối xác suất qua hàm phi tuyến mà không cần tuyến tính hóa. Khác với EKF, UKF không sử dụng khai triển Taylor mà sử dụng Unscented Transform (UT) để xấp xỉ phân phối của biến ngẫu nhiên sau khi đi qua hàm phi tuyến [6, 7].

a. Unscented Transform (UT)

Giả sử biến ngẫu nhiên $x \in \mathbf{R}^L$ có trung bình $\bar{x}$ và hiệp phương sai P_x , cần xác định phân phối của biến:

$$y = f(x) \quad (2.25)$$

Thay vì tuyến tính hóa $f(\cdot)$, UT sử dụng một tập các điểm xác định gọi là *sigma points* để biểu diễn phân phối của x . Các tham số của Unscented Transform được xác định bởi

bộ (α, β, κ) , trong đó α điều chỉnh độ phân tán của sigma points quanh giá trị trung bình, β phản ánh thông tin tiên nghiệm về phân phối của biến ngẫu nhiên, còn κ là tham số tỉ lệ phụ. Từ đó, tham số tỉ lệ λ được tính bởi:

$$\lambda = \alpha^2(L + \kappa) - L \quad (2.26)$$

Tập sigma points được xây dựng như sau:

$$X_0 = \bar{x} \quad (2.27)$$

$$X_i = \bar{x} + \left(\sqrt{(L + \lambda)P_x} \right)_i, \quad i = 1, \dots, L \quad (2.28)$$

$$X_{i+L} = \bar{x} - \left(\sqrt{(L + \lambda)P_x} \right)_i, \quad i = 1, \dots, L \quad (2.29)$$

trong đó $\sqrt{(L + \lambda)P_x}$ thường được tính thông qua phân rã Cholesky, và ký hiệu $(\cdot)_i$ chỉ cột thứ i của ma trận căn. Các trọng số trung bình và trọng số hiệp phương sai được xác định bởi:

$$W_0^{(m)} = \frac{\lambda}{L + \lambda} \quad (2.30)$$

$$W_0^{(c)} = \frac{\lambda}{L + \lambda} + (1 - \alpha^2 + \beta) \quad (2.31)$$

$$W_i^{(m)} = W_i^{(c)} = \frac{1}{2(L + \lambda)}, \quad i = 1, \dots, 2L. \quad (2.32)$$

Sau đó, các sigma points được lan truyền qua hàm phi tuyến:

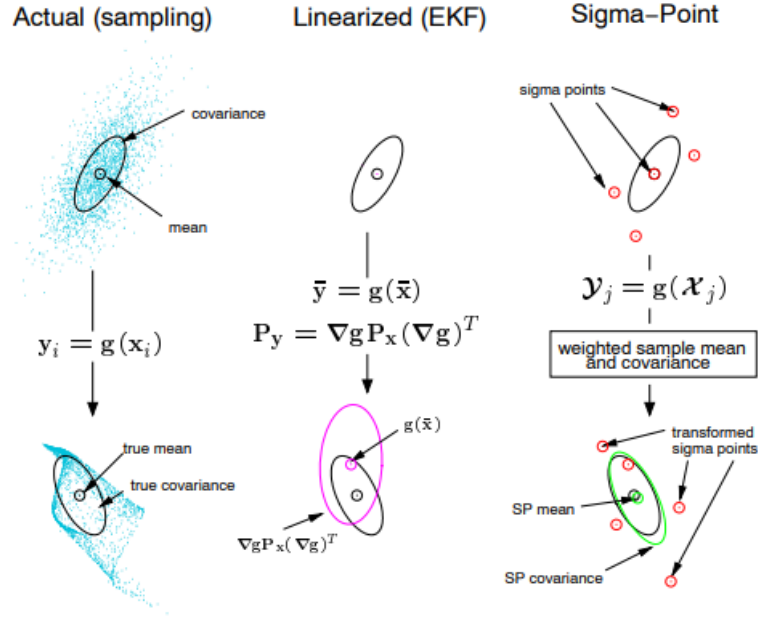
$$Y_i = f(X_i) \quad (2.33)$$

Trung bình và hiệp phương sai của biến sau biến đổi được xấp xỉ theo công thức:

$$\bar{y} = \sum_{i=0}^{2L} W_i^{(m)} Y_i \quad (2.34)$$

$$P_y = \sum_{i=0}^{2L} W_i^{(c)} (Y_i - \bar{y})(Y_i - \bar{y})^T \quad (2.35)$$

Các trọng số $W_i^{(m)}$ và $W_i^{(c)}$ được chọn sao cho bảo toàn trung bình và hiệp phương sai ban đầu. Theo Wan và van der Merwe, UT có thể xấp xỉ trung bình và hiệp phương sai chính xác đến bậc hai đối với các phi tuyến tổng quát, và trong nhiều trường hợp Gaussian có thể đạt độ chính xác cao hơn EKF vốn chỉ dựa trên xấp xỉ bậc nhất [6, 7].



Hình 2.8. So sánh tuyến tính hóa EKF và Unscented Transform với sigma points

Hình 2.8 cho thấy khác biệt cốt lõi giữa ba cách tiếp cận. Nếu dùng lấy mẫu dày đặc, phân bố sau biến đổi có thể được phản ánh khá sát nhưng chi phí tính toán lớn. EKF chỉ tuyến tính hóa quanh giá trị trung bình nên có thể làm sai lệch trung bình và hiệp phương sai khi hàm phi tuyến mạnh. Trong khi đó, UT sử dụng một số lượng nhỏ sigma points được chọn có chủ đích để lan truyền trực tiếp qua hàm phi tuyến, sau đó tái tạo lại trung bình và hiệp phương sai của đầu ra. Nhờ vậy, UT thường đạt được sự cân bằng tốt giữa độ chính xác và chi phí tính toán trong các bài toán ước lượng phi tuyến [6, 7].

Trong thực tế, β thường được chọn bằng 2 đối với phân phối Gaussian, còn α và κ được hiệu chỉnh theo mức phân tán mong muốn của sigma points [6].

b. Unscented Kalman Filter (UKF)

UKF áp dụng UT vào bài toán lọc trạng thái cho hệ phi tuyến:

$$x_k = f(x_{k-1}, u_{k-1}, w_{k-1}) \quad (2.36)$$

$$z_k = h(x_k, v_k) \quad (2.37)$$

Tại mỗi bước thời gian, từ ước lượng $\hat{x}_{k-1}$ và hiệp phương sai P_{k-1} , bộ lọc sinh ra $2L+1$ sigma points. Ký hiệu $X_{k-1}^{(i)}$ là sigma point thứ i tại thời điểm $k-1$. Các sigma points này được lan truyền qua mô hình động học để thu được các điểm dự đoán:

$$X_{k|k-1}^{(i)} = f(X_{k-1}^{(i)}, u_{k-1}) \quad (2.38)$$

Từ đó, trạng thái dự đoán và hiệp phương sai dự đoán được tính bởi:

$$\hat{x}_k^- = \sum_{i=0}^{2L} W_i^{(m)} X_{k|k-1}^{(i)} \quad (2.39)$$

$$P_k^- = \sum_{i=0}^{2L} W_i^{(c)} \left(X_{k|k-1}^{(i)} - \hat{x}_k^- \right) \left(X_{k|k-1}^{(i)} - \hat{x}_k^- \right)^T + Q_k \quad (2.40)$$

Tiếp theo, các sigma points dự đoán được lan truyền qua mô hình quan sát:

$$Z_k^{(i)} = h \left(X_{k|k-1}^{(i)} \right) \quad (2.41)$$

Trung bình phép đo dự đoán, hiệp phương sai đo và hiệp phương sai chéo được xác định theo:

$$\hat{z}_k = \sum_{i=0}^{2L} W_i^{(m)} Z_k^{(i)} \quad (2.42)$$

$$P_{zz} = \sum_{i=0}^{2L} W_i^{(c)} \left(Z_k^{(i)} - \hat{z}_k \right) \left(Z_k^{(i)} - \hat{z}_k \right)^T + R_k \quad (2.43)$$

$$P_{xz} = \sum_{i=0}^{2L} W_i^{(c)} \left(X_{k|k-1}^{(i)} - \hat{x}_k^- \right) \left(Z_k^{(i)} - \hat{z}_k \right)^T \quad (2.44)$$

Từ đó, gain Kalman và bước cập nhật của UKF được viết:

$$K_k = P_{xz} P_{zz}^{-1} \quad (2.45)$$

$$\hat{x}_k = \hat{x}_k^- + K_k (z_k - \hat{z}_k) \quad (2.46)$$

$$P_k = P_k^- - K_k P_{zz} K_k^T \quad (2.47)$$

2.2.5. Biểu diễn bản đồ trong SLAM

Trong bài toán SLAM, bản đồ là biểu diễn toán học của môi trường để robot có thể gắn các quan sát cảm biến với không gian làm việc và khai thác chúng cho định vị. Theo tài liệu của Herath và St-Onge, tùy theo loại cảm biến, cấu trúc môi trường và mục tiêu bài toán, bản đồ trong robot di động có thể được biểu diễn dưới nhiều dạng khác nhau như bản đồ lưới chiếm chỗ (*occupancy grid map*), bản đồ đặc trưng hay landmark map và bản đồ topo [5].

a. Bản đồ lưới chiếm chỗ

Bản đồ lưới chiếm chỗ biểu diễn môi trường dưới dạng các ô rời rạc, trong đó mỗi ô mang thông tin về xác suất bị chiếm dụng, trống hoặc chưa biết. Cách biểu diễn này phù hợp với dữ liệu đo khoảng cách từ LiDAR vì cho phép hợp nhất trực tiếp các tia quét

vào không gian hai chiều và tạo ra một bản đồ trực quan của môi trường trong nhà. Ưu điểm chính của occupancy grid là mô tả tốt các vật cản có hình dạng bất kỳ và không đòi hỏi phải xác định trước các đặc trưng hình học nổi bật. Tuy nhiên, khi tăng độ phân giải hoặc mở rộng khu vực làm việc, chi phí bộ nhớ và tính toán cũng tăng lên đáng kể [5].

b. Bản đồ đặc trưng (landmark map)

Trong bản đồ đặc trưng, môi trường được mô tả thông qua một tập các phần tử đại diện như điểm mốc, góc, cạnh hoặc đoạn thẳng có thể được phát hiện lặp lại qua nhiều lần quan sát. Mỗi landmark thường được gắn với tọa độ và độ bất định của nó trong hệ quy chiếu bản đồ. Kiểu biểu diễn này đặc biệt phù hợp với các phương pháp SLAM dựa trên bộ lọc như EKF-SLAM hoặc UKF-SLAM, bởi trạng thái của robot và các landmark có thể được ghép chung trong một vector trạng thái mở rộng để đồng thời ước lượng và cập nhật tương quan giữa chúng [5]. Nhược điểm của cách tiếp cận này là chất lượng bản đồ phụ thuộc mạnh vào bước trích xuất đặc trưng và liên kết dữ liệu; nếu landmark không ổn định hoặc bị ghép nhầm, sai số có thể lan truyền trực tiếp vào toàn bộ bộ lọc.

c. Ý nghĩa đối với đồ án

Đối với hệ robot di động trong nhà sử dụng LiDAR 2D, encoder và IMU, bản đồ cần vừa đủ giàu thông tin để hỗ trợ định vị, vừa đủ gọn để duy trì khả năng xử lý thời gian thực trên nền tảng nhúng. Vì vậy, trong phạm vi đồ án này, hướng biểu diễn bản đồ bằng landmark là phù hợp hơn với kiến trúc UKF-SLAM đã lựa chọn. Các landmark được hình thành từ những đặc trưng hình học ổn định trích xuất từ dữ liệu LiDAR, sau đó được quản lý trong vector trạng thái mở rộng và cập nhật thông qua cơ chế liên kết dữ liệu. Cách biểu diễn này tạo nền tảng cho các nội dung ở Chương 3 như state augmentation, quản lý landmark, trích xuất đặc trưng bằng PCA và so khớp bằng khoảng cách Mahalanobis.

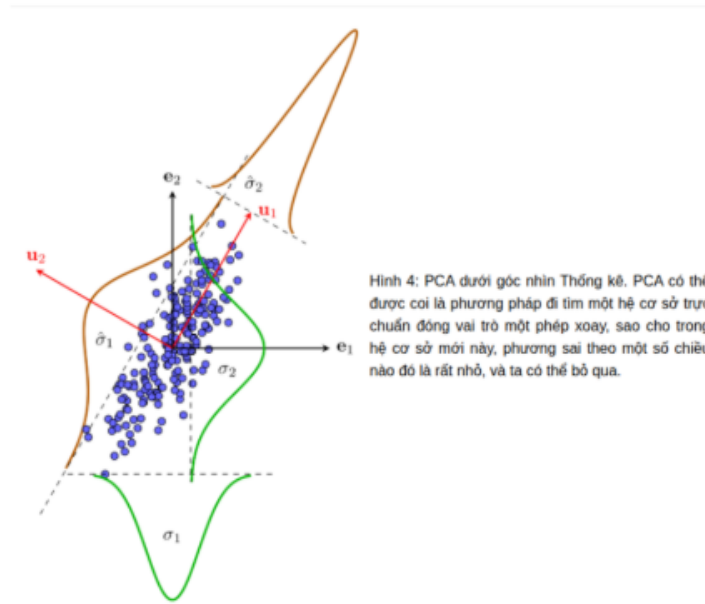
2.2.6. UKF-SLAM, PCA và khoảng cách Mahalanobis

a. Khái quát về UKF-SLAM

Về nguyên lý hoạt động, UKF-SLAM kế thừa chu trình cơ bản của các bộ lọc Bayes đệ quy gồm ba bước: dự đoán, quan sát và cập nhật. Ở bước dự đoán, bộ lọc sử dụng mô hình chuyển động và tín hiệu điều khiển để suy ra trạng thái kế tiếp. Ở bước quan sát, robot đo các đặc trưng môi trường và thực hiện liên kết dữ liệu để xác định quan sát mới tương ứng với landmark nào đã tồn tại trong bản đồ hoặc có phải là landmark mới hay không. Cuối cùng, ở bước cập nhật, thông tin dự đoán và thông tin đo được kết hợp để tạo ra ước lượng cải thiện hơn. Điểm khác biệt của UKF-SLAM so với EKF-SLAM là phân bố xác suất được lan truyền qua mô hình phi tuyến bằng sigma points của Unscented Transform thay vì dựa trên tuyến tính hóa Jacobian [6, 7, 5].

Từ đó có thể hiểu UKF-SLAM là mở rộng trực tiếp của Unscented Kalman Filter cho trường hợp trạng thái bao gồm cả robot và bản đồ. Vector trạng thái tổng hợp tại thời điểm k được mở rộng liên tục khi phát hiện thêm các mốc đặc trưng mới (*landmark*) trong môi trường. Ma trận hiệp phương sai của hệ thống không chỉ lưu trữ độ bất định của tư thế robot và vị trí các landmark, mà còn duy trì ma trận hiệp phương sai chéo (cross-covariance) giữa chúng. Đây là yếu tố cốt lõi giúp thông tin sửa sai từ phép đo landmark có thể lan truyền ngược lại để hiệu chỉnh tư thế robot, duy trì tính nhất quán toàn cục cho bản đồ.

b. Phân tích thành phần chính (PCA)



Hình 2.9. Minh họa thuật toán Principal Component Analysis.

Phân tích thành phần chính (*Principal Component Analysis* – PCA) là một phương pháp thống kê dùng để xác định các hướng có phương sai lớn nhất trong tập dữ liệu đa chiều, từ đó biểu diễn lại dữ liệu trong một hệ cơ sở mới gọn hơn mà vẫn bảo toàn phần lớn thông tin cấu trúc. Về bản chất, PCA thực hiện phép xoay hệ tọa độ sao cho trục đầu tiên (PC1) trùng với hướng có phương sai lớn nhất, còn trục thứ hai (PC2) vuông góc với trục thứ nhất và nằm theo hướng có phương sai lớn thứ hai. Trong hệ cơ sở mới này, nếu phương sai theo một số chiều rất nhỏ thì các chiều tương ứng có thể được lược bỏ mà không làm mất đáng kể thông tin của tập dữ liệu gốc.

Về mặt toán học, với tập điểm dữ liệu $\{p_i\}_{i=1}^n \subset \mathbb{R}^d$, PCA thường được thực hiện qua ba bước. Trước hết, tính vector trung bình của tập điểm:

$$\bar{p} = \frac{1}{n} \sum_{i=1}^n p_i \quad (2.48)$$

Tiếp theo, xây dựng ma trận hiệp phương sai từ tập điểm đã được trừ trung bình:

$$\Sigma = \frac{1}{n} \sum_{i=1}^n (p_i - \bar{p})(p_i - \bar{p})^T \quad (2.49)$$

Cuối cùng, thực hiện phân tích trị riêng của ma trận hiệp phương sai:

$$\Sigma v_k = \lambda_k v_k \quad (2.50)$$

trong đó λ_k là trị riêng thứ k và v_k là vector riêng tương ứng. Các vector riêng chính là các thành phần chính, còn trị riêng phản ánh mức độ phương sai của dữ liệu theo từng hướng. Trong bài toán LiDAR 2D, hai trị riêng thường được sắp xếp theo thứ tự $\lambda_1 \geq \lambda_2$ [5].

Trong ngữ cảnh trích xuất đặc trưng từ dữ liệu LiDAR 2D, PCA được áp dụng trên từng cụm điểm quét nhằm nhận dạng cấu trúc hình học cục bộ. Với mỗi cụm điểm, có thể định nghĩa một chỉ số tuyến tính như sau:

$$\eta = \frac{\lambda_2}{\lambda_1 + \lambda_2} \quad (2.51)$$

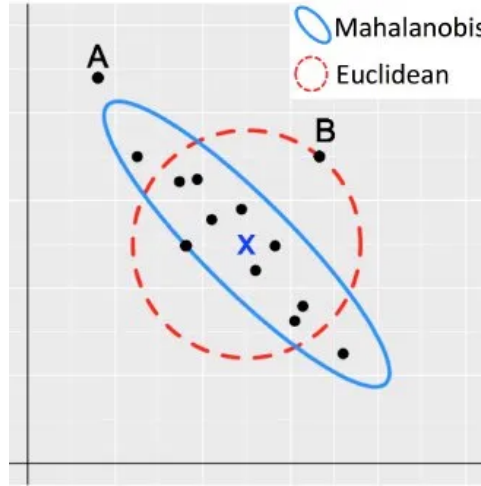
Chỉ số η nhận giá trị trong khoảng $[0, 0.5]$. Khi η nhỏ, tức là $\lambda_1 \gg \lambda_2$, phương sai của cụm điểm tập trung chủ yếu theo một hướng; điều này cho thấy cụm điểm có cấu trúc tuyến tính rõ ràng và đặc trưng tương ứng có thể được phân loại là đoạn thẳng, chẳng hạn như tường hoặc các cạnh phẳng. Ngược lại, khi η tiến gần 0.5, tức là $\lambda_1 \approx \lambda_2$, phương sai được phân bố tương đối đều theo các hướng, cho thấy cụm điểm có cấu trúc đẳng hướng hơn; khi đó, đặc trưng tương ứng thường được phân loại là góc hoặc điểm mốc cô lập. Ngưỡng phân loại η_{th} thường được lựa chọn theo thực nghiệm, phụ thuộc vào môi trường vận hành và mật độ điểm quét của cảm biến.



Hình 2.10. Minh họa hệ trục mới sau biến đổi PCA với hai thành phần chính PC1 và PC2.

c. Khoảng cách Mahalanobis

Khoảng cách Mahalanobis là thước đo mức độ sai khác giữa một điểm quan sát và một phân bố xác suất, trong đó đồng thời xét đến giá trị trung bình và cấu trúc hiệp phương sai của phân bố đó [9]. Khác với khoảng cách Euclid thông thường — vốn coi mọi hướng là như nhau — khoảng cách Mahalanobis co giãn theo hình dạng của phân bố, nhờ đó phản ánh chính xác hơn mức độ bất thường của một quan sát trong không gian mà các chiều có tương quan với nhau. Như hình minh họa cho thấy, một điểm có thể nằm xa tâm phân bố theo nghĩa Euclid nhưng vẫn có khoảng cách Mahalanobis nhỏ nếu nó lệch theo hướng có phương sai lớn; ngược lại, một điểm lệch ít hơn về mặt hình học nhưng rơi vào hướng có phương sai nhỏ lại có thể bị xem là bất thường hơn.



Hình 2.11. So sánh giữa khoảng cách Euclid và khoảng cách Mahalanobis.

Cho vector quan sát $z \in \mathbb{R}^d$, vector trung bình μ và ma trận hiệp phương sai S , khoảng cách Mahalanobis được định nghĩa bởi:

$$d_M = \sqrt{(z - \mu)^T S^{-1} (z - \mu)} \quad (2.52)$$

Khi S là ma trận đơn vị, biểu thức trên rút gọn về khoảng cách Euclid thông thường. Trong trường hợp tổng quát, ma trận S^{-1} đóng vai trò chuẩn hóa không gian quan sát theo phương sai của từng chiều và theo tương quan giữa các chiều. Nói cách khác, các hướng có phương sai lớn sẽ bị co lại, còn các hướng có phương sai nhỏ sẽ được kéo giãn ra trước khi tính khoảng cách; vì vậy, giá trị khoảng cách thu được mang ý nghĩa thống kê sát hơn với phân bố dữ liệu thực tế [9].

Trong bài toán liên kết dữ liệu của UKF-SLAM, tại mỗi bước cập nhật, hệ thống cần quyết định quan sát mới $z(k)$ thu được từ LiDAR tương ứng với landmark nào đã tồn tại trong bản đồ. Với mỗi landmark thứ i trong vector trạng thái mở rộng, bộ lọc dự đoán một quan sát kỳ vọng $\hat{z}_i$ và ma trận hiệp phương sai đổi mới S_i . Khi đó, khoảng cách Mahalanobis giữa quan sát thực tế và quan sát dự đoán của landmark thứ i được

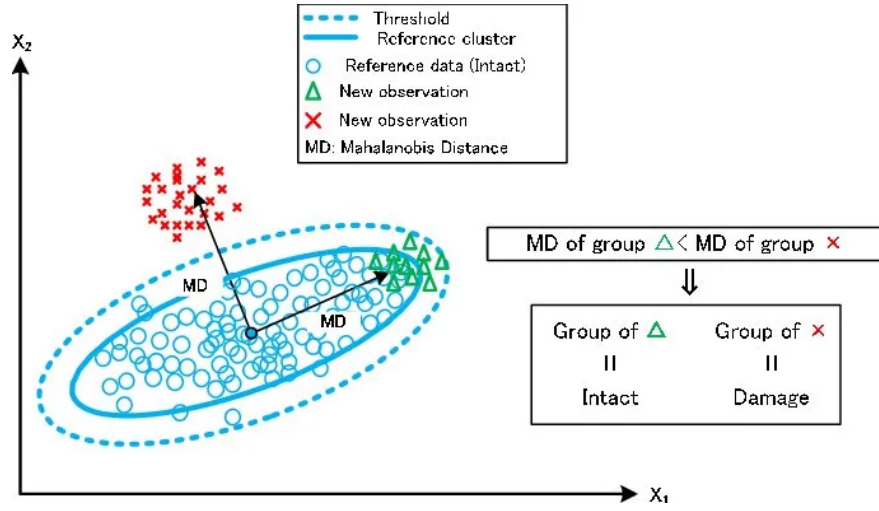
tính như sau:

$$d_i = \sqrt{(z - \hat{z}_i)^T S_i^{-1} (z - \hat{z}_i)} \quad (2.53)$$

Quan sát z sẽ được liên kết với landmark i^* thỏa mãn tiêu chí sau:

$$i^* = \underset{i}{\operatorname{argmin}} d_i, \quad \text{với} \quad d_{i^*} < d_{th} \quad (2.54)$$

trong đó d_{th} là ngưỡng chấp nhận liên kết. Nếu không tồn tại landmark nào thỏa mãn điều kiện ngưỡng, quan sát hiện tại sẽ được xem là một đặc trưng mới và được khởi tạo thêm vào bản đồ. Trong thực tế, ngưỡng d_{th} thường được lựa chọn dựa trên phân bố χ^2 với số bậc tự do bằng chiều của không gian quan sát, nhằm bảo đảm rằng quyết định liên kết có ý nghĩa thống kê nhất quán [5].



Hình 2.12. Khoảng cách Mahalanobis trong bài toán so khớp quan sát mới với cụm tham chiếu và vùng ngưỡng chấp nhận.

Ưu điểm cốt lõi của tiêu chí Mahalanobis so với khoảng cách Euclid thuần túy là khả năng tự thích nghi với độ bất định của từng landmark. Những landmark có hiệp phương sai lớn, tức còn nhiều bất định, sẽ tương ứng với vùng chấp nhận liên kết rộng hơn; ngược lại, những landmark đã được quan sát lặp lại nhiều lần và có hiệp phương sai nhỏ sẽ có vùng chấp nhận hẹp hơn. Nhờ đó, cơ chế liên kết dữ liệu trở nên phù hợp hơn với trạng thái ước lượng hiện tại của bộ lọc, góp phần duy trì tính ổn định và tính nhất quán của lời giải UKF-SLAM trong suốt quá trình robot hoạt động [9, 5].

Chương 3.

Ước lượng trạng thái với Adaptive UKF và UKF-SLAM

3.1. Thiết kế Adaptive UKF cho sensor fusion

Bộ lọc trạng thái được thiết kế nhằm hợp nhất thông tin từ encoder, IMU và từ kế để ước lượng chính xác trạng thái chuyển động của robot. Trong hệ thống thực nghiệm, Adaptive UKF đóng vai trò là tầng *sensor fusion* trung tâm, tạo ra bản tin odometry ổn định để cung cấp cho các khối xử lý mức cao hơn. Các thành phần cốt lõi của bộ lọc được trình bày lần lượt qua mô hình trạng thái, mô hình quan sát, cơ chế thích nghi và thuật toán UKF.

3.1.1. Mô hình trạng thái (State Model)

Trong hệ thống đề xuất, trạng thái của robot tại thời điểm k được mô tả bởi vector:

$$x_k = [x_k, y_k, \theta_k, v_k, \omega_k]^T \quad (3.1)$$

trong đó x_k, y_k là tọa độ của robot trong hệ quy chiếu toàn cục, θ_k là góc hướng, v_k là vận tốc tuyến tính và ω_k là vận tốc góc tại thời điểm k . Cấu trúc trạng thái này đủ gọn để duy trì khả năng xử lý thời gian thực trên nền tảng nhúng đồng thời thể hiện đầy đủ trạng thái robot với ràng buộc trên mặt phẳng 2D (bám đất và không trượt ngang).

Do robot sử dụng cấu trúc bánh xe vi sai, mô hình động học phi tuyến được xây dựng dựa trên tâm cong tức thời (*Instantaneous Center of Curvature – ICC*). Với khoảng thời gian lấy mẫu Δt , trạng thái được cập nhật theo:

$$\begin{cases} x_{k+1} = x_k + \frac{v_k}{\omega_k} [\sin(\theta_k + \omega_k \Delta t) - \sin(\theta_k)] \\ y_{k+1} = y_k - \frac{v_k}{\omega_k} [\cos(\theta_k + \omega_k \Delta t) - \cos(\theta_k)] \\ \theta_{k+1} = \theta_k + \omega_k \Delta t \\ v_{k+1} = v_k + \eta_v \\ \omega_{k+1} = \omega_k + \eta_\omega \end{cases} \quad (3.2)$$

trong đó η_v và η_ω là nhiễu quá trình, dùng để mô hình hóa sai số động học và các bất định chưa được phản ánh đầy đủ trong mô hình danh định. Trong phần hiện thực, hai thành phần nhiễu này được gắn vào vận tốc tuyến tính và vận tốc góc, do đó UKF được xây dựng dưới dạng trạng thái mở rộng có thêm hai biến nhiễu phụ trợ.

Trong trường hợp $|\omega_k|$ rất nhỏ, việc chia cho một giá trị gần bằng không có thể gây mất ổn định số. Vì vậy, hệ thống chuyển sang xấp xỉ chuyển động thẳng:

$$\begin{cases} x_{k+1} = x_k + v_k \cos(\theta_k) \Delta t \\ y_{k+1} = y_k + v_k \sin(\theta_k) \Delta t \\ \theta_{k+1} = \theta_k \end{cases} \quad (3.3)$$

Trong mã nguồn, cơ chế này được hiện thực bằng cách kiểm tra ngưỡng nhỏ của vận tốc góc và chọn giữa hai mô hình: chuyển động gần thẳng hoặc chuyển động theo quỹ đạo cong. Việc lựa chọn nhánh được thực hiện trên từng sigma point, không phải trên toàn bộ một bước thời gian. Do đó, tại cùng một thời điểm cập nhật, một số sigma points có thể được lan truyền theo mô hình chuyển động thẳng, trong khi các sigma points khác vẫn được lan truyền theo mô hình ICC. Cách xử lý đó phù hợp với đặc tính vật lý của robot bánh xe vi sai, đồng thời tránh được các vấn đề số học trong bước lan truyền sigma points.

3.1.2. Mô hình quan sát (Measurement Model)

Trong hệ thống đề xuất, các cảm biến không đo trực tiếp trạng thái tuyệt đối của robot mà chủ yếu cung cấp các đại lượng odometry hoặc *dead-reckoning* được tích lũy theo thời gian. Vì vậy, mô hình quan sát được xây dựng dựa trên các trạng thái suy diễn từ encoder, IMU và từ kế, sau đó được hợp nhất trong cùng một vector đo để phục vụ bước cập nhật của Adaptive UKF.

a. Encoder dead-reckoning

Encoder được sử dụng để xây dựng wheel odometry thông qua mô hình động học robot vi sai. Từ vận tốc tuyến tính v_k^{enc} và vận tốc góc ω_k^{enc} , trạng thái odometry của encoder được cập nhật theo:

$$\begin{cases} x_{k+1}^{enc} = x_k^{enc} + v_k^{enc} \cos(\theta_k^{enc}) \Delta t \\ y_{k+1}^{enc} = y_k^{enc} + v_k^{enc} \sin(\theta_k^{enc}) \Delta t \\ \theta_{k+1}^{enc} = \theta_k^{enc} + \omega_k^{enc} \Delta t \end{cases} \quad (3.4)$$

Từ đó, phép đo encoder thực tế được mô tả bởi:

$$z_k^{enc} = [x_k^{enc}, y_k^{enc}, \theta_k^{enc}, v_k^{enc}, \omega_k^{enc}]^T + \xi_k^{enc} \quad (3.5)$$

trong đó ξ_k^{enc} là nhiễu đo của encoder. Trong phần hiện thực, các đại lượng này tương ứng với odometry tích phân từ encoder cùng với vận tốc tuyến tính và vận tốc góc đo được tại từng thời điểm.

b. IMU dead-reckoning

IMU chủ yếu được sử dụng để theo dõi góc hướng và vận tốc góc của robot. Cụ thể, góc yaw được suy ra bằng cách tích phân vận tốc góc đo được quanh trục thẳng đứng:

$$\theta_{k+1}^{imu} = \theta_k^{imu} + \omega_k^{imu} \Delta t \quad (3.6)$$

Từ đó, vector đo của IMU được viết dưới dạng:

$$z_k^{imu} = [\theta_k^{imu}, \omega_k^{imu}]^T + \xi_k^{imu} \quad (3.7)$$

trong đó ξ_k^{imu} là nhiễu đo IMU. Mô hình này phản ánh đúng cách triển khai trong hệ thống thực nghiệm, nơi IMU không được dùng để ước lượng trực tiếp vị trí mà chủ yếu cung cấp thông tin quay để hỗ trợ bộ lọc theo dõi tư thế của robot.

c. Magnetometer heading measurement

Từ kế được sử dụng để hiệu chỉnh góc hướng tuyệt đối nhằm giảm trôi góc tích lũy từ IMU. Góc hướng được xác định từ vector từ trường đo được theo công thức:

$$\theta_{mag} = \text{atan2}(m_y, m_x) - \theta_{base} \quad (3.8)$$

trong đó θ_{base} là góc offset ban đầu của từ kế, được xác định trong giai đoạn khởi tạo để đưa phép đo về cùng mốc tham chiếu với các cảm biến khác. Khi đó, phép đo magnetometer được viết:

$$z_k^{mag} = [\theta_k^{mag}] + \xi_k^{mag} \quad (3.9)$$

trong đó ξ_k^{mag} là nhiễu đo của từ kế.

d. Vector đo tổng quát

Tại mỗi bước cập nhật, hệ thống tổ chức vector đo dưới dạng:

$$z_k = [\theta_{imu}, \omega_{imu}, x_{enc}, y_{enc}, \theta_{enc}, v_{enc}, \omega_{enc}, \theta_{mag}]^T \quad (3.10)$$

Khi đó, mô hình quan sát tổng quát được viết theo dạng phi tuyến:

$$z_k = h(x_k) + \xi_k \quad (3.11)$$

trong đó hàm quan sát có thể được biểu diễn dưới dạng:

$$h(x_k) = [\theta_k, \omega_k, x_k, y_k, \theta_k, v_k, \omega_k, \theta_k]^T \quad (3.12)$$

Trong hiện thực, dù ánh xạ đo được tổ chức dưới dạng tuyến tính theo ma trận H , bản chất của toàn bộ quá trình ước lượng vẫn là phi tuyến vì trạng thái dự đoán đã được lan truyền qua mô hình động học robot vì sai ở bước trước. Do trạng thái góc có tính tuần hoàn, tất cả các sai số góc và innovation liên quan đến θ đều được chuẩn hóa về miền: $(-\pi, \pi]$ nhằm tránh hiện tượng nhảy góc khi robot quay vượt quá 2π . Các mô hình quan sát này phản ánh trực tiếp cấu trúc của measurement vector và cơ chế fusion được triển khai trong hệ thống Adaptive UKF thực nghiệm.

3.1.3. Cơ chế thích nghi (Adaptive Mechanism)

Trong các hệ robot di động thực tế, đặc tính nhiễu của encoder, IMU và các nguồn odometry không cố định mà thay đổi theo trạng thái chuyển động của robot và điều kiện môi trường. Khi robot tăng tốc, giảm tốc, quay nhanh hoặc di chuyển trên bề mặt có độ ma sát không đồng đều, sai số của wheel odometry và dữ liệu quán tính thường tăng đáng kể. Vì vậy, việc sử dụng các ma trận hiệp phương sai cố định có thể làm bộ lọc trở nên quá tự tin hoặc ngược lại quá bảo thủ, từ đó làm giảm chất lượng ước lượng trạng thái.

Để tăng khả năng thích nghi, hệ thống sử dụng cơ chế Adaptive UKF dựa trên innovation, hay residual, của phép đo. Innovation tại thời điểm k được xác định bởi:

$$\tilde{y}_k = z_k - \hat{z}_k \quad (3.13)$$

trong đó z_k là vector đo thực tế từ cảm biến và $\hat{z}_k$ là vector đo dự đoán thu được từ UKF thông qua mô hình quan sát. Innovation phản ánh mức độ sai khác giữa trạng thái dự đoán và dữ liệu cảm biến thực tế; do đó, nó là chỉ báo quan trọng cho việc đánh giá mức độ phù hợp của mô hình nhiễu hiện tại.

Trong hệ thống thực nghiệm, cơ chế thích nghi được áp dụng chủ yếu cho ma trận nhiễu quá trình Q_k và một phần cho ma trận nhiễu đo R_k . Cụ thể, nhiễu quá trình của vận tốc tuyến tính được điều chỉnh dựa trên hai thành phần: sai lệch giữa vận tốc encoder và vận tốc trạng thái ước lượng, cùng với mức biến thiên vận tốc giữa các lần đo liên tiếp. Giá trị thích nghi được xác định theo:

$$Q_v = Q_{v0} + a_1 \max(0, e_v^2 - \varepsilon_v) + a_2 \max(0, \Delta v^2 - \delta_v) \quad (3.14)$$

trong đó:

$$e_v = v_{enc} - \hat{v} \quad (3.15)$$

là sai lệch giữa vận tốc encoder và vận tốc ước lượng của bộ lọc, còn:

$$\Delta v = v_{enc,k} - v_{enc,k-1} \quad (3.16)$$

biểu diễn độ biến thiên vận tốc theo thời gian. Các hệ số a_1 và a_2 quyết định mức độ thích nghi của bộ lọc, trong khi ε_v và δ_v đóng vai trò ngưỡng kích hoạt nhằm tránh việc tăng nhiễu do các dao động nhỏ không đáng kể.

Tương tự, nhiều quá trình của vận tốc góc được cập nhật theo:

$$Q_\omega = Q_{\omega 0} + a_1 \max(0, e_\omega^2 - \varepsilon_\omega) + a_2 \max(0, \Delta\omega^2 - \delta_\omega) \quad (3.17)$$

với:

$$e_\omega = \frac{\omega_{enc} + \omega_{imu}}{2} - \hat{\omega} \quad (3.18)$$

và:

$$\Delta\omega^2 = \frac{(\omega_{enc,k} - \omega_{enc,k-1})^2 + (\omega_{imu,k} - \omega_{imu,k-1})^2}{2} \quad (3.19)$$

Mục tiêu chính của cơ chế innovation-adaptive là bảo đảm độ ổn định của bộ lọc trong khi hàm hệ thống vẫn dựa trên mô hình vận tốc gần như không đổi. Khi robot chuyển động đều hoặc dữ liệu cảm biến có nhiễu nhỏ, các ngưỡng ε_ω và δ_ω giúp hạn hệ thống vẫn giữ niềm tin vào trạng thái hiện tại, nhờ đó trạng thái ước lượng vẫn đủ mượt và không dao động theo nhiễu tức thời. Ngược lại, khi robot quay nhanh hoặc thay đổi chuyển động đột ngột, sai lệch innovation và độ biến thiên vận tốc góc tăng lên làm Q_ω lớn hơn, giúp bộ lọc giảm mức độ tin cậy vào mô hình vận tốc không đổi và phản hồi nhanh hơn theo dữ liệu cảm biến. Vì vậy, cơ chế thích nghi này tạo sự cân bằng giữa hai yêu cầu: giữ state mượt trong nhiễu mà vẫn bảo đảm tốc độ đáp ứng khi trạng thái chuyển động của robot thay đổi.

Ngoài nhiễu quá trình, hiệp phương sai đo của encoder cũng được điều chỉnh theo sai số tích lũy odometry. Do odometry được tích phân theo thời gian nên sai số vị trí thường tăng dần theo quãng đường di chuyển. Trong hệ thống thực nghiệm, nhiễu đo encoder được cập nhật theo:

$$R_{enc,x} = R_{enc,x0} + k_d^2(x_{enc}^2 + y_{enc}^2) \quad (3.20)$$

$$R_{enc,y} = R_{enc,y0} + k_d^2(x_{enc}^2 + y_{enc}^2) \quad (3.21)$$

$$R_{enc,\theta} = R_{enc,\theta 0} + (k_\theta |\theta_{enc}|)^2 \quad (3.22)$$

trong đó k_d và k_θ là các hệ số điều chỉnh thực nghiệm. Tương tự, nhiễu đo của IMU cũng được tăng theo drift tích lũy của góc yaw:

$$R_{imu} = R_{imu,0} + (k_{imu} |\theta_{imu}|)^2 \quad (3.23)$$

Cơ chế thích nghi này giúp bộ lọc phản ứng linh hoạt hơn trước các điều kiện vận hành thay đổi, đồng thời hạn chế hiện tượng bộ lọc quá tự tin vào các nguồn đo có sai số lớn. Nhờ đó, Adaptive UKF duy trì được tính ổn định và tính nhất quán của ước lượng trạng thái tốt hơn so với UKF sử dụng nhiễu cố định. Các cơ chế trên phản ánh trực tiếp cấu trúc thích nghi đã được triển khai trong hệ thống thực nghiệm.

3.1.4. Sensor Fusion và Unscented Kalman Filter

Adaptive UKF trong hệ thống đóng vai trò là tầng *sensor fusion* trung tâm, thực hiện hợp nhất dữ liệu từ encoder, IMU và từ kế để tạo ra ước lượng trạng thái thống nhất của robot. Khác với EKF sử dụng tuyến tính hóa Jacobian quanh điểm làm việc hiện tại, UKF sử dụng Unscented Transform để lan truyền trực tiếp phân bố xác suất qua mô hình phi tuyến mà không cần tính đạo hàm Jacobian.

Trong hệ thống đề xuất, UKF được xây dựng dưới dạng *augmented-state UKF* nhằm đưa trực tiếp nhiễu quá trình vào quá trình sinh sigma points. Trạng thái mở rộng được xác định bởi:

$$x_k^a = \begin{bmatrix} x_k \\ w_k \end{bmatrix} = [x, y, \theta, v, \omega, \eta_v, \eta_\omega]^T \quad (3.24)$$

trong đó năm phần tử đầu tiên là trạng thái robot $[x, y, \theta, v, \omega]^T$, còn hai phần tử cuối là nhiễu quá trình $[\eta_v, \eta_\omega]^T$. Thứ tự này khớp với hiện thực, trong đó trạng thái chiếm các vị trí 0 đến 4 và nhiễu quá trình chiếm các vị trí 5 đến 6 của vector mở rộng. Với cách biểu diễn này, phần nhiễu không còn được xem như một hiệu chỉnh cộng ngoài sau bước dự đoán, mà được xem là một phần của trạng thái mở rộng trong quá trình tạo sigma points.

Ma trận hiệp phương sai mở rộng được xây dựng dưới dạng khối:

$$P_k^a = \begin{bmatrix} P_k & 0 \\ 0 & Q_k \end{bmatrix} \quad (3.25)$$

trong đó P_k là hiệp phương sai trạng thái và Q_k là hiệp phương sai nhiễu quá trình. Từ trạng thái mở rộng $\hat{x}_k^a$ và ma trận hiệp phương sai mở rộng P_k^a , bộ lọc sinh ra tập sigma points theo đúng quy tắc Unscented Transform đã trình bày trong các công thức (33)–(35), với tham số λ được xác định như trong công thức (39). Nếu L_a là số chiều của trạng thái mở rộng, các sigma points mở rộng được gom thành ma trận $\mathcal{X}_k^a$, trong đó mỗi cột là một vector sigma point:

$$\mathcal{X}_k^a = [X_0^a \quad X_1^a \quad \dots \quad X_{2L_a}^a] \in \mathbb{R}^{L_a \times (2L_a+1)}. \quad (3.26)$$

Mỗi vector $X_i^a \in \mathbb{R}^{L_a}$ chứa đầy đủ phần trạng thái robot và phần nhiễu quá trình của sigma point thứ i .

Sau khi sinh sigma points, phần trạng thái robot và phần nhiễu của cùng sigma point được tách ra để xây dựng vận tốc tuyến tính và vận tốc góc hiệu dụng. Cụ thể, với sigma point thứ i , các thành phần $v_k^{(i)}$, $\omega_k^{(i)}$, $\eta_v^{(i)}$ và $\eta_\omega^{(i)}$ đều được lấy từ cùng một vector sigma point mở rộng X_i^a :

$$v^{(i)} = v_k^{(i)} + \eta_v^{(i)} \quad (3.27)$$

$$\omega^{(i)} = \omega_k^{(i)} + \eta_\omega^{(i)} \quad (3.28)$$

Trong quá trình lan truyền sigma points qua mô hình động học robot vi sai, hệ thống cần xử lý đặc biệt đối với các trường hợp vận tốc tuyến tính hoặc vận tốc góc rất nhỏ. Trong thực tế, khi $|v^{(i)}|$ hoặc $|\omega^{(i)}|$ tiến gần về 0, nhiễu cảm biến và sai số số học có thể làm phát sinh các dao động không mong muốn hoặc gây mất ổn định cho mô hình ICC do phép chia cho giá trị rất nhỏ.

Vì vậy, trong hệ thống thực nghiệm, các sigma points được áp dụng cơ chế ngưỡng chuyển động (*motion threshold*). Nếu giá trị vận tốc tuyến tính hoặc vận tốc góc nhỏ hơn ngưỡng xác định trước, chúng sẽ được đưa trực tiếp về 0:

$$v^{(i)} = \begin{cases} 0, & |v^{(i)}| < v_{th} \\ v^{(i)}, & \text{ngược lại} \end{cases} \quad (3.29)$$

$$\omega^{(i)} = \begin{cases} 0, & |\omega^{(i)}| < \omega_{th} \\ \omega^{(i)}, & \text{ngược lại} \end{cases} \quad (3.30)$$

trong đó v_{th} và ω_{th} là các ngưỡng chuyển động tối thiểu được lựa chọn theo thực nghiệm.

Sau bước thresholding, hệ thống tiếp tục kiểm tra giá trị vận tốc góc của từng sigma point. Nếu:

$$|\omega^{(i)}| \leq \varepsilon \quad (3.31)$$

với ε là một hằng số rất nhỏ, mô hình chuyển động cong ICC sẽ được thay bằng mô hình chuyển động thẳng:

$$\begin{cases} x_{k+1}^{(i)} = x_k^{(i)} + v^{(i)} \cos(\theta_k^{(i)}) \Delta t \\ y_{k+1}^{(i)} = y_k^{(i)} + v^{(i)} \sin(\theta_k^{(i)}) \Delta t \\ \theta_{k+1}^{(i)} = \theta_k^{(i)} \end{cases} \quad (3.32)$$

Ngược lại, khi $|\omega^{(i)}| > \varepsilon$, sigma point được lan truyền theo mô hình ICC đầy đủ:

$$\begin{cases} x_{k+1}^{(i)} = x_k^{(i)} + \frac{v^{(i)}}{\omega^{(i)}} \left[\sin(\theta_k^{(i)} + \omega^{(i)} \Delta t) - \sin(\theta_k^{(i)}) \right] \\ y_{k+1}^{(i)} = y_k^{(i)} - \frac{v^{(i)}}{\omega^{(i)}} \left[\cos(\theta_k^{(i)} + \omega^{(i)} \Delta t) - \cos(\theta_k^{(i)}) \right] \\ \theta_{k+1}^{(i)} = \theta_k^{(i)} + \omega^{(i)} \Delta t \end{cases} \quad (3.33)$$

Các sigma points sau đó được lan truyền qua mô hình:

$$\mathcal{X}_{k+1|k} = [X_{k+1|k}^{(0)} \quad X_{k+1|k}^{(1)} \quad \dots \quad X_{k+1|k}^{(2L)}] \in \mathbb{R}^{L_x \times (2L+1)}, \quad (3.34)$$

với:

$$X_{k+1|k}^{(i)} = f(X_k^{(i)}) \quad (3.35)$$

trong đó mỗi cột của $\mathcal{X}_{k+1|k}$ là một sigma point trạng thái sau bước dự đoán, còn L_x là số chiều của vector trạng thái robot. Cách xây dựng augmented-state này cho phép UKF

mô hình hóa trực tiếp ảnh hưởng của nhiễu quá trình trong bước dự đoán thay vì chỉ cộng thêm ma trận Q_k sau khi lan truyền trạng thái. Nhờ đó, bộ lọc phản ánh chính xác hơn sự lan truyền bất định qua mô hình phi tuyến của robot.

Từ các sigma points đã lan truyền, trạng thái trung bình dự đoán được tính theo công thức (2.38), với $W_m = [W_0^{(m)} \quad W_1^{(m)} \quad \dots \quad W_{2L}^{(m)}]$ là vector trọng số.

$$\hat{x}_{k+1}^- = \sum_{i=0}^{2L} W_i^{(m)} X_{k+1|k}^{(i)} = \mathcal{X}_{k+1|k} W_m^T. \quad (3.36)$$

Đối với thành phần góc θ , bộ lọc không sử dụng trung bình đại số thông thường mà áp dụng trung bình tròn thông qua tổng các giá trị sin và cos:

$$\bar{\theta} = \text{atan2}(\sin \theta \cdot W_m^T, \cos \theta \cdot W_m^T) \quad (3.37)$$

Hiệp phương sai dự đoán được xác định từ độ lệch giữa từng sigma point dự đoán và trạng thái trung bình dự đoán:

$$dx_i = X_{k+1|k}^{(i)} - \hat{x}_{k+1}^- \quad (3.38)$$

Vector sai lệch này biểu diễn mức độ phân tán của sigma point thứ i quanh trạng thái dự đoán trung bình. Đóng góp của sigma point đó vào độ bất định chung được mô tả bởi tích ngoài $dx_i dx_i^T$. Do các sigma points trong Unscented Transform không có mức đóng góp bằng nhau, mỗi tích ngoài phải được nhân với trọng số hiệp phương sai tương ứng $W_i^{(c)}$. Vì vậy, hiệp phương sai dự đoán được viết dưới dạng tổng có trọng số:

$$P_{k+1}^- = \sum_{i=0}^{2L} W_i^{(c)} dx_i dx_i^T \quad (3.39)$$

Công thức trên cho thấy hiệp phương sai không chỉ phụ thuộc vào khoảng cách giữa từng sigma point và trung bình, mà còn phụ thuộc vào vai trò của sigma point đó trong Unscented Transform.

Trong phần triển khai, thay vì cộng tuần tự từng hạng tử $W_i^{(c)} dx_i dx_i^T$, các vector sai lệch được gom thành ma trận $D_x = [dx_0, dx_1, \dots, dx_{2L}]$ và các trọng số được gom thành vector $W^{(c)} = [W_0^{(c)}, W_1^{(c)}, \dots, W_{2L}^{(c)}]$. Ký hiệu $\odot$ trong biểu thức này không dùng theo nghĩa tích Hadamard tổng quát giữa hai ma trận cùng kích thước, mà biểu diễn thao tác nhân từng cột của ma trận sai lệch với trọng số tương ứng. Nói cách khác, $D_x \odot W^{(c)}$ nghĩa là cột dx_i của D_x được nhân với scalar $W_i^{(c)}$. Nhân kết quả này với D_x^T sẽ thu được đúng tổng các tích ngoài có trọng số:

$$P_{k+1}^- = (D_x \odot W^{(c)}) D_x^T \quad (3.40)$$

Ở bước cập nhật, các sigma points dự đoán tiếp tục được ánh xạ sang không gian đo thông qua mô hình quan sát:

$$Z_k^{(i)} = h(X_{k+1|k}^{(i)}) \quad (3.41)$$

Từ đó, phép đo dự đoán được tính theo:

$$\hat{z}_k = \sum_{i=0}^{2L} W_i^{(m)} Z_k^{(i)} = Z_k W_m^T \quad (3.42)$$

Hiệp phương sai đối mới và hiệp phương sai chéo được xác định bởi (2.42) (2.43), tuy nhiên, tương tự công thức (3.40) thao tác nhân trọng số được thực hiện bằng broadcasting:

$$D_z = Z_k - \hat{z}_k \quad (3.43)$$

$$P_{zz} = (D_z \odot W^{(c)}) D_z^T + R_k \quad (3.44)$$

$$P_{xz} = (D_x \odot W^{(c)}) D_z^T \quad (3.45)$$

Các công thức (3.39-3.40), (3.42), (3.44-3.45) không làm thay đổi bản chất của thuật toán, mà chỉ chuyển phép cộng từng hạng tử trong công thức tổng thành phép nhân ma trận. Nhờ đó, phần hiện thực có thể tránh các vòng lặp lồng nhau và tận dụng tối đa các phép toán tuyến tính đã được tối ưu trong NumPy. Cách xử lý này cũng được áp dụng trong hầu hết các phép tính ở phần SLAM.

Gain Kalman được xác định bởi:

$$K_k = P_{xz} P_{zz}^{-1} \quad (3.46)$$

Sau đó, trạng thái được hiệu chỉnh theo:

$$\hat{x}_k = \hat{x}_k^- + K_k(z_k - \hat{z}_k) \quad (3.47)$$

Đồng thời, ma trận hiệp phương sai được cập nhật theo:

$$P_k = P_k^- - K_k P_{zz} K_k^T \quad (3.48)$$

Nhờ cơ chế sensor fusion như vậy, hệ thống tận dụng được ưu điểm của từng loại cảm biến. Encoder cung cấp thông tin chuyển động ngắn hạn với tần số cao; IMU phản ứng nhanh với thay đổi vận tốc góc và hỗ trợ ổn định hướng; trong khi từ kế giúp hiệu chỉnh drift hướng dài hạn của hệ thống quán tính. Việc kết hợp các nguồn đo này giúp robot duy trì được ước lượng trạng thái ổn định hơn so với khi chỉ sử dụng một cảm biến riêng lẻ.

3.1.5. Pseudo-code Adaptive UKF Sensor Fusion

Thuật toán 3.1. Adaptive UKF Sensor Fusion

Input: $\hat{x}_{k-1}, P_{k-1}, Q_k, R_k, u_k, z_k, \Delta t$

Output: $\hat{x}_k, P_k$

1. Augmented State

1: $x_k^a \leftarrow [\hat{x}_{k-1}^T, 0_w^T]^T$

2. Augmented Covariance

2: $P_k^a \leftarrow \begin{bmatrix} P_{k-1} & 0 \\ 0 & Q_k \end{bmatrix}$

3. Sinh sigma points

3: $L \leftarrow \dim(x_k^a)$

4: $\lambda \leftarrow \alpha^2(L + \kappa) - L$

5: $S \leftarrow \text{chol}((L + \lambda)P_k^a)$

6: $X_0 \leftarrow x_k^a$

7: **for** $i = 1$ **to** L **do**

8: $X_i \leftarrow x_k^a + \text{column}_i(S)$

9: $X_{i+L} \leftarrow x_k^a - \text{column}_i(S)$

10: **end for**

4. Prediction Step

11: **for** mỗi sigma point X_i **do**

12: Tách $[x, y, \theta, v, \omega, \eta_v, \eta_\omega]$ từ X_i

13: $v \leftarrow v + \eta_v, \omega \leftarrow \omega + \eta_\omega$

14: **if** $|v| < v_{th}$ **then**

15: $v \leftarrow 0$

16: **end if**

17: **if** $|\omega| < \omega_{th}$ **then**

18: $\omega \leftarrow 0$

19: **end if**

20: **if** $|\omega| \leq \varepsilon$ **then**

21: $x \leftarrow x + v \cos(\theta) \Delta t$

22: $y \leftarrow y + v \sin(\theta) \Delta t$

23: **else**

24: $x \leftarrow x + \frac{v}{\omega} [\sin(\theta + \omega \Delta t) - \sin(\theta)]$

25: $y \leftarrow y - \frac{v}{\omega} [\cos(\theta + \omega \Delta t) - \cos(\theta)]$

26: $\theta \leftarrow \theta + \omega \Delta t$

27: **end if**

28: $X_{i,\text{pred}} \leftarrow [x, y, \theta, v, \omega]^T$

29: **end for**

5. Tính trạng thái dự đoán

30: $\hat{x}_k^- \leftarrow \text{weighted_mean}(X_{\text{pred}})$

31: $\hat{\theta} \leftarrow \text{atan2}(\sum_i W_i^{(m)} \sin \theta_i, \sum_i W_i^{(m)} \cos \theta_i)$

6. Tính hiệp phương sai dự đoán

```

32:  $D_x \leftarrow X_{\text{pred}} - \hat{x}_k^-$ 
33:  $P_k^- \leftarrow (D_x \odot W^{(c)}) D_x^T$ 
7. Adaptive Noise Update
34:  $e_v \leftarrow v_{enc} - \hat{v}$ 
35:  $e_\omega \leftarrow 0.5(\omega_{enc} + \omega_{imu}) - \hat{\omega}$ 
36:  $Q_v \leftarrow Q_v + a_1 \max(0, e_v^2 - \varepsilon_v) + a_2 \max(0, \Delta v^2 - \delta_v)$ 
37:  $Q_\omega \leftarrow Q_\omega + a_1 \max(0, e_\omega^2 - \varepsilon_\omega) + a_2 \max(0, \Delta \omega^2 - \delta_\omega)$ 
8. Measurement Prediction
38: for mỗi sigma point  $X_{i,\text{pred}}$  do
39:    $Z_i \leftarrow h(X_{i,\text{pred}})$ 
40: end for
41:  $\hat{z}_k \leftarrow \text{weighted\_mean}(Z_i)$ 
9. Innovation Covariance
42:  $D_z \leftarrow Z - \hat{z}_k$ 
43:  $P_{zz} \leftarrow (D_z \odot W^{(c)}) D_z^T + R_k$ 
44:  $P_{xz} \leftarrow (D_x \odot W^{(c)}) D_z^T$ 
10. Kalman Update
45:  $K_k \leftarrow P_{xz} P_{zz}^{-1}$ 
46:  $\hat{x}_k \leftarrow \hat{x}_k^- + K_k(z_k - \hat{z}_k)$ 
47:  $P_k \leftarrow P_k^- - K_k P_{zz} K_k^T$ 
48: return  $\hat{x}_k, P_k$ 

```

3.2. Thuật toán UKF-SLAM

Khởi UKF-SLAM nhận odometry đã được hợp nhất ở tầng trước và dữ liệu quét LiDAR để đồng thời ước lượng robot pose và bản đồ landmark. Trong kiến trúc đề xuất, gia số chuyển động giữa hai bản tin odometry liên tiếp được sử dụng làm đầu vào dự đoán của SLAM. Cách tổ chức này phù hợp với kiến trúc hệ thống, vì Adaptive UKF đã đảm nhiệm vai trò hợp nhất encoder-IMU-từ kế và cung cấp odometry ổn định cho tầng SLAM.

3.2.1. Biểu diễn trạng thái và tham số UKF

Trong thuật toán UKF-SLAM, trạng thái của robot và bản đồ landmark được biểu diễn chung trong một vector trạng thái mở rộng. Robot được mô hình hóa trên mặt phẳng với ba bậc tự do gồm vị trí và góc hướng, trong khi mỗi landmark được lưu dưới dạng tọa độ Cartesian trong hệ bản đồ. Vector trạng thái tại thời điểm đang xét được xác định bởi:

$$x = [x_R, y_R, \theta_R, m_{1x}, m_{1y}, \dots, m_{Mx}, m_{My}]^T \quad (3.49)$$

trong đó (x_R, y_R, θ_R) là robot pose trong hệ bản đồ, (m_{ix}, m_{iy}) là tọa độ của landmark thứ i , và M là số landmark hiện có trong bản đồ. Do số lượng landmark thay đổi trong quá trình vận hành, kích thước vector trạng thái cũng thay đổi theo thời gian:

$$n = 3 + 2M \quad (3.50)$$

Covariance matrix tương ứng có kích thước:

$$P \in \mathbb{R}^{n \times n} \quad (3.51)$$

Ma trận này không chỉ lưu độ bất định của từng biến trạng thái mà còn biểu diễn cross covariance giữa robot pose và toàn bộ landmark trong bản đồ. Các cross covariance này đóng vai trò quan trọng trong SLAM, vì sai số của robot pose ảnh hưởng trực tiếp tới vị trí landmark được khởi tạo hoặc cập nhật, đồng thời thông tin quan sát landmark cũng được lan truyền ngược lại để hiệu chỉnh robot pose.

UKF-SLAM sử dụng Unscented Transform để lan truyền phân bố xác suất qua mô hình phi tuyến thay vì tuyến tính hóa Jacobian như EKF-SLAM. Các công thức xác định tham số (α, β, κ) , tham số tỉ lệ λ , trọng số $W_i^{(m)}, W_i^{(c)}$ và quy tắc sinh $2n+1$ sigma points đã được trình bày trong phần cơ sở lý thuyết về Unscented Transform ở Chương 2. Trong mục này, các công thức đó được sử dụng lại với n là kích thước hiện thời của vector trạng thái SLAM.

3.2.2. Prediction step từ gia số odometry

Giả sử tại hai thời điểm liên tiếp robot có odometry:

$$o_{k-1} = (x_{k-1}^o, y_{k-1}^o, \theta_{k-1}^o) \quad o_k = (x_k^o, y_k^o, \theta_k^o) \quad (3.52)$$

Sai khác chuyển động trong hệ tọa độ toàn cục được xác định bởi:

$$\Delta x^o = x_k^o - x_{k-1}^o \quad (3.53)$$

$$\Delta y^o = y_k^o - y_{k-1}^o \quad (3.54)$$

$$\Delta \theta = \text{atan2}(\sin(\theta_k^o - \theta_{k-1}^o) \cos(\theta_k^o - \theta_{k-1}^o)) \quad (3.55)$$

Gia số chuyển động sau đó được đưa về hệ tọa độ robot:

$$\begin{bmatrix} \Delta x_R \\ \Delta y_R \\ \Delta \theta \end{bmatrix} = \begin{bmatrix} \cos \theta_{k-1}^o & \sin \theta_{k-1}^o & 0 \\ -\sin \theta_{k-1}^o & \cos \theta_{k-1}^o & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \Delta x^o \\ \Delta y^o \\ \Delta \theta \end{bmatrix}. \quad (3.56)$$

Cách biểu diễn này cho phép mô hình dự đoán hoạt động trong hệ tọa độ robot, từ đó giảm ảnh hưởng của sai số góc khi robot quay lớn.

Nhiều động học được xây dựng thích nghi theo độ dài chuyển động và độ lớn góc quay. Trước hết, độ dài chuyển động trong hệ robot được tính bởi:

$$d = \sqrt{\Delta x_R^2 + \Delta y_R^2} \quad (3.57)$$

Sau đó, ma trận nhiễu trong hệ robot là:

$$Q_R = \text{diag} (a^2 d + \epsilon, a^2 d + \epsilon, b^2 |\Delta \theta|^2 + \epsilon) \quad (3.58)$$

trong đó a là hệ số nhiễu tịnh tiến, b là hệ số nhiễu quay, còn ϵ là hằng số nhỏ bảo đảm ổn định số. Do Q_R được xác định trong hệ robot, ma trận này được quay sang hệ bản đồ theo hướng hiện tại của robot:

$$Q_G = R(\theta_R) Q_R R(\theta_R)^T \quad (3.59)$$

Ma trận nhiễu toàn trạng thái được xây dựng bằng cách đặt Q_G vào khối 3×3 tương ứng với robot pose, trong khi các khối landmark được gán bằng không do landmark được giả thiết tĩnh trong prediction step:

$$Q = \begin{bmatrix} Q_G & 0 \\ 0 & 0 \end{bmatrix} \quad (3.60)$$

Prediction

Từ trạng thái hiện tại và covariance tương ứng, hệ thống sinh tập sigma points:

$$\mathcal{X}_{k-1} = [x, x + \sqrt{(n + \lambda)P}, x - \sqrt{(n + \lambda)P}] \quad (3.61)$$

Toàn bộ sigma points được lan truyền đồng thời ở dạng ma trận thay vì xử lý tuần tự từng điểm. Với từng sigma point, robot pose được cập nhật theo gia số chuyển động trong hệ robot:

$$\begin{cases} x'_R = x_R + \cos \theta_R \Delta x_R - \sin \theta_R \Delta y_R \\ y'_R = y_R + \sin \theta_R \Delta x_R + \cos \theta_R \Delta y_R \\ \theta'_R = \theta_R + \Delta \theta \end{cases} \quad (3.62)$$

Trong khi đó, toàn bộ landmark được giữ nguyên: do bản đồ được giả thiết tĩnh trong mô hình SLAM.

$$m'_i = m_i, \quad (3.63)$$

Với landmark thứ j , hai hàng tương ứng trong ma trận sigma points là $3+2j$ và $3+2j+1$. Sau bước lan truyền, ma trận sigma points dự đoán được ký hiệu là $\mathcal{X}'$ hoặc $\mathcal{X}_{k|k-1}$:

$$\mathcal{X}' = \begin{bmatrix} x'_{R,0} & x'_{R,1} & \cdots & x'_{R,2n} \\ y'_{R,0} & y'_{R,1} & \cdots & y'_{R,2n} \\ \theta'_{R,0} & \theta'_{R,1} & \cdots & \theta'_{R,2n} \\ m_{0x,0} & m_{0x,1} & \cdots & m_{0x,2n} \\ m_{0y,0} & m_{0y,1} & \cdots & m_{0y,2n} \\ \vdots & \vdots & \ddots & \vdots \\ m_{(M-1)x,0} & m_{(M-1)x,1} & \cdots & m_{(M-1)x,2n} \\ m_{(M-1)y,0} & m_{(M-1)y,1} & \cdots & m_{(M-1)y,2n} \end{bmatrix}. \quad (3.64)$$

Trạng thái dự đoán được khôi phục bằng trung bình có trọng số của các sigma points đã lan truyền:

$$\hat{x}_k^- = \sum_{i=0}^{2n} W_i^{(m)} \mathcal{X}'_i = \mathcal{X}' W_m^T \quad (3.65)$$

Riêng góc robot được tính lại bằng trung bình vòng:

$$\bar{\theta}_R = \text{atan2}(\sin \theta \cdot W_m^T, \cos \theta \cdot W_m^T) \quad (3.66)$$

Sai lệch sigma points được xác định bởi:

$$D_x = \mathcal{X}' - \hat{x}_k^- \quad (3.67)$$

Sau khi chuẩn hóa thành phần góc của D_x , covariance matrix dự đoán được tính theo:

$$P_k^- = (D_x \odot W^{(c)}) D_x^T + Q \quad (3.68)$$

3.2.3. Mô hình quan sát landmark

Sau bước dự đoán trạng thái, UKF-SLAM cần dự đoán phép đo LiDAR tương ứng với từng landmark hiện có trong bản đồ. Mục tiêu của mô hình quan sát là ánh xạ landmark từ hệ tọa độ bản đồ sang không gian đo của LiDAR, từ đó tạo ra phép đo dự đoán để phục vụ liên kết dữ liệu và cập nhật bộ lọc.

Trong hệ thống đề xuất, mỗi landmark thứ j được lưu trong vector trạng thái dưới dạng tọa độ Descartes trong hệ bản đồ:

$$m_j = [m_{jx}, m_{jy}]^T \quad (3.69)$$

Trong khi đó, đặc trưng LiDAR thực tế sau bước trích xuất được biểu diễn trong hệ tọa độ robot bằng cặp khoảng cách–góc phương vị:

$$z_j^{\text{obs}} = [r_j^{\text{obs}}, \phi_j^{\text{obs}}]^T \quad (3.70)$$

trong đó r_j^{obs} là khoảng cách từ robot tới đặc trưng quan sát được, còn ϕ_j^{obs} là góc phương vị tương đối so với hướng hiện tại của robot. Ký hiệu z^{obs} là phép đo thực tế sẽ trình bày trong phần 3.2.4. Do landmark được lưu trong hệ bản đồ nhưng phép đo LiDAR được tạo ra trong hệ robot, hàm quan sát có dạng phi tuyến theo robot pose.

Xét sigma point dự $X_i \in X'$ có robot pose $(x'_{R,i}, y'_{R,i}, \theta'_{R,i})$. Sai lệch tọa độ giữa landmark m_j và robot được xác định bởi:

$$dx_{ij} = m_{jx} - x'_{R,i} \quad (3.71)$$

$$dy_{ij} = m_{jy} - y'_{R,i} \quad (3.72)$$

Khi đó, phép đo dự đoán ứng với sigma point i và landmark j được tính theo:

$$\hat{r}_{ij} = \sqrt{dx_{ij}^2 + dy_{ij}^2} \quad (3.73)$$

$$\hat{\phi}_{ij} = \text{atan2}(dy_{ij}, dx_{ij}) - \theta'_{R,i} \quad (3.74)$$

Với M landmark trong bản đồ, vector phép đo dự đoán toàn phần được sắp xếp theo từng cặp khoảng cách và góc phương vị:

$$\hat{z}^{\text{full}} = [\hat{r}_1, \hat{\phi}_1, \dots, \hat{r}_M, \hat{\phi}_M]^T \quad (3.75)$$

Các sigma points sau khi đi qua hàm quan sát $h(\cdot)$ được tổ chức thành ma trận phép đo dự đoán. Với sigma point thứ i , vector đo dự đoán toàn phần chính là:

$$\hat{z}_i^{\text{full}} = h(\mathcal{X}_i), \quad i = 0, 1, \dots, 2n. \quad (3.76)$$

$$\hat{\mathcal{Z}} = [\hat{z}_0^{\text{full}} \quad \hat{z}_1^{\text{full}} \quad \dots \quad \hat{z}_{2n}^{\text{full}}] = \begin{bmatrix} \hat{r}_{01} & \hat{r}_{11} & \dots & \hat{r}_{(2n)1} \\ \hat{\phi}_{01} & \hat{\phi}_{11} & \dots & \hat{\phi}_{(2n)1} \\ \vdots & \vdots & \ddots & \vdots \\ \hat{r}_{0M} & \hat{r}_{1M} & \dots & \hat{r}_{(2n)M} \\ \hat{\phi}_{0M} & \hat{\phi}_{1M} & \dots & \hat{\phi}_{(2n)M} \end{bmatrix}. \quad (3.77)$$

Trong đó, $\hat{z}_i^{\text{full}}$ là vector phép đo dự đoán toàn phần sinh ra từ sigma point thứ i . Thay vì trình bày riêng công thức trung bình cho từng landmark, hệ thống tính phép đo trung bình theo dạng broadcasting trên toàn bộ ma trận measurement sigma points. Nếu $\hat{\mathcal{Z}}$ được lưu theo từng hàng phép đo và từng cột sigma point, các hàng khoảng cách và bearing được tách bởi:

$$R_{\mathcal{Z}} = \hat{\mathcal{Z}}_{0::2,:}, \quad \Phi_{\mathcal{Z}} = \hat{\mathcal{Z}}_{1::2,:} \quad (3.78)$$

Khi đó, vector khoảng cách trung bình và vector bearing trung bình được tính đồng thời cho toàn bộ landmark:

$$\bar{r} = R_{\mathcal{Z}} W_m^T, \quad (3.79)$$

$$\bar{\phi} = \text{atan2}(\sin(\Phi_z)W_m^T, \cos(\Phi_z)W_m^T), \quad (3.80)$$

trong đó các hàm $\sin(\cdot)$ và $\cos(\cdot)$ được áp dụng theo từng phần tử để bảo toàn tính tuần hoàn của góc. Vector phép đo dự đoán toàn phần được tạo bằng cách ghép xen kẽ hai vector $\bar{r}$ và $\bar{\phi}$:

$$\hat{z}_{0::2}^{\text{full}} = \bar{r}, \quad \hat{z}_{1::2}^{\text{full}} = \bar{\phi}. \quad (3.81)$$

Sai lệch của các sigma points trong không gian đo được xác định bởi:

$$D_{\hat{z}} = \hat{\mathcal{Z}} - \hat{z}^{\text{full}}. \quad (3.82)$$

Từ các ma trận sai lệch trên, covariance của phép đo toàn phần được xác định bởi:

$$P_{zz}^{\text{full}} = (D_{\hat{z}} \odot W^{(c)}) D_{\hat{z}}^T \quad (3.83)$$

trong khi cross covariance giữa trạng thái và phép đo là:

$$P_{x\hat{z}}^{\text{full}} = (D_x \odot W^{(c)}) D_{\hat{z}}^T \quad (3.84)$$

3.2.4. Feature extraction từ LiDAR

Dữ liệu đầu vào của khối SLAM là các bản tin quét LiDAR dạng LaserScan. Mục tiêu của bước trích xuất đặc trưng là chuyển dữ liệu quét thô thành tập landmark ổn định, đồng thời gán cho mỗi landmark một ma trận observation covariance tương ứng để phục vụ quá trình liên kết dữ liệu và cập nhật UKF-SLAM. Việc rút gọn scan thành đặc trưng giúp giảm số lượng phép đo cần xử lý, tránh đưa toàn bộ đám mây điểm vào bộ lọc và cải thiện khả năng chạy thời gian thực.

Trước hết, hệ thống loại bỏ các điểm đo không hợp lệ, bao gồm giá trị vô hạn, NaN, các khoảng cách vượt giới hạn cảm biến hoặc nhỏ hơn ngưỡng tối thiểu. Với mỗi tia quét hợp lệ có khoảng cách r_i và góc quét ϕ_i , tọa độ Descartes trong hệ robot được tính bởi:

$$x_i = r_i \cos \phi_i, \quad (3.85)$$

$$y_i = r_i \sin \phi_i. \quad (3.86)$$

Mỗi điểm LiDAR được biểu diễn dưới dạng:

$$p_i = [x_i, y_i, r_i, \phi_i, \text{id}_i]^T, \quad (3.87)$$

trong đó id_i là chỉ số tia quét tương ứng trong scan gốc. Cách lưu thêm chỉ số tia giúp hệ thống duy trì thông tin thứ tự quét khi phân đoạn và gom cụm đặc trưng.

Sau bước chuyển đổi, tập điểm được phân đoạn thành các đoạn quét liên tục dựa trên độ gián đoạn khoảng cách giữa hai tia lân cận. Nếu:

$$|r_{i+1} - r_i| > \tau_d, \quad (3.88)$$

hệ thống xem đây là ranh giới giữa hai đoạn quét khác nhau, với τ_d là ngưỡng phân đoạn khoảng cách. Các đoạn quá ngắn hoặc chứa quá ít điểm sẽ bị loại bỏ để hạn chế nhiễu, phân xạ rời rạc và các vật thể không đủ ổn định để trở thành landmark.

Trên từng đoạn quét, hệ thống đánh giá độ cong cục bộ bằng phương pháp Principal Component Analysis (PCA) trên cửa sổ trượt. Với tập lân cận quanh điểm thứ i , covariance matrix của tọa độ 2D được xác định bởi:

$$C_i = \frac{1}{N} \sum_{k=1}^N (p_k - \bar{p})(p_k - \bar{p})^T, \quad (3.89)$$

trong đó N là số điểm trong cửa sổ và $\bar{p}$ là trung bình tọa độ của cửa sổ. Gọi $\lambda_1 \geq \lambda_2$ là hai trị riêng của C_i . Độ cong cục bộ được xấp xỉ bởi:

$$c_i = \frac{\lambda_{\min}}{\lambda_1 + \lambda_2}, \quad (3.90)$$

trong đó $\lambda_{\min} = \lambda_2$, còn ϵ là hằng số nhỏ tránh chia cho 0. Trong hiện thực, phép tính PCA được tối ưu bằng các tổng tích lũy của x , y , x^2 , y^2 và xy trên cửa sổ trượt, nhờ đó không cần gọi phân rã trị riêng đầy đủ cho từng điểm.

Các điểm có độ cong lớn hơn ngưỡng:

$$c_i > \tau_c \quad (3.91)$$

được chọn làm ứng viên đặc trưng. Những điểm này thường nằm tại góc tường, cạnh vật thể hoặc vùng có thay đổi hình học mạnh, nên có khả năng tái quan sát ổn định hơn so với các điểm nằm trên đoạn phẳng dài. Ngoài ngưỡng độ cong, hệ thống cũng kiểm tra khoảng cách hợp lệ để loại bỏ các ứng viên quá gần hoặc quá xa so với miền làm việc đáng tin cậy của LiDAR.

Các ứng viên đặc trưng tiếp tục được gom cụm theo độ gần nhau về góc quét. Hai điểm ứng viên được xem là thuộc cùng một cụm nếu chênh lệch góc giữa chúng nhỏ hơn ngưỡng:

$$|\phi_i - \phi_j| < \tau_\phi. \quad (3.92)$$

Kích thước tối thiểu của cụm được điều chỉnh thích nghi theo khoảng cách đo. Các cụm ở gần robot yêu cầu nhiều điểm hơn để tránh nhiễu và phân xạ lẻ, trong khi các cụm ở xa được phép chứa ít điểm hơn do mật độ tia quét giảm theo khoảng cách. Cơ chế này giúp cân bằng giữa độ ổn định của landmark gần và khả năng duy trì đặc trưng ở vùng xa.

Mỗi cụm đặc trưng sau đó được chuyển thành một landmark quan sát. Với cụm gồm N điểm, khoảng cách đại diện được tính bằng trung bình:

$$\bar{r} = \frac{1}{N} \sum_{i=1}^N r_i, \quad (3.93)$$

trong khi góc đại diện được tính bằng trung bình vòng:

$$\bar{\phi} = \text{atan2} \left(\frac{1}{N} \sum_{i=1}^N \sin \phi_i, \frac{1}{N} \sum_{i=1}^N \cos \phi_i \right). \quad (3.94)$$

Phép đo landmark cuối cùng là:

$$z = [\bar{r}, \bar{\phi}]^T. \quad (3.95)$$

Cách tính trung bình vòng cho góc giúp tránh sai lệch khi cụm điểm nằm gần biên $\pm\pi$.

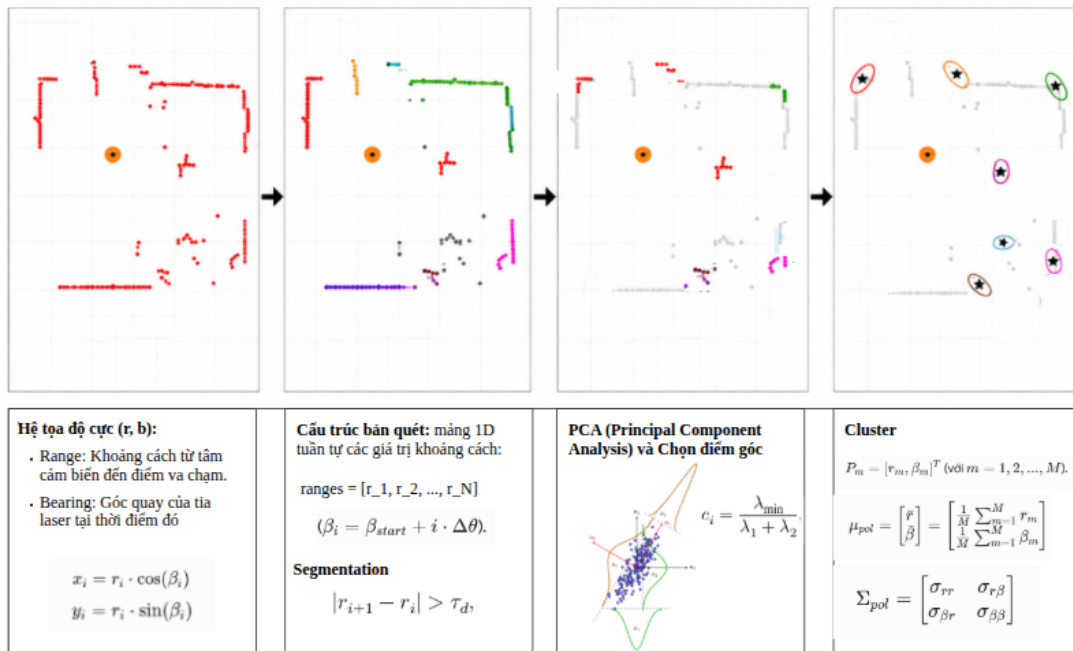
Observation covariance của cụm được tính từ covariance thực nghiệm của các cặp (r_i, ϕ_i) :

$$R_{\text{cluster}} = \text{cov}([r_i, \phi_i]), \quad (3.96)$$

sau đó cộng thêm nhiễu nền của cảm biến LiDAR:

$$R_{\text{obs}} = R_{\text{cluster}} + R, \quad (3.97)$$

trong đó R là ma trận nhiễu cơ sở của cảm biến. Nếu một cụm điểm bị phân tán mạnh do nhiễu, phản xạ hoặc bề mặt vật thể phức tạp, R_{obs} sẽ tự tăng lên. Nhờ vậy, bộ lọc UKF-SLAM giảm mức độ tin cậy vào các landmark kém ổn định trong bước liên kết dữ liệu và cập nhật Kalman gain, từ đó tăng tính bền vững của bản đồ.



Hình 3.1. Trích xuất đặc trưng từ lidar

3.2.5. Data association và update

Sau bước trích xuất đặc trưng từ LiDAR, hệ thống thu được tập các landmark quan sát trong hệ robot:

$$\mathcal{F} = \{(z_i, R_i)\}, \quad (3.98)$$

trong đó:

$$z_i = [r_i, \phi_i]^T \quad (3.99)$$

là phép đo landmark thứ i , còn R_i là observation covariance tương ứng được ước lượng từ cụm điểm LiDAR. Nhiệm vụ của bước liên kết dữ liệu là xác định mỗi landmark quan sát tương ứng với landmark nào trong bản đồ hiện có, hoặc liệu đó có phải là một landmark mới hay không.

Nếu bản đồ chưa chứa landmark nào, toàn bộ đặc trưng quan sát được chuyển trực tiếp sang danh sách landmark mới. Khi bản đồ đã tồn tại landmark, hệ thống tiến hành so sánh từng quan sát z_i với toàn bộ predicted measurement $\hat{z}_j$ thu được từ bước measurement dự đoán. Innovation giữa quan sát và landmark dự đoán được xác định bởi:

$$\nu_{ij} = \hat{z}_j - z_i. \quad (3.100)$$

Trước khi tính khoảng cách Mahalanobis, hệ thống áp dụng một công lọc thô nhằm loại bỏ nhanh các cặp không phù hợp:

$$|\nu_r| < \tau_r, \quad (3.101)$$

$$|\nu_\phi| < \tau_\phi, \quad (3.102)$$

trong đó τ_r là ngưỡng khoảng cách và τ_ϕ là ngưỡng góc. Cách lọc sơ bộ này giúp giảm đáng kể số lượng cặp cần tính Mahalanobis distance khi số landmark trong bản đồ tăng lớn.

Đối với các ứng viên còn lại, khoảng cách Mahalanobis được xác định bởi:

$$d_{ij}^2 = \nu_{ij}^T (S_j + R_i)^{-1} \nu_{ij}, \quad (3.103)$$

trong đó S_j là khối 2×2 trên đường chéo của S^{full} tương ứng với landmark thứ j , còn R_i là covariance của quan sát thứ i . Một cặp quan sát–landmark được xem là hợp lệ nếu:

$$d_{ij}^2 < \chi_{\text{th}}^2. \quad (3.104)$$

Toàn bộ các cặp hợp lệ được sắp xếp theo khoảng cách Mahalanobis tăng dần và được lựa chọn bằng chiến lược Global Nearest Neighbor (GNN) một-một. Cụ thể, mỗi landmark quan sát chỉ được ghép với một landmark trong bản đồ, đồng thời mỗi landmark

trong bản đồ chỉ nhận một quan sát. Những đặc trưng không được ghép sẽ được đưa vào danh sách landmark mới để khởi tạo ở bước tiếp theo. Khi một landmark cũ được quan sát lại, score của landmark đó được tăng lên để hỗ trợ quá trình quản lý bản đồ.

Sau khi hoàn thành data association, hệ thống thực hiện batch update thay vì cập nhật tuần tự từng phép đo riêng lẻ. Giả sử có m quan sát được ghép, actual measurement vector được tạo bởi:

$$z = [r_1, \phi_1, \dots, r_m, \phi_m]^T. \quad (3.105)$$

Measurement vector dự đoán tương ứng là:

$$\hat{z} = [\hat{r}_{j_1}, \hat{\phi}_{j_1}, \dots, \hat{r}_{j_m}, \hat{\phi}_{j_m}]^T, \quad (3.106)$$

trong đó $j_1, \dots, j_m$ là các chỉ số landmark đã được liên kết. Các khối tương ứng của measurement covariance S và cross covariance P_{xz} được trích xuất trực tiếp từ S^{full} và P_{xz}^{full} theo chỉ số landmark đã ghép:

$$S = S^{\text{full}}[\mathcal{J}, \mathcal{J}], \quad P_{xz} = P_{xz}^{\text{full}}[:, \mathcal{J}], \quad (3.107)$$

trong đó $\mathcal{J}$ là tập chỉ số các thành phần range-bearing tương ứng với các landmark đã liên kết. trong đó I_m là ma trận đơn vị kích thước m , còn $\otimes$ là tích Kronecker. Ma trận innovation cuối cùng là:

$$S_m = S + I_m \otimes R. \quad (3.108)$$

Sau đó hệ thống thực hiện phân rã Cholesky:

$$S_m = L_S L_S^T. \quad (3.109)$$

Thay vì tính nghịch đảo trực tiếp của S_m , Kalman gain được xác định bằng cách giải hai hệ tam giác:

$$K = P_{xz} S_m^{-1}. \quad (3.110)$$

Cách tính này tương ứng với việc giải hệ $L_S L_S^T K^T = P_{xz}^T$, giúp tăng ổn định số so với nghịch đảo ma trận trực tiếp.

Trạng thái và covariance được cập nhật theo:

$$x_k = x_k^- + K(z - \hat{z}), \quad (3.111)$$

$$P_k = P_k^- - K S_m K^T. \quad (3.112)$$

3.2.6. Khởi tạo, loại bỏ và hiển thị landmark

Sau bước liên kết dữ liệu, các đặc trưng không được ghép với landmark hiện có được xem là landmark mới. Hệ thống sử dụng các quan sát này để mở rộng trạng thái SLAM và cập nhật bản đồ. Giả sử một landmark mới được quan sát dưới dạng:

$$z = [r, \phi]^T, \quad (3.113)$$

trong đó r là khoảng cách từ robot tới landmark và ϕ là góc phương vị tương đối trong hệ robot. Từ tư thế hiện tại của robot (x_R, y_R, θ_R) , tọa độ landmark trong hệ bản đồ được xác định bởi:

$$m_x = x_R + r \cos(\theta_R + \phi), \quad (3.114)$$

$$m_y = y_R + r \sin(\theta_R + \phi). \quad (3.115)$$

Vector trạng thái sau đó được mở rộng bằng cách ghép thêm tọa độ landmark mới:

$$x \leftarrow [x^T, m_x, m_y]^T. \quad (3.116)$$

Việc thêm landmark không chỉ yêu cầu mở rộng vector trạng thái mà còn cần cập nhật đầy đủ covariance matrix nhằm bảo toàn tương quan giữa robot, các landmark cũ và landmark mới. Để thực hiện điều này, hệ thống sử dụng phép lan truyền tuyến tính từ robot pose và measurement noise của quan sát. Đặt:

$$\psi = \theta_R + \phi. \quad (3.117)$$

Jacobian của phép biến đổi từ robot pose sang tọa độ landmark được xác định bởi:

$$G_r = \frac{\partial(m_x, m_y)}{\partial(x_R, y_R, \theta_R)} = \begin{bmatrix} 1 & 0 & -r \sin \psi \\ 0 & 1 & r \cos \psi \end{bmatrix}. \quad (3.118)$$

Jacobian theo phép đo range-bearing là:

$$G_z = \frac{\partial(m_x, m_y)}{\partial(r, \phi)} = \begin{bmatrix} \cos \psi & -r \sin \psi \\ \sin \psi & r \cos \psi \end{bmatrix}. \quad (3.119)$$

Khối cross covariance giữa trạng thái hiện tại và landmark mới được xác định bởi:

$$P_{xm} = P_{xr} G_r^T, \quad (3.120)$$

trong đó P_{xr} là ba cột đầu của current covariance matrix, tương ứng với robot pose. Self covariance của landmark mới là:

$$P_{mm} = G_r P_{rr} G_r^T + G_z R_{\text{obs}} G_z^T + \sigma_{\text{new}}^2 I, \quad (3.121)$$

trong đó P_{rr} là khối covariance của robot pose, R_{obs} là landmark observation covariance, còn $\sigma_{\text{new}}^2 I$ là nhiễu khởi tạo bổ sung nhằm tăng ổn định số. Augmented covariance matrix sau khi thêm landmark có dạng khối:

$$P_{\text{new}} = \begin{bmatrix} P & P_{xm} \\ P_{xm}^T & P_{mm} \end{bmatrix}. \quad (3.122)$$

Cách mở rộng này giúp landmark mới không bị tách rời khỏi phần bản đồ đã có, mà vẫn duy trì cross covariance với robot pose và các landmark cũ thông qua các khối cross covariance.

Mỗi landmark được gán một score phản ánh mức độ ổn định và tần suất được tái quan sát. Khi một landmark được ghép thành công trong bước data association, score của landmark đó được tăng lên. Các landmark không được quan sát lại thường có score thấp hơn tương đối so với các landmark ổn định, do đó sẽ bị ưu tiên loại bỏ khi bản đồ đạt giới hạn kích thước.

Để giới hạn kích thước trạng thái SLAM và bảo đảm tốc độ xử lý thời gian thực, hệ thống áp dụng giới hạn tối đa số landmark trong bản đồ. Khi số landmark đạt ngưỡng cấu hình nhưng xuất hiện landmark mới, hệ thống loại bỏ landmark có score thấp nhất trước khi thêm landmark mới. Quá trình loại bỏ landmark bao gồm xóa hai phần tử tọa độ landmark khỏi vector trạng thái, loại bỏ hai hàng và hai cột tương ứng trong covariance matrix, đồng thời cập nhật lại metadata như score và trạng thái quan sát của landmark. Nếu landmark thứ j bị loại bỏ, chỉ số bắt đầu của landmark đó trong vector trạng thái là:

$$q_j = 3 + 2j. \quad (3.123)$$

Khi đó, các phần tử q_j và $q_j + 1$ được xóa khỏi cả vector trạng thái lẫn covariance matrix. Cơ chế này giúp hệ thống duy trì tập landmark ổn định và có chất lượng cao hơn thay vì giữ lại toàn bộ landmark quan sát được trong suốt quá trình vận hành.

Sau mỗi chu kỳ cập nhật, UKF-SLAM xuất robot pose, pose covariance và tập landmark để phục vụ các khối phía sau cũng như quá trình đánh giá thực nghiệm. Robot pose được công bố dưới dạng bản tin odometry, trong đó vị trí và góc hướng được lấy từ ba thành phần đầu của vector trạng thái, còn pose covariance được lấy từ khối 3×3 tương ứng trong ma trận P .

Bản đồ landmark được xuất dưới dạng các marker trực quan trên ROS 2. Mỗi landmark được biểu diễn bằng một marker tại tọa độ (m_x, m_y) trong hệ bản đồ. Landmark được quan sát ở chu kỳ hiện tại và landmark không được quan sát ở chu kỳ hiện tại được hiển thị bằng màu sắc khác nhau nhằm hỗ trợ đánh giá trực quan chất lượng SLAM. Cách trực quan hóa này giúp người vận hành kiểm tra nhanh các landmark ổn định, các landmark mới sinh và các landmark không còn được tái quan sát thường xuyên trong quá trình robot di chuyển.

3.2.7. Pseudo-code UKF-SLAM

Thuật toán 3.2. UKF-SLAM với đặc trưng LiDAR

Input: $x_{k-1}, P_{k-1}, \Delta u_k = [\Delta x_R, \Delta y_R, \Delta \theta]^T, s_k$

Output: $x_k, P_k, \mathcal{M}_k$

// 1. Kinematic Prediction

```

1:  $Q \leftarrow R(\theta_{k-1})Q_R(\Delta u_k)R(\theta_{k-1})^T$ 
2:  $\mathcal{X}_{k-1} \leftarrow \text{UT}(x_{k-1}, P_{k-1})$ 
3: for each  $\mathcal{X}_{k-1}^{(i)} \in \mathcal{X}_{k-1}$  do
4:    $\mathcal{X}_{\text{pred},R}^{(i)} \leftarrow f(\mathcal{X}_{k-1,R}^{(i)}, \Delta u_k)$ 
5:    $\mathcal{X}_{\text{pred},M}^{(i)} \leftarrow \mathcal{X}_{k-1,M}^{(i)}$ 
6:    $\theta^{(i)} \leftarrow \text{atan2}(\sin \theta^{(i)}, \cos \theta^{(i)})$ 
7: end for
8:  $x_k^- \leftarrow \sum_i W_i^{(m)} \mathcal{X}_{\text{pred}}^{(i)}$ 
9:  $\theta_k^- \leftarrow \text{atan2}(\sum_i W_i^{(m)} \sin \theta^{(i)}, \sum_i W_i^{(m)} \cos \theta^{(i)})$ 
10:  $P_k^- \leftarrow \sum_i W_i^{(c)} (\mathcal{X}_{\text{pred}}^{(i)} - x_k^-) (\mathcal{X}_{\text{pred}}^{(i)} - x_k^-)^T + Q$ 

```

// 2. Measurement Prediction

```

11: for each  $\mathcal{X}_{\text{pred}}^{(i)}$  and each landmark  $j \in \mathcal{M}_{k-1}$  do
12:    $\hat{r}_{ij} \leftarrow \sqrt{(m_{jx}^{(i)} - x_R^{(i)})^2 + (m_{jy}^{(i)} - y_R^{(i)})^2}$ 
13:    $\hat{\phi}_{ij} \leftarrow \text{atan2}(m_{jy}^{(i)} - y_R^{(i)}, m_{jx}^{(i)} - x_R^{(i)}) - \theta_R^{(i)}$ 
14:    $\mathcal{Z}_{ij}^{(i)} \leftarrow [\hat{r}_{ij}, \hat{\phi}_{ij}]^T$ 
15: end for
16:  $\hat{z}^{\text{full}} \leftarrow \sum_i W_i^{(m)} \mathcal{Z}^{(i)}$ 
17:  $S^{\text{full}} \leftarrow \sum_i W_i^{(c)} (\mathcal{Z}^{(i)} - \hat{z}^{\text{full}}) (\mathcal{Z}^{(i)} - \hat{z}^{\text{full}})^T$ 
18:  $P_{xz}^{\text{full}} \leftarrow \sum_i W_i^{(c)} (\mathcal{X}_{\text{pred}}^{(i)} - x_k^-) (\mathcal{Z}^{(i)} - \hat{z}^{\text{full}})^T$ 

```

// 3. Feature Extraction

```

19:  $\mathcal{P} \leftarrow \text{FilterAndTransform}(s_k)$ 
20:  $c_i \leftarrow \text{SlidingWindowPCA}(\mathcal{P})$ 
21:  $\mathcal{C} \leftarrow \{p_i \in \mathcal{P} \mid c_i > c_{\text{th}}\}$ 
22:  $\mathcal{F} \leftarrow \{(z_i, R_i) \mid \text{Cluster}(\mathcal{C})\}$ 

```

$$\triangleright R_i = \text{cov}([r_i, \phi_i]) + R_{\text{sensor}}$$

// 4. Data Association

```

23: if  $\mathcal{M}_{k-1} == \emptyset$  then
24:    $\mathcal{F}_{\text{new}} \leftarrow \mathcal{F}$ 
25: else
26:   for each  $z_i \in \mathcal{F}$  and each  $\hat{z}_j$  do
27:      $\nu_{ij} \leftarrow z_i - \hat{z}_j$ 
28:     if  $|\nu_r| < \tau_r$  and  $|\nu_\phi| < \tau_\phi$  then
29:        $d_{ij}^2 \leftarrow \nu_{ij}^T (S_j + R_i)^{-1} \nu_{ij}$ 
30:       if  $d_{ij}^2 < \chi_{\text{th}}^2$  then
31:         Add  $(i, j, d_{ij}^2)$  to  $\mathcal{A}_{\text{cand}}$ 
32:       end if

```



```

33:         end if
34:     end for
35:      $\mathcal{A} \leftarrow \text{GNN}(\mathcal{A}_{\text{cand}})$ 
36: end if

    // 5. Batch Measurement Update
37: if  $\mathcal{A} \neq \emptyset$  then
38:      $R_m \leftarrow I_m \otimes R_{\text{obs}}$ 
39:      $K \leftarrow P_{xz}(S + R_m)^{-1}$ 
40:      $x_k \leftarrow x_k^- + K(z - \hat{z})$ 
41:      $P_k \leftarrow P_k^- - K(S + R_m)K^T$ 
42:      $\theta_k \leftarrow \text{atan2}(\sin \theta_k, \cos \theta_k)$ 
43: else
44:      $x_k \leftarrow x_k^-, P_k \leftarrow P_k^-$ 
45: end if

    // 6. Landmark Management
46: for each unassociated  $z_i \in \mathcal{F}$  do
47:     if  $|\mathcal{M}| < N_{\text{max}}$  then
48:          $m_{ix} \leftarrow x_R + r_i \cos(\theta_R + \phi_i)$ 
49:          $m_{iy} \leftarrow y_R + r_i \sin(\theta_R + \phi_i)$ 
50:          $x_k \leftarrow [x_k^T, m_i^T]^T$ 
51:         Expand  $P_k$  via G-Transformation Matrix
52:     else
53:          $j^* \leftarrow \arg \min_j (\text{score}_j)$ 
54:         Replace  $m_{j^*}$  with  $m_i$  in  $x_k$  and  $P_k$ 
55:     end if
56: end for
57: return  $x_k, P_k, \mathcal{M}_k$ 

```

Chương 4.

Thiết kế hệ thống và thực nghiệm đánh giá

4.1. Hệ thống robot và giao diện API dữ liệu

4.1.1. Tổng quan hệ thống robot thực nghiệm

Robot thực nghiệm được xây dựng trên nền tảng robot di động bánh xe vi sai với hai bánh chủ động đặt đối xứng hai bên và một bánh tự do phía sau để duy trì cân bằng. Cấu trúc này phù hợp cho môi trường trong nhà nhờ khả năng quay tại chỗ, di chuyển linh hoạt trong không gian hẹp và dễ mô hình hóa bằng động học vi sai.



Hình 4.1. Robot thực nghiệm dùng để thu thập dữ liệu cảm biến.

Bộ xử lý trung tâm của hệ thống là Raspberry Pi 5 chạy Ubuntu và ROS 2 Jazzy. Thiết bị này đảm nhiệm việc thực thi các node ROS 2, ghi log dữ liệu, thực hiện sensor fusion và triển khai các thuật toán Adaptive UKF và UKF-SLAM. Ở tầng điều khiển thấp, vi điều khiển STM32 được sử dụng để xử lý các tác vụ thời gian thực liên quan đến động cơ và encoder.

Robot được trang bị nhiều nguồn cảm biến khác nhau nhằm cung cấp dữ liệu cho quá trình định vị và xây dựng bản đồ. Encoder bánh xe cung cấp vận tốc bánh và wheel odometry. IMU ICM20948 cung cấp dữ liệu vận tốc góc và gia tốc tuyến tính phục vụ ước lượng trạng thái chuyển động. Từ kế được sử dụng để hỗ trợ hiệu chỉnh hướng quay và giảm sai lệch tích lũy của gyroscope. Ngoài ra, cảm biến RPLIDAR cung cấp dữ liệu khoảng cách môi trường dưới dạng scan 2D phục vụ cho bài toán SLAM.

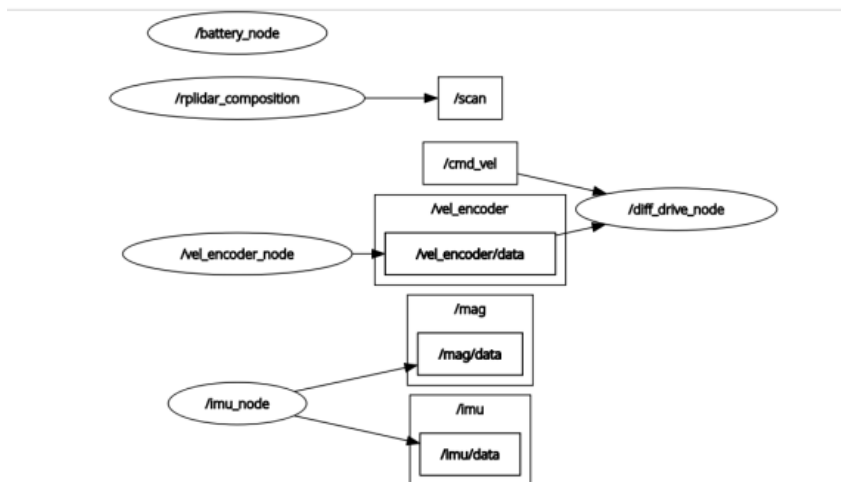
4.1.2. Giao diện API và luồng dữ liệu ROS 2

Hệ thống phần mềm được tổ chức theo mô hình publish–subscribe của ROS 2. Mỗi cảm biến được quản lý bởi một node riêng biệt chịu trách nhiệm đọc dữ liệu phần cứng và công bố dưới dạng message ROS 2 chuẩn. Nhờ đó, các thuật toán xử lý phía sau chỉ phụ thuộc vào topic và kiểu message mà không phụ thuộc trực tiếp vào giao tiếp phần cứng của từng thiết bị.

Các topic dữ liệu chính được sử dụng trong hệ thống được trình bày trong Bảng 4.1.

Bảng 4.1. Các topic dữ liệu chính trong hệ thống ROS 2.

Topic	Kiểu message	Nội dung	Tần số
/vel_encoder/data	custom message	Vận tốc bánh xe, odometry encoder	23 Hz
/imu/data	sensor_msgs/msg/Imu	Gyroscope, accelerometer	13 Hz
/mag/data	sensor_msgs/msg/MagneticField	Dữ liệu từ kế	20 Hz
/scan	sensor_msgs/msg/LaserScan	Dữ liệu quét môi trường	10 Hz
/cmd_vel	geometry_msgs/msg/Twist	Lệnh vận tốc robot	Theo lệnh

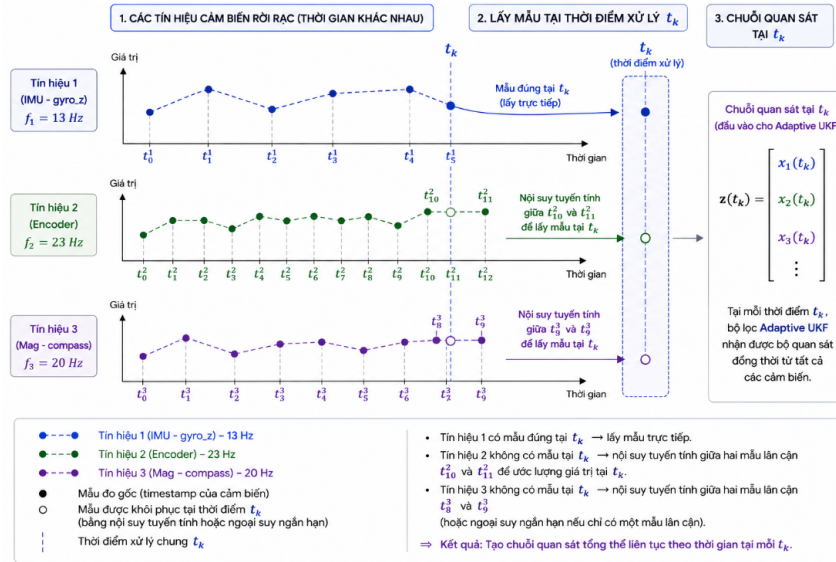


Hình 4.2. Sơ đồ giao tiếp API với phần cứng robot.

Trong message sensor_msgs/msg/Imu, thành phần vận tốc góc quanh trục z được sử dụng chủ yếu cho quá trình sensor fusion nhằm cải thiện khả năng ước lượng hướng quay của robot. Message sensor_msgs/msg/LaserScan chứa thông tin góc quét, độ phân giải góc, mảng khoảng cách ranges [], timestamp và frame_id.

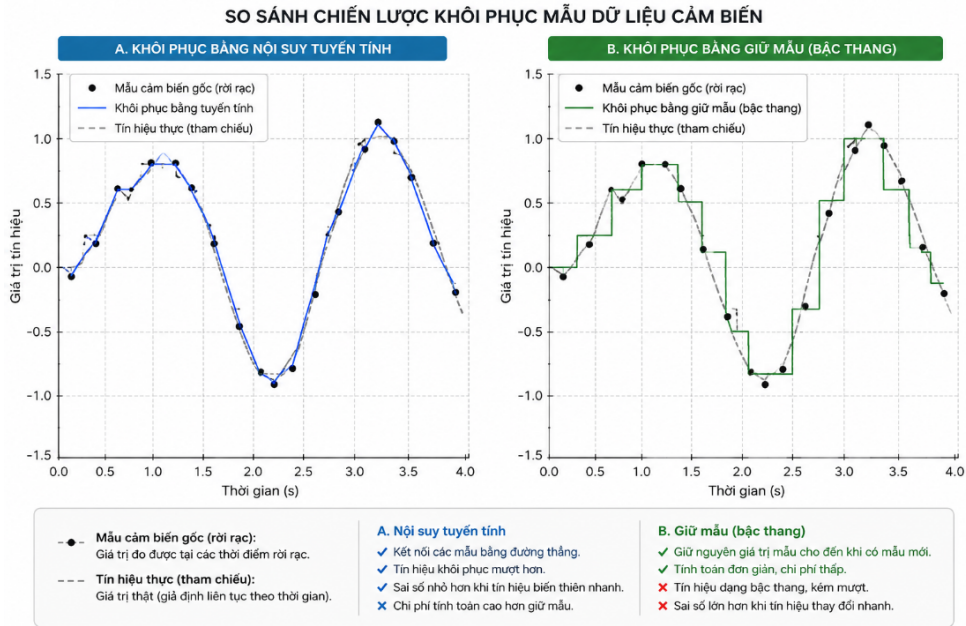
Do các cảm biến hoạt động với tần số khác nhau và có độ trễ truyền dữ liệu riêng, chuỗi quan sát thu được không đồng bộ hoàn toàn theo thời gian. Nếu sử dụng trực tiếp các mẫu dữ liệu rời rạc, bộ lọc Adaptive UKF có thể gặp hiện tượng thiếu dữ liệu tại thời điểm cập nhật hoặc xuất hiện sai lệch do chênh lệch timestamp giữa các cảm biến.

Để giải quyết vấn đề này, hệ thống sử dụng cơ chế khôi phục mẫu dữ liệu nhằm xây dựng chuỗi quan sát gần liên tục theo thời gian trước khi thực hiện sensor fusion. Mỗi nguồn dữ liệu được lưu trong một bộ đệm timestamp riêng biệt. Tại thời điểm xử lý, thuật toán tìm các mẫu dữ liệu gần nhất và thực hiện xấp xỉ giá trị cảm biến tại thời điểm cần cập nhật.



Hình 4.3. Quá trình lấy mẫu lại các tín hiệu cảm biến không đồng bộ để tạo vector quan sát tại thời điểm xử lý chung t_k .

Hai chiến lược chính được sử dụng gồm giữ mẫu bậc không (zero-order hold) và nội suy tuyến tính. Với phương pháp giữ mẫu, giá trị cảm biến được giữ nguyên cho đến khi xuất hiện mẫu mới, tạo ra tín hiệu dạng bậc thang. Phương pháp này có ưu điểm là đơn giản và chi phí tính toán thấp nhưng dễ gây gián đoạn khi tín hiệu thay đổi nhanh. Ngược lại, nội suy tuyến tính giả thiết tín hiệu biến đổi tuyến tính giữa hai mẫu liên tiếp, nhờ đó tạo ra tín hiệu mượt hơn và phản ánh tốt hơn xu hướng thay đổi thực tế của cảm biến.



Hình 4.4. So sánh phương pháp giữ mẫu bậc không và nội suy tuyến tính khi khôi phục dữ liệu cảm biến không đồng bộ.

Hình 4.4 minh họa sự khác biệt giữa hai phương pháp khôi phục mẫu dữ liệu. Phương pháp giữ mẫu tạo ra chuỗi quan sát dạng rời rạc theo từng mức giá trị, trong khi nội suy tuyến tính cho phép tái tạo tín hiệu gần liên tục hơn theo thời gian. Trong hệ thống đề xuất, nội suy tuyến tính được ưu tiên cho các đại lượng biến đổi liên tục như vận tốc hoặc vận tốc góc nhằm cải thiện độ ổn định của Adaptive UKF.

Với hai mẫu dữ liệu tại thời điểm t_0 và t_1 , giá trị tại thời điểm xử lý t_{min} được tính theo:

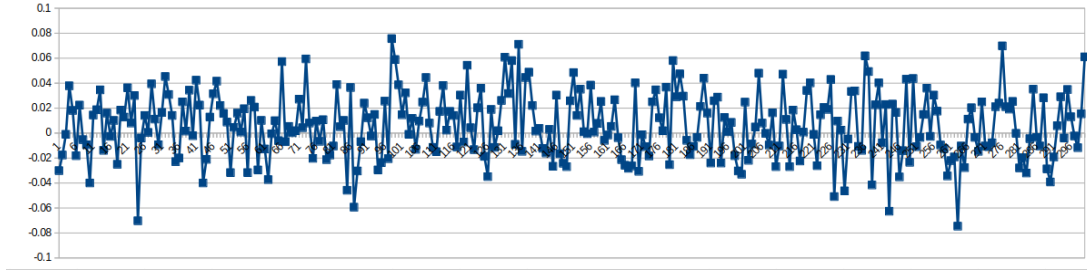
$$s(t_{min}) = s(t_0) + \frac{s(t_1) - s(t_0)}{t_1 - t_0}(t_{min} - t_0). \quad (4.1)$$

Ngoài ra, trong trường hợp thiếu dữ liệu ngắn hạn, hệ thống cho phép ngoại suy trong khoảng thời gian nhỏ để tránh gián đoạn quá trình sensor fusion. Tham số `time_sync_thres = 0.01` được sử dụng làm ngưỡng sai lệch timestamp tối đa giữa các cảm biến. Nếu độ lệch vượt quá 10 ms, hệ thống sẽ ưu tiên sử dụng dữ liệu đã được khôi phục bằng nội suy hoặc giữ mẫu thay cho dữ liệu trực tiếp. Cơ chế này giúp bộ lọc Adaptive UKF duy trì hoạt động ổn định ngay cả khi các cảm biến phát dữ liệu không đồng bộ hoàn toàn.

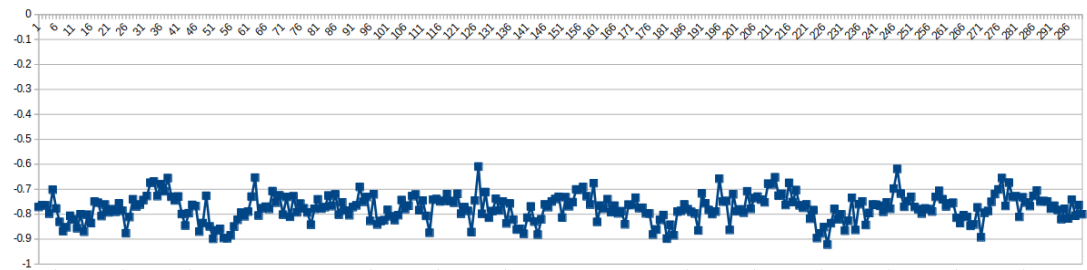
4.1.3. IMU – Vận tốc góc và đặc tính nhiễu gyroscope

Đặc tính nhiễu khi robot đứng yên và khi quay đều.

Để đánh giá đặc tính nhiễu của gyroscope, robot được đặt đứng yên hoàn toàn trên mặt phẳng và dữ liệu vận tốc góc quanh trục z được thu thập liên tục ở tần số khoảng 13 Hz. Ngoài ra, hệ thống cũng ghi nhận tín hiệu gyro_z trong trường hợp robot quay đều để so sánh sự thay đổi mức nhiễu giữa trạng thái tĩnh và trạng thái chuyển động.



Hình 4.5. Tín hiệu vận tốc góc yaw của IMU khi robot đứng yên.



Hình 4.6. Tín hiệu vận tốc góc yaw của IMU khi robot quay đều.

Hình 4.5 và Hình 4.6 trình bày tín hiệu gyro_z trong hai trường hợp: robot đứng yên và robot quay đều. Khi robot đứng yên, tín hiệu vận tốc góc dao động quanh giá trị 0 thay vì bằng 0 tuyệt đối do ảnh hưởng của bias cảm biến và nhiễu thấp tần. Các thống kê thực nghiệm thu được cho thấy giá trị trung bình và phương sai xấp xỉ:

$$\mu_{\text{gyro}} \approx 0.0059 \text{ rad/s}, \quad \sigma_{\text{gyro}}^2 \approx 0.000695 \text{ (rad/s)}^2. \quad (4.2)$$

Giá trị trung bình khác 0 phản ánh sai lệch bias tĩnh của gyroscope. Nếu không được hiệu chỉnh, sai số này sẽ tích lũy theo thời gian và gây lệch góc yaw trong quá trình tích phân vận tốc góc.

Trong trường hợp robot quay đều với vận tốc góc khoảng 0.8 rad/s, tín hiệu gyro_z dao động quanh giá trị trung bình khoảng -0.78 rad/s . Đồng thời, phương sai tăng lên khoảng:

$$\sigma_{\text{rotate}}^2 \approx 0.00288 \text{ (rad/s)}^2. \quad (4.3)$$

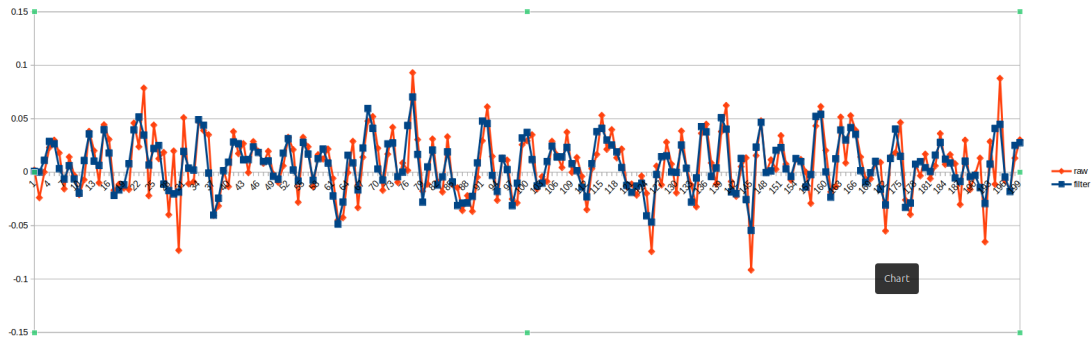
Kết quả này cho thấy khi robot chuyển động, rung động cơ khí và nhiễu động từ động cơ làm mức nhiễu của IMU tăng đáng kể so với trạng thái đứng yên.

Bias tĩnh của gyroscope được hiệu chỉnh thông qua tham số `vel_offset` trong file cấu hình của Adaptive UKF.

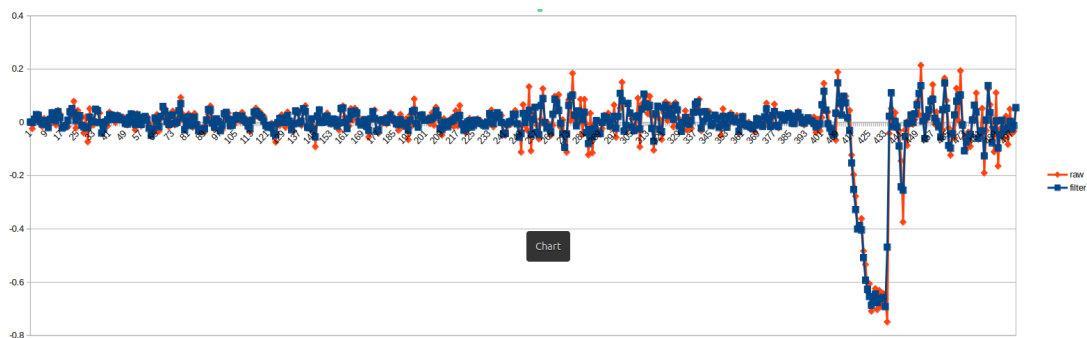
Bộ lọc thông thấp.

Một hướng tiếp cận phổ biến khi xử lý dữ liệu gyroscope là áp dụng bộ lọc thông thấp (LPF) trước khi đưa tín hiệu vào bộ lọc trạng thái nhằm giảm nhiễu cao tần. Tuy nhiên, qua thực nghiệm trên hệ thống này, hiệu quả của LPF không rõ rệt do tần số lấy mẫu của IMU tương đối thấp, chỉ khoảng 10–20 Hz.

Hình 4.7 và Hình 4.8 trình bày so sánh giữa tín hiệu gyroscope thô và tín hiệu sau khi lọc bằng Butterworth bậc hai với tần số cắt $f_c = 4$ Hz trên dữ liệu lấy mẫu $f_s = 13$ Hz. Kết quả thống kê cho thấy phương sai của tín hiệu sau lọc đạt khoảng $0.000688 \text{ (rad/s)}^2$, trong khi tín hiệu thô có phương sai khoảng $0.000430 \text{ (rad/s)}^2$. Điều này cho thấy bộ lọc thông thấp không cải thiện đáng kể mức nhiễu của tín hiệu gyroscope trong điều kiện thực nghiệm.



Hình 4.7. So sánh tín hiệu gyroscope thô và sau lọc thông thấp trong đoạn robot gần trạng thái đứng yên.



Hình 4.8. So sánh tín hiệu gyroscope thô và sau lọc thông thấp trong đoạn có biến thiên vận tốc góc lớn.

Ngoài ra, bộ lọc IIR nhân quả còn gây ra hiện tượng trễ pha. Với độ trễ khoảng 40 ms. Đối với bài toán định vị thời gian thực, độ trễ này ảnh hưởng trực tiếp đến độ chính xác của quá trình sensor fusion.

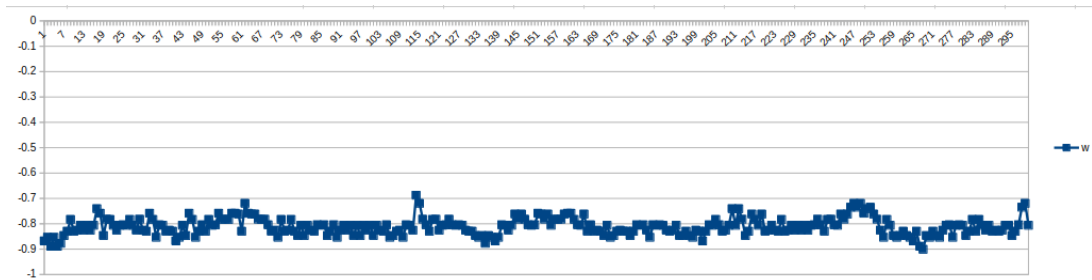
Trong quá trình triển khai thực tế, hệ thống sử dụng trực tiếp dữ liệu IMU thô làm đầu vào cho Adaptive UKF thay vì áp dụng LPF riêng biệt. Bộ lọc Adaptive UKF thực hiện smoothing trong không gian trạng thái thông qua mô hình động học và cơ chế adaptive noise estimation, nhờ đó vẫn đạt kết quả ổn định mà không gặp vấn đề về trễ tín hiệu do lọc thông thấp gây ra. Nhiễu đo của IMU được mô hình hóa trong cấu hình:

```
imu:  
  vel_offset: [0.0055]  
  R: [0.01, 0.00288]
```

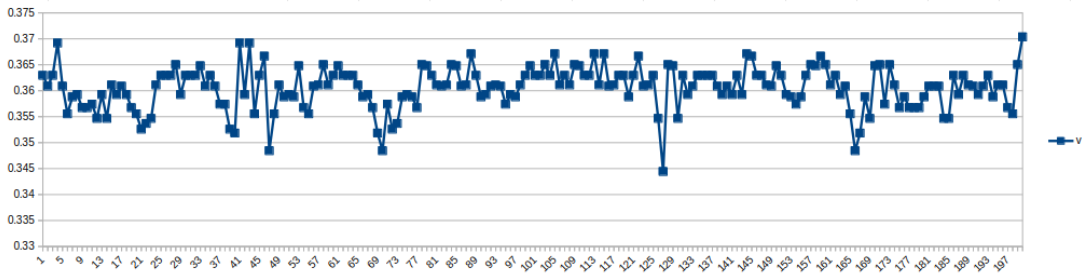
4.1.4. Encoder – Wheel odometry và đặc tính nhiễu

Encoder đo trực tiếp chuyển động của hai bánh xe. Trong chương này, phần encoder chỉ tập trung vào đặc tính thực nghiệm của dữ liệu vận tốc sau khi đã được node /vel_encoder_node quy đổi do robot cung cấp với tần số 23Hz; các công thức động học và tích phân wheel odometry đã được trình bày ở phần cơ sở lý thuyết.

Để đánh giá độ ổn định của encoder trong quá trình chuyển động, robot được cho quay đều với vận tốc gần như không đổi và dữ liệu encoder được thu thập liên tục. Hình 4.9 và Hình 4.10 trình bày tín hiệu vận tốc góc và vận tốc tuyến tính được suy ra từ encoder bánh xe.



Hình 4.9. Vận tốc góc của robot suy ra từ encoder bánh xe khi robot quay đều.



Hình 4.10. Vận tốc tuyến tính của robot suy ra từ encoder bánh xe khi robot chuyển động đều.

Đồ thị vận tốc góc cho thấy tín hiệu dao động quanh giá trị trung bình khoảng:

$$\omega \approx -0.81 \text{ rad/s}, \quad \sigma_{\omega}^2 \approx 0.00115 \text{ (rad/s)}^2. \quad (4.4)$$

Sai số dao động chủ yếu đến từ độ phân giải encoder, sai lệch cơ khí giữa hai bánh xe và hiện tượng rung động trong quá trình robot quay.

Đồ thị vận tốc tuyến tính cho thấy tín hiệu ổn định hơn đáng kể, với giá trị trung bình và phương sai xấp xỉ:

$$v \approx 0.3605 \text{ m/s}, \quad \sigma_v^2 \approx 1.68 \times 10^{-5} \text{ (m/s)}^2. \quad (4.5)$$

Kết quả cho thấy encoder cung cấp thông tin vận tốc tuyến tính khá ổn định trong điều kiện robot chuyển động đều. Tuy nhiên, vận tốc góc có mức dao động lớn hơn do ảnh hưởng trực tiếp từ sai số vị sai giữa hai bánh xe. Trong hệ thống đề xuất, dữ liệu encoder được sử dụng như nguồn quan sát vận tốc đầu vào cho Adaptive UKF nhằm hỗ trợ quá trình dự đoán trạng thái chuyển động của robot.

Sai số dài hạn của encoder vẫn xuất hiện do tích lũy sai số tích phân, sai số bán kính bánh xe, sai số khoảng cách hai bánh và trượt bánh khi robot tăng tốc hoặc quay nhanh. Đặc biệt, sai số vị trí odometry tăng dần theo quãng đường robot di chuyển. Để phản ánh đặc tính này, hệ thống sử dụng cơ chế adaptive covariance cho odometry:

$$R_{enc}^k = R_{enc,0} + k_d^2 (x_{enc}^2 + y_{enc}^2), \quad k_d = R_{enc_scale} = 0.01. \quad (4.6)$$

Điều này có nghĩa rằng khi robot di chuyển càng xa vị trí gốc, bộ lọc sẽ giảm dần mức độ tin tưởng vào wheel odometry.

4.1.5. Từ kế – Tham chiếu góc hướng tuyệt đối

Từ kế được sử dụng để cung cấp tham chiếu hướng tuyệt đối cho robot dựa trên vector từ trường Trái Đất. Tuy nhiên, góc đo trực tiếp từ từ kế phụ thuộc vào môi trường từ địa phương và hướng đặt robot tại thời điểm khởi động, do đó không thể sử dụng trực tiếp làm góc tham chiếu toàn cục.

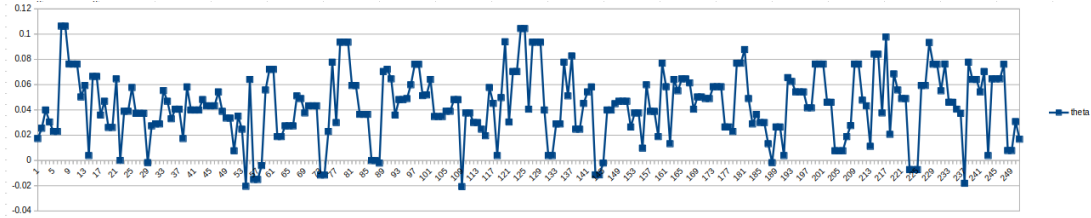
Để bảo đảm hệ thống luôn bắt đầu từ cùng một mốc hướng, quá trình khởi tạo được thực hiện ngay khi bật nguồn. Hệ thống thu thập 10 mẫu đầu tiên của góc từ kế và tính giá trị trung bình vòng để tạo offset tham chiếu:

$$\theta_{mag,base} = \text{atan2} \left(\sum_i \sin \theta_i, \sum_i \cos \theta_i \right). \quad (4.7)$$

Sau khi khởi tạo, góc từ kế sử dụng trong hệ thống được chuyển sang dạng tương đối:

$$\theta_{mag}^k = \text{atan2} (m_y^k, m_x^k) - \theta_{mag,base}, \quad (4.8)$$

trong đó m_x^k và m_y^k là hai thành phần từ trường tại thời điểm k . Góc tương đối này tiếp tục được chuẩn hóa về miền $[-\pi, \pi]$ trước khi đưa vào vector quan sát của Adaptive UKF. Cách tiếp cận này giúp robot luôn bắt đầu với góc tham chiếu gần $\theta = 0$ độ lập với hướng đặt ban đầu hoặc sai lệch từ trường địa phương.



Hình 4.11. Dữ liệu góc yaw từ từ kế trong quá trình robot hoạt động.

Hình 4.11 trình bày dữ liệu góc yaw từ từ kế trong quá trình robot hoạt động. Kết quả thực nghiệm cho thấy tín hiệu dao động quanh giá trị trung bình khoảng:

$$\mu_{mag} \approx 0.043 \text{ rad}, \quad (4.9)$$

với phương sai:

$$\sigma_{mag}^2 \approx 0.000712 \text{ rad}^2. \quad (4.10)$$

So với gyroscope, tín hiệu từ kế có mức biến thiên chậm hơn nhưng chịu ảnh hưởng rõ rệt của nhiễu điện từ môi trường, đặc biệt từ động cơ DC, khung kim loại và các nguồn từ trường trong nhà. Dù phương sai khá thấp nhưng biên độ dao động lại khá lớn và sai lệch so với tham chiếu là 0.043 rad. Vì vậy, trong hệ thống đề xuất, từ kế chủ yếu được sử dụng như nguồn hiệu chỉnh hướng dài hạn cho Adaptive UKF.

Giá trị nhiễu đo của từ kế được thiết lập:

$$R_{mag} = 0.1 \text{ rad}^2, \quad (4.11)$$

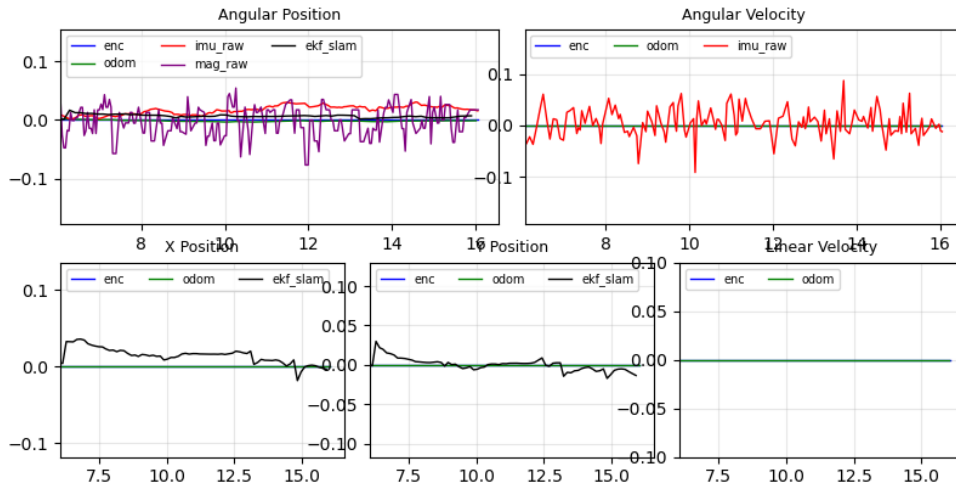
nhằm phản ánh mức nhiễu thực tế của môi trường hoạt động trong nhà.

4.1.6. Kết quả so sánh các nguồn cảm biến trong chuyển động

Các mục 4.1.3–4.1.5 đã phân tích đặc tính từng cảm biến riêng lẻ trong điều kiện tĩnh hoặc chuyển động đơn giản. Phần này tổng hợp lại bằng cách quan sát đồng thời các nguồn tín hiệu trong các pha chuyển động thực tế của robot, nhằm làm rõ ưu nhược điểm tương đối giữa từng cảm biến và cơ sở thực nghiệm cho thiết kế sensor fusion. Năm tín hiệu được theo dõi song song gồm encoder tích phân (enc), Adaptive UKF (odom), IMU tích phân thô (mu_raw), từ kế (mag_raw) và UKF-SLAM (ekf_slam).

Pha 1 – Robot đứng yên.

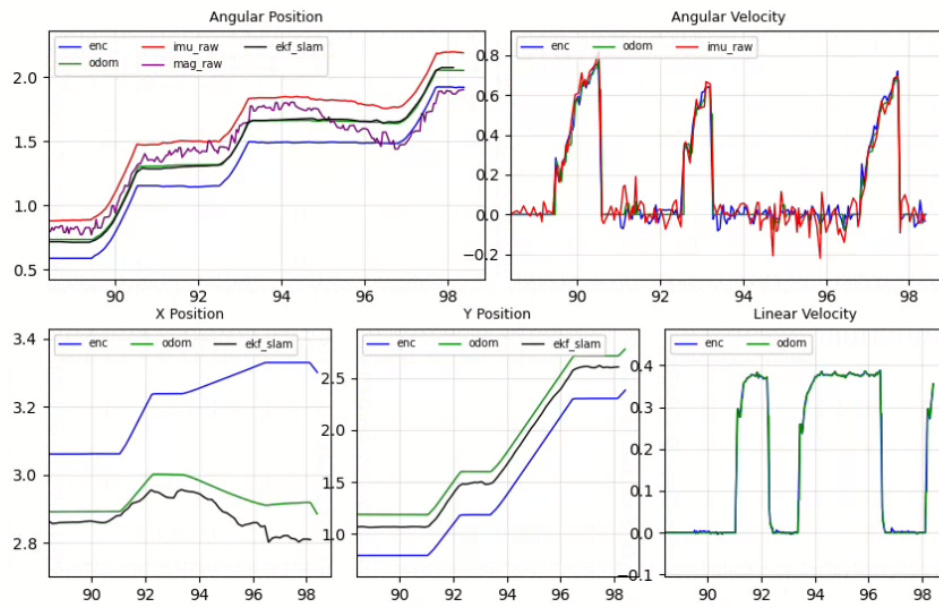
Trong pha robot đứng yên, vận tốc tuyến tính xấp xỉ 0 và các đồ thị vị trí x/y gần như phẳng hoàn toàn. Trên đồ thị góc hướng, năm đường tín hiệu bám sát nhau và chỉ dao động nhỏ trong biên độ khoảng ± 0.05 rad. Đây là pha mà encoder và IMU có thể tự kiểm tra chéo nhau tốt nhất vì robot không có chuyển động thực, do đó sai số chưa bị tích lũy bởi quá trình tích phân quãng đường hoặc góc quay.



Hình 4.12. So sánh các nguồn cảm biến khi robot đứng yên, vận tốc tuyến tính xấp xỉ 0 và góc hướng dao động nhỏ.

Pha 2 – Quay lớn và dừng đột ngột.

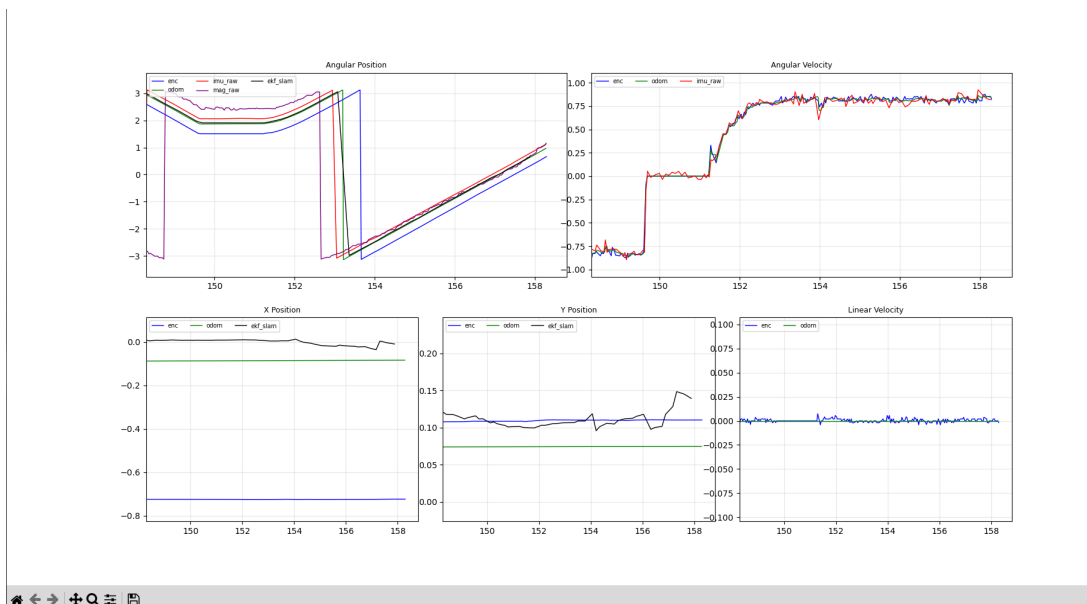
Trong pha quay nhanh, mu_raw và enc bắt đầu phân kỳ ngay khi vận tốc góc tăng lớn. Từ kế mag_raw đo góc tuyệt đối nên không bị drift tích lũy, nhưng dao động nhiều hơn các nguồn còn lại do nhiễu từ trường khi động cơ hoạt động. Adaptive UKF (odom) và UKF-SLAM (ekf_slam) bám gần mag_raw hơn so với encoder và IMU tích phân thô. Về cơ chế, khi innovation góc tăng lớn trong pha quay nhanh, thành phần nhiễu quá trình Q_ω trong cơ chế adaptive tăng theo công thức ở Chương 2, khiến bộ lọc giảm tin tưởng vào mô hình dự đoán và tăng trọng số cho phép đo.



Hình 4.13. So sánh các nguồn cảm biến trong đoạn chuyển động có thay đổi góc quay lớn.

Pha 3 – Quay liên tục đều.

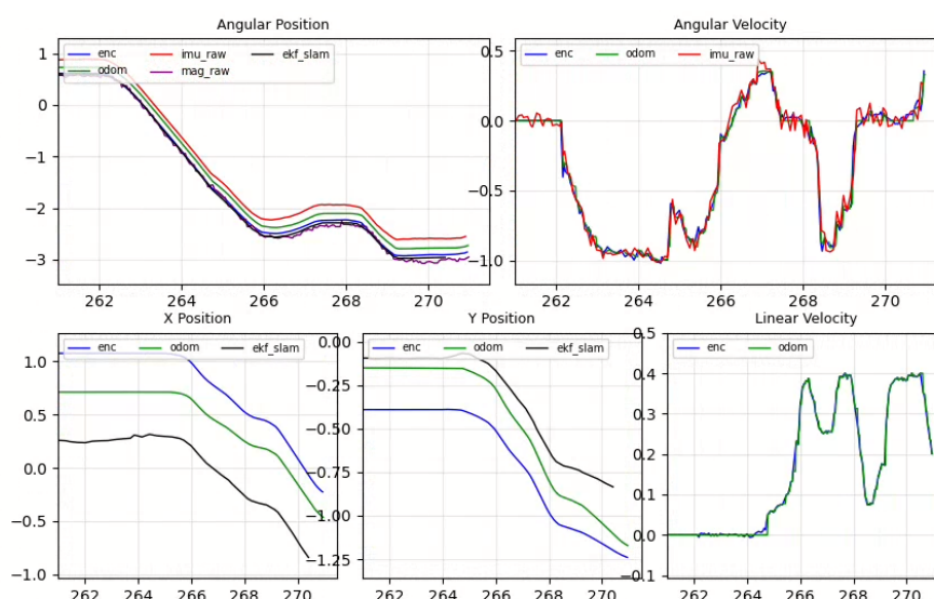
Hình 4.14 thể hiện pha robot quay với vận tốc gần không đổi, trong đó góc hướng giảm gần tuyến tính theo thời gian. Đây là pha bộc lộ rõ nhất hiện tượng drift tích lũy của các nguồn góc tương đối.



Hình 4.14. So sánh các nguồn cảm biến trong pha quay liên tục, thể hiện sai khác giữa góc tích phân từ từng nguồn đo.

Trong pha này, enc và mu_raw có xu hướng phân kỳ dần với tốc độ khác nhau: encoder chịu ảnh hưởng của sai số vị sai bánh xe, còn IMU tích phân thô chịu ảnh hưởng của bias gyroscope tích lũy. Từ kể không bị drift dài hạn nhưng nhiễu từ trường tăng khi robot đang chạy do động cơ DC và môi trường trong nhà. Adaptive UKF duy trì giá trị trung gian hợp lý giữa các nguồn, không đi theo hoàn toàn encoder hay IMU, mà hội tụ theo trọng số Kalman gain dựa trên covariance của từng nguồn. Trong cùng pha, vận tốc tuyến tính tương đối ổn định và vị trí x/y thay đổi chậm theo quỹ đạo cong; encoder cho vận tốc tuyến tính mượt hơn đáng kể so với vận tốc góc, phù hợp với phương sai $\sigma_v^2 \approx 1.68 \times 10^{-5} \text{ (m/s)}^2$ đã trình bày ở mục 4.1.4.

Pha 4 – Chuyển trạng thái.



Hình 4.15. So sánh các nguồn cảm biến trong pha chuyển trạng thái, khi vận tốc góc thay đổi rõ rệt theo thời gian.

Dựa trên quan sát với ekf_slam làm tham chiếu, ba nguồn cảm biến bộc lộ đặc tính bù trừ nhau rõ rệt. Encoder bám ekf_slam tốt nhất trong điều kiện di chuyển thẳng đều, nhưng sai lệch tăng nhanh trong các pha quay do trượt bánh và sai số vị sai cơ khí. Vì vậy, encoder là nguồn đáng tin cậy nhất cho vận tốc tuyến tính tức thời, nhưng không đủ tin cậy cho góc hướng trong hành trình dài có nhiều lần quay.

IMU phản ứng nhanh và không bị ảnh hưởng bởi trượt bánh, nhưng bias tĩnh và nhiễu tăng khi chuyển động khiến góc tích phân mu_raw phân kỳ khỏi ekf_slam theo thời gian. Đặc tính này làm IMU phù hợp cho theo dõi thay đổi góc ngắn hạn hơn là ước lượng góc tuyệt đối dài hạn. Ngược lại, từ kể không bị drift dài hạn và có xu hướng hội tụ về tham chiếu ekf_slam trong dài hạn, nhưng nhiễu từ trường môi trường trong nhà làm tín hiệu kém mượt và đôi khi có sai lệch đột ngột khi robot đi qua vùng nhiễu từ. Do

đó, từ kế phù hợp làm nguồn tham chiếu góc dài hạn nhưng không nên sử dụng độc lập vì nhiều ngắn hạn lớn.

Adaptive UKF (odom) duy trì bám ekf_slam tốt trong tất cả các pha nhờ khai thác bù trừ ba nguồn: tin tưởng encoder về vận tốc tuyến tính, tin tưởng IMU về phản ứng góc ngắn hạn, và neo vào từ kế để ngăn drift dài hạn. Cơ chế adaptive điều chỉnh Q theo innovation đóng vai trò quan trọng trong pha quay nhanh: khi encoder và IMU đều có sai số lớn, bộ lọc tự động tăng Q_ω và dồn trọng số về phía từ kế, giúp odom phục hồi về gần ekf_slam nhanh hơn bất kỳ nguồn đơn lẻ nào.

Bảng 4.2. Tóm tắt đặc tính các nguồn cảm biến và bộ lọc theo kịch bản chuyển động.

Nguồn	Định danh	Đi thẳng	Quay nhanh	Drift dài hạn	Góc đột ngột	Nhiều ngắn hạn	Trượt bánh
Encoder	Wheel encoder; tần số ROS khoảng 23 Hz	Tốt nhất cho vận tốc tuyến tính và đoạn thẳng đều	Trung bình; sai lệch tăng khi quay gấp do slip và sai số vi sai cơ khí	Trung bình–cao nếu tích phân odometry lâu	Nhanh ở mức vận tốc bánh, nhưng yaw suy ra dễ lệch khi mất bám	Nhỏ ở v , trung bình ở ω	Cao
IMU	ICM20948; /imu/data khoảng 13 Hz; dùng chủ yếu gyro_z	Tốt	Trung bình; phản ứng nhanh nhưng nhiễu tăng khi robot rung/chạy	Trung bình–cao do bias tích phân	Rất nhanh	Trung bình; tăng khi robot chuyển động	Không
Từ kế	/mag/data khoảng 20 Hz; dùng làm tham chiếu heading tuyệt đối	Tốt nếu môi trường từ yên tĩnh	Khá; không drift nhưng dao động bởi nhiễu từ	Nhỏ	Nhanh về góc tuyệt đối nhưng có thể giật khi nhiễu từ đổi nhanh	Lớn theo môi trường	Không
Adaptive UKF	Bộ ước lượng hợp nhất encoder, IMU và từ kế; publish /odometry/data	Tốt	Tốt nhất trong nhóm odometry nhờ kết hợp nhiều nguồn	Nhỏ nếu từ kế còn tin cậy	Nhanh nhưng mượt hơn IMU thô	Trung bình–thấp	Giảm thiểu; vẫn bị ảnh hưởng gián tiếp

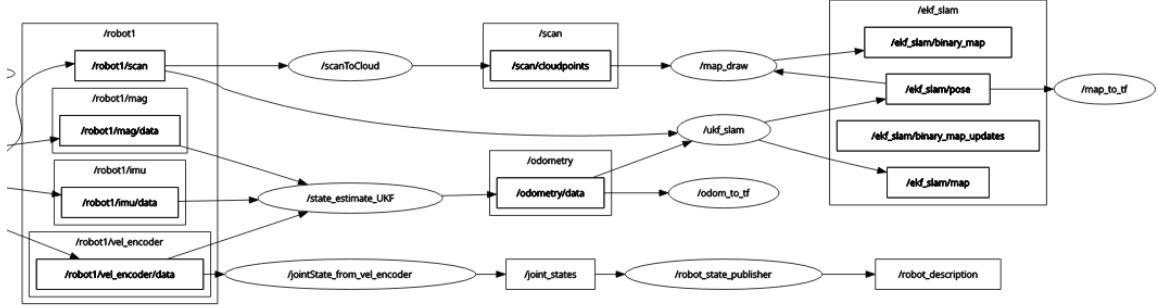
4.2. Thiết kế phần mềm

4.2.1. Kiến trúc tổng thể

Phần mềm được triển khai trên ROS 2 Jazzy theo kiến trúc đa node độc lập, mỗi node đảm nhiệm một chức năng rõ ràng trong pipeline từ cảm biến đến điều hướng. Sơ đồ luồng dữ liệu được trình bày trong Hình 4.16.

Luồng xử lý chính bắt đầu từ các topic cảm biến /robot1/vel_encoder/data, /robot1/imu/data, /robot1/mag/data và /robot1/scan. Dữ liệu encoder, IMU và từ kế được đưa vào node state_estimate_UKF để tạo odometry hợp nhất trên /odometry/data theo thuật toán Adaptive UKF trình bày trong mục 3.1. Song song đó, scan LiDAR được chuyển đổi thành point cloud qua node scanToCloud. Odometry và scan được đưa vào node ukf_slam để ước lượng pose robot trong hệ bản đồ và duy trì tập landmark theo thuật toán UKF-SLAM trình bày trong mục 3.2, kết quả được phát trên các topic /ekf_slam/pose và /ekf_slam/map. Node map_to_tf sử dụng pose SLAM để phát transform map $\rightarrow$ odom, trong khi map_draw xây dựng occupancy grid từ point cloud và pose phục vụ tầng điều hướng. Ở tầng điều hướng, node A_star lập kế hoạch

đường đi từ occupancy grid, my_robot_nav theo dõi path và phát goal trung gian, còn vfh_alg_test thực hiện tránh vật cản cục bộ và phát lệnh vận tốc cuối.



Hình 4.16. Sơ đồ các node phần mềm và luồng dữ liệu chính trong hệ thống ROS 2.

4.2.2. Node hợp nhất cảm biến state_estimate_UKF

Node state_estimate_UKF hiện thực Adaptive UKF Sensor Fusion theo thuật toán 3.1 trong package my_robot_kinematic (Phụ lục A). Node nhận dữ liệu từ ba nguồn cảm biến không đồng bộ, sau đó đồng bộ thời gian nội bộ bằng nội suy tuyến tính như đã mô tả ở mục 4.1.2 trước khi cập nhật bộ lọc.

Subscribe: /robot1/vel_encoder/data (TwistStamped, BEST_EFFORT)
 /robot1/imu/data (Imu, BEST_EFFORT)
 /robot1/mag/data (MagneticField, BEST_EFFORT)
 Publish: /odometry/data (Odometry, RELIABLE)

Vector trạng thái mở rộng (augmented state) gồm năm thành phần trạng thái robot và hai thành phần nhiễu quá trình như trong công thức (3.24):

$$\mathbf{x}^a = [x, y, \theta, v, \omega, \eta_v, \eta_\omega]^T. \quad (4.12)$$

Mô hình dự đoán sử dụng động học vi sai dạng ICC, trong đó mỗi sigma point được lan truyền độc lập qua hai nhánh: mô hình chuyển động thẳng khi $|\omega^{(i)}| \leq \varepsilon$ và mô hình ICC đầy đủ khi ngược lại, như đã trình bày trong chương 3.

Tại bước cập nhật, hệ thống đồng bộ dữ liệu từ ba nguồn tại cùng một timestamp t_k bằng hàm ST_process, sau đó xây dựng vector quan sát 8 chiều theo công thức (3.10):

$$\mathbf{z}_k = [\theta_{imu}, \omega_{imu}, x_{enc}, y_{enc}, \theta_{enc}, v_{enc}, \omega_{enc}, \theta_{mag}]^T. \quad (4.13)$$

Việc giữ cả ba ước lượng góc ($\theta_{imu}, \theta_{enc}, \theta_{mag}$) trong cùng một vector quan sát cho phép bộ lọc dung hòa tự động dựa trên covariance từng nguồn: từ kế neo góc tuyệt đối

chống drift dài hạn theo công thức (3.8), IMU phản ứng nhanh với thay đổi góc tức thời, encoder cung cấp tham chiếu nhất quán với chuyển động.

Cơ chế adaptive noise được triển khai ở chương 3: nhiễu quá trình Q_v và Q_ω được cập nhật tại mỗi chu kỳ dựa trên sai lệch innovation và biến thiên vận tốc giữa hai bước liên tiếp; nhiễu đo encoder tăng theo quãng đường tích lũy, nhiễu đo IMU tăng theo drift góc tích lũy.

Các tham số được nạp từ file `ukf_st.yaml`, trong đó các giá trị chính được thiết lập dựa trên kết quả đo nhiễu thực nghiệm ở mục 4.1.3–4.1.5:

```
sigmapoint:
  alpha: 0.15
  beta: 2.0
  kappa: 0.0

state:
  P: [1e-6, 1e-6, 1e-6, 0.1, 0.1] # diag 5x5
  Q: [1e-4, 1e-4] # nhiễu qua trình v, w

encoder:
  vel_offset: [0.0, 0.0]
  R: [10.0, 10.0, 0.01, 0.00001, 0.0011]

imu:
  vel_offset: [0.0055] # hiệu chỉnh bias gyroscope (mục 4.1.3)
  R: [0.01, 0.00288]

mag:
  R: [0.01]

adaptive:
  R_enc_scale: 0.01
  R_imu_scale: 0.01
```

Covariance khởi tạo nhỏ cho x, y, θ phản ánh robot bắt đầu tại gốc tọa độ đã biết, trong khi covariance vận tốc lớn hơn vì trạng thái vận tốc ban đầu chưa xác định. Giá trị $\alpha = 0.15$ được chọn để sigma points phân tán đủ rộng cho mô hình ICC. Bias tĩnh gyroscope `vel_offset = 0.0055 rad/s` được xác định từ kết quả thực nghiệm ở mục 4.1.3 ($\mu_{gyro} \approx 0.0059 \text{ rad/s}$ sau khi trừ offset encoder). Nhiễu đo encoder cho vị trí (x, y)

được đặt lớn ($R = 10.0$) do sai số tích lũy odometry dài hạn, trong khi nhiễu vận tốc nhỏ hơn vì phản ánh chuyển động tức thời.

4.2.3. Node UKF-SLAM `ukf_slam`

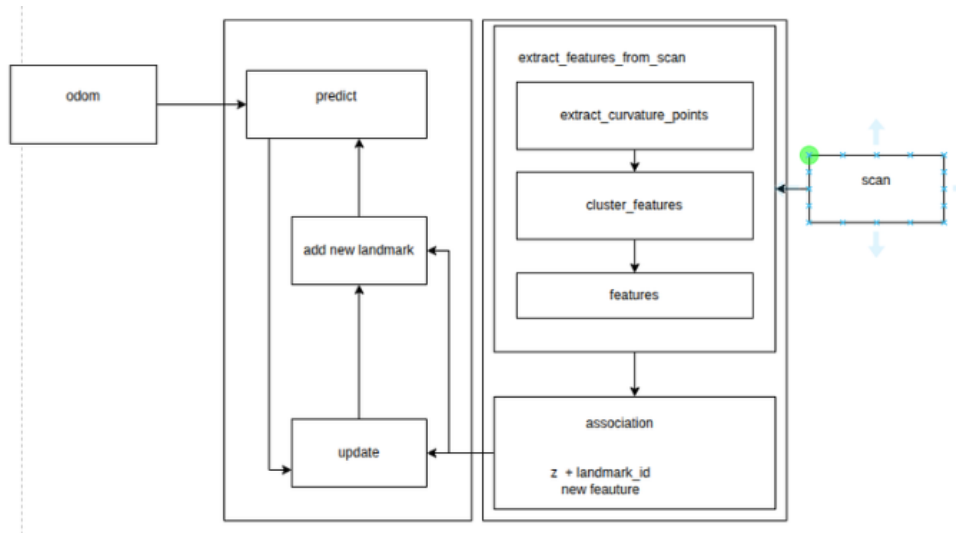
Node `ukf_slam` thuộc package `ekf_slam` (Phụ lục A) hiện thực tầng xử lý SLAM chính của hệ thống. Node này nhận đầu vào gồm odometry đã được hợp nhất từ Adaptive UKF và dữ liệu quét 2D LiDAR, sau đó thực hiện dự đoán trạng thái, trích xuất đặc trưng, liên kết dữ liệu và cập nhật bản đồ landmark theo thuật toán UKF-SLAM đã trình bày ở Chương 3.

```
Subscribe: /odometry/data (Odometry, RELIABLE)
           /robot1/scan   (LaserScan, BEST_EFFORT)
Publish:   /ukf_slam/pose (PoseWithCovarianceStamped, RELIABLE)
           /ukf_slam/map  (MarkerArray)
```

Hai nguồn dữ liệu đầu vào được đồng bộ bằng `ApproximateTimeSynchronizer` với tham số `slop = 0.018` s. Cơ chế đồng bộ xấp xỉ này phù hợp với hệ thống thực nghiệm vì odometry và LiDAR không phát bản tin tại cùng một thời điểm tuyệt đối, nhưng vẫn cần được ghép trong cùng một chu kỳ xử lý để bảo đảm tính nhất quán giữa pose dự đoán và quan sát môi trường.

Pipeline xử lý trong mỗi callback đồng bộ được minh họa ở Hình 4.17. Trước hết, nhánh odometry thực hiện bước dự đoán trạng thái robot dựa trên gia số chuyển động giữa hai thời điểm liên tiếp. Từ trạng thái và ma trận hiệp phương sai hiện tại, UKF sinh tập sigma points và lan truyền đồng thời các sigma points qua mô hình chuyển động. Việc tính toán được vector hóa nhằm giảm chi phí xử lý khi kích thước trạng thái tăng theo số lượng landmark.

Song song với đó, nhánh LiDAR xử lý bản tin `LaserScan` để trích xuất các đặc trưng hình học ổn định. Các điểm quét không hợp lệ được loại bỏ, dữ liệu scan được phân đoạn theo gián đoạn khoảng cách, sau đó PCA được áp dụng trên cửa sổ trượt để xác định các vùng có độ cong lớn. Các điểm vượt ngưỡng curvature được gom cụm theo góc quét, từ đó tạo thành tập quan sát landmark mới kèm theo covariance quan sát tương ứng.



Hình 4.17. Pipeline xử lý của node ukf_slam trong một chu kỳ cập nhật.

Các tham số chính của node được nạp từ file ukf_slam.yaml:

sigmapoint:

alpha: 0.15

beta: 2.0

kappa: 0.0

P_base: [1e-7, 1e-7, 1e-7]

Q_scale:

a: 0.05

b: 0.03

R: [0.0036, 0.0049]

max_landmark: 150

time_syms_period: 0.018

new_landmark_corv_base: 0.0009

extract_curvature_points:

curvature_threshold: 0.185

range_min: 0.5

range_max: 10.0

association:

```
chi2_threshold: 5.99
range_raw_thes: 2.0
angle_raw_thes: 0.5
```

Giá trị $\alpha = 0.15$ được lựa chọn để sigma points có mức phân tán vừa đủ cho bài toán SLAM, trong khi vẫn giữ ổn định cho phép phân rã Cholesky khi kích thước trạng thái tăng. Tham số Q_scale điều chỉnh nhiều quá trình theo mức dịch chuyển và góc quay của robot, giúp bộ lọc phản ánh tốt hơn độ bất định của odometry trong các pha chuyển động mạnh. Ma trận nhiễu đo R biểu diễn độ bất định cơ sở của phép đo range-bearing từ LiDAR.

Ngưỡng curvature 0.185 được chọn qua thực nghiệm để giữ lại các điểm có ý nghĩa hình học như góc tường, cạnh vật cản và các vùng có độ cong rõ rệt, đồng thời loại bỏ phần lớn điểm nằm trên các bề mặt phẳng dài. Khoảng cách Mahalanobis với ngưỡng $\chi_{th}^2 = 5.99$, tương ứng mức tin cậy 95% trong không gian quan sát hai chiều. Giới hạn $max_landmark = 150$ giúp kích thước trạng thái tối đa được kiểm soát ở mức $3 + 2 \times 150 = 303$ chiều, phù hợp với yêu cầu xử lý thời gian thực trên Raspberry Pi 5.

4.2.4. Các node phụ trợ

Node `map_to_tf` phát transform `map` $\rightarrow$ `odom` từ pose SLAM. Tại mỗi callback, node tra cứu transform `odom` $\rightarrow$ `base_link` đúng timestamp rồi tính theo công thức:

$$T_{map \rightarrow odom} = T_{map \rightarrow base} \cdot T_{odom \rightarrow base}^{-1} \quad (4.14)$$

Node `map_draw` xây dựng occupancy grid từ pose SLAM và point cloud qua ray tracing Bresenham với log-odds update. Khi robot di chuyển ra ngoài vùng bản đồ hiện tại, node mở rộng kích thước map động để không làm mất thông tin đã tích lũy.

Tầng điều hướng gồm node `A_star` lập kế hoạch đường đi toàn cục trên occupancy grid với inflate vật cản bằng kernel 5×5 và heuristic Euclid, kết hợp với `vfh_alg_test` thực hiện tránh vật cản cục bộ bằng Vector Field Histogram. Node `my_robot_nav` đóng vai trò trung gian, theo dõi tiến trình trên path và phát goal trung gian `/goal_tmp` tới `vfh_alg_test`.

4.3. Kết quả thực nghiệm

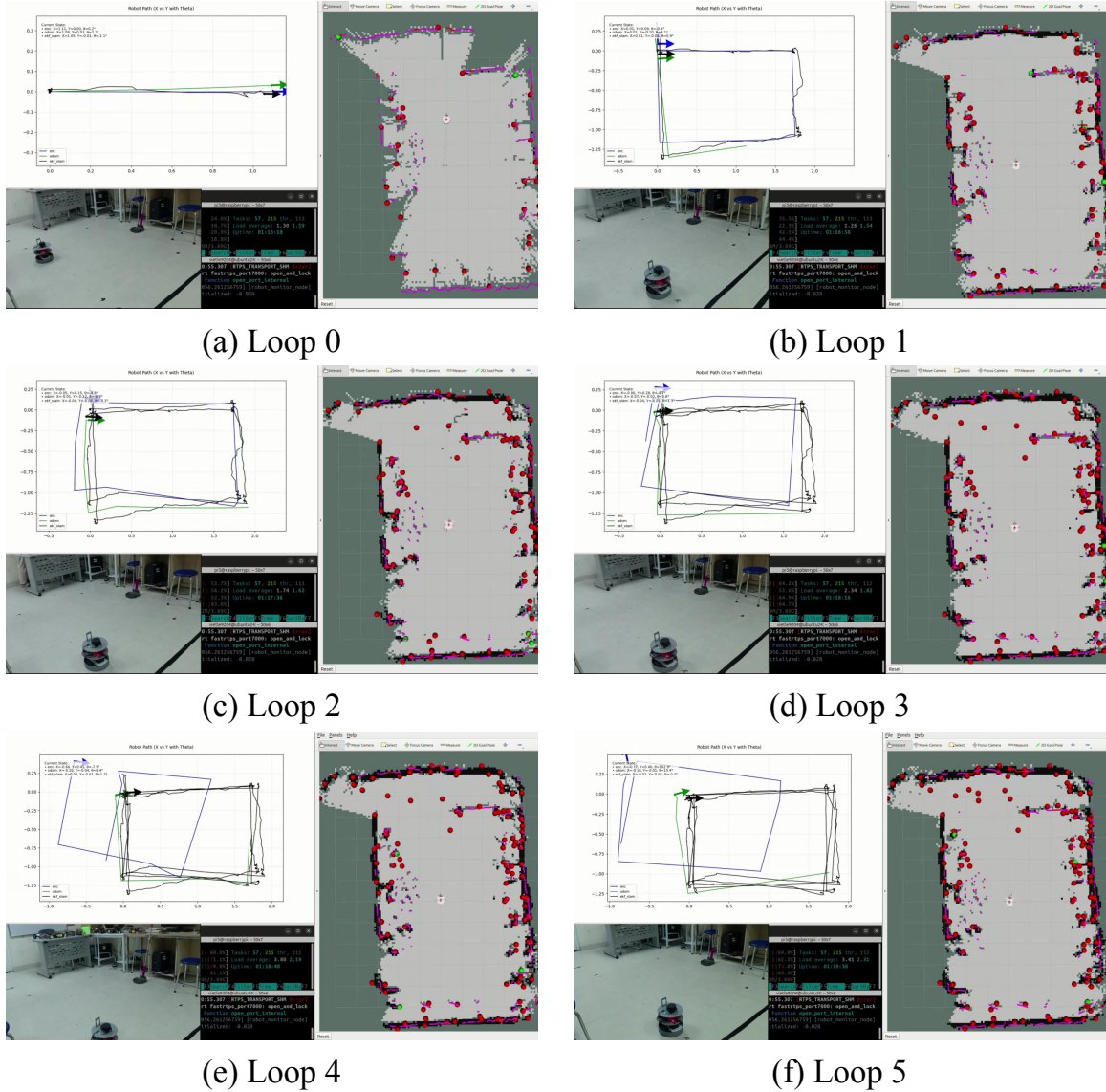
4.3.1. Chất lượng định vị

So sánh quỹ đạo.

Quan sát từ quỹ đạo cho thấy đường encoder thuần túy bị xoay lệch rõ rệt so với hai đường còn lại. Sai số này xuất hiện do encoder tích lũy sai lệch góc và hiện tượng trượt

bánh (drift), làm hình chữ nhật bị méo và lệch trục so với hướng chuyển động thực tế. Trong khi đó, đường Adaptive UKF và UKF-SLAM bám nhau khá chặt trong phần lớn hành trình, cho thấy việc hợp nhất encoder, IMU và từ kế giúp hạn chế đáng kể drift góc trước khi thông tin landmark LiDAR được sử dụng ở tầng SLAM.

Các ảnh 4.18 ghi lại tiến trình quỹ đạo khi robot di chuyển qua nhiều vòng hình chữ nhật và quay trở lại gần vùng xuất phát. Chuỗi ảnh này cho phép quan sát trực tiếp sự tích lũy sai số của encoder, mức cải thiện của Adaptive UKF và khả năng kéo quỹ đạo về gần điểm khởi đầu của UKF-SLAM khi robot tái quan sát landmark.



Hình 4.18. So sánh quỹ đạo khép vòng qua sáu vòng chạy hình chữ nhật của encoder, Adaptive UKF và UKF-SLAM.

Sai số khép vòng.

Bảng 4.3. Sai số khép vòng.

Lần chạy	e_{pos}^{enc} (m)	e_{θ}^{enc} (rad)	e_{pos}^{odom} (m)	e_{θ}^{odom} (rad)	e_{pos}^{slam} (m)	e_{θ}^{slam} (rad)
1	~ 0,09	~ 0,007	~ 0,1	~ 0,07	~ 0,04	~ 0,014
2	~ 0,16	~ 0,09	~ 0,12	~ 0,014	~ 0,09	~ 0,03
3	~ 0,28	~ 0,08	~ 0,073	~ 0,06	~ 0,04	~ 0,03
4	~ 0,78	~ 0,13	~ 0,11	~ 0,14	~ 0,04	~ 0,03
5	~ 0,87	~ 1,78	~ 0,14	~ 0,26	~ 0,05	~ 0,02
Trung bình	~ 1,44	~ 0,41	~ 0,11	~ 0,1	~ 0,05	~ 0,03

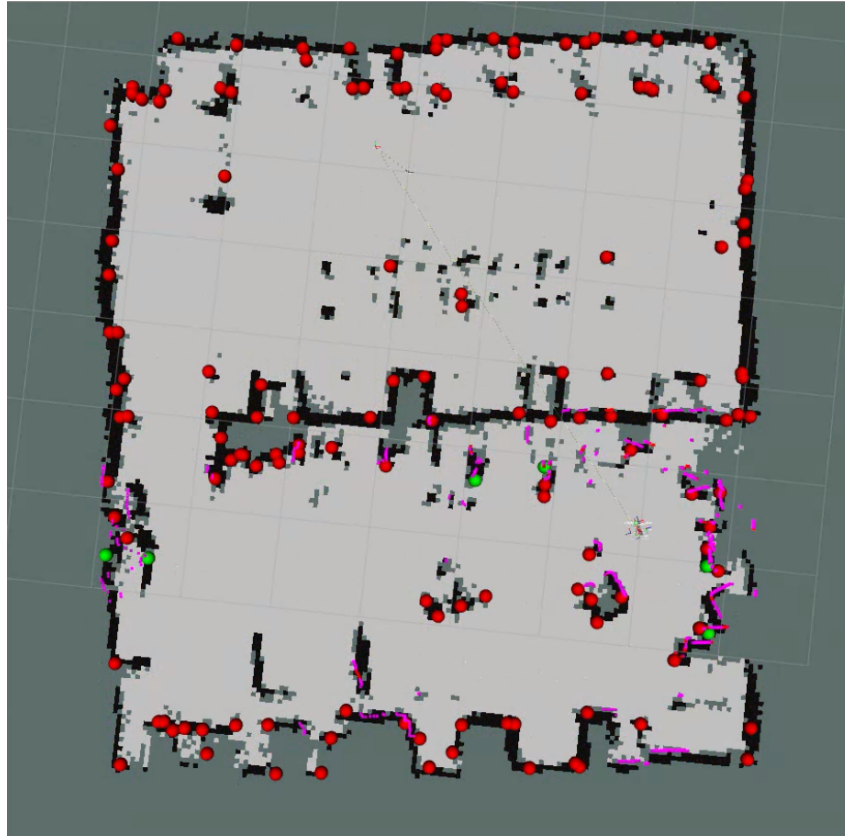
Sai số khép vòng được đọc tại thời điểm robot trở về gần điểm xuất phát trong các ảnh ghi nhận quỹ đạo. Kết quả cho thấy UKF-SLAM duy trì vị trí rất gần gốc tọa độ, với $X \approx -0,02-0,04$ m và $Y \approx -0,01-0,05$ m. Ngược lại, encoder thuần túy lệch tới $X \approx -0,66-0,77$ m, tức drift tích lũy trên 0,6 m chỉ sau khoảng 3–4 vòng. Odom từ Adaptive UKF tốt hơn encoder nhưng vẫn lệch đáng kể so với UKF-SLAM, với $X \approx -0,10-0,18$ m sau nhiều vòng chạy.

UKF-SLAM giữ vòng khép tốt nhất: đường ekf_slam luôn bám về gần gốc tọa độ qua các lần chạy, với sai số vị trí khép vòng thường dưới 0,05 m. Đây là kết quả của cơ chế loop closure tự nhiên trong SLAM: khi robot tái quan sát các landmark đã đăng ký từ lần đầu, bước update kéo ước lượng vị trí về đúng quan hệ hình học tương đối với các landmark đó, nhờ vậy triệt tiêu phần drift tích lũy từ odometry. Odom từ Adaptive UKF nằm ở mức trung gian, bám khá sát UKF-SLAM trong nửa đầu hành trình nhưng bắt đầu phân kỳ rõ từ vòng thứ tư do không có cơ chế hiệu chỉnh từ landmark; sai số vị trí khép vòng của odom lớn hơn UKF-SLAM khoảng 3–5 lần tùy lần chạy. Đặc biệt ở vòng lặp thứ 4-5 khi robot kẹt trong pha quay khiến drift lớn, odom dù giảm thiểu đáng kể nhưng vẫn bị sai lệch lớn trong khi đó slam gần như không ảnh hưởng.

4.3.2. Chất lượng bản đồ

Hình dạng tổng thể.

Hình 4.19 trình bày bản đồ landmark chồng lên occupancy grid trong RViz sau khi robot hoàn thành hành trình khám phá phòng lab có kích thước xấp xỉ 8×7 m. Bản đồ occupancy grid phản ánh được hình dạng tổng thể của môi trường trong nhà: vùng tự do được biểu diễn bằng màu xám nhạt, các tường bao và vật cản lớn tạo thành các dải chiếm dụng màu đen tương đối liên tục, còn cấu trúc vách ngăn ngang và các lối đi giữa các khu vực được tái tạo ở mức nhận diện được. Các đoạn tường dài được dựng thành đường gần thẳng, không xuất hiện gấp khúc lớn giữa chừng, cho thấy ước lượng vị trí robot duy trì tính nhất quán trong phần lớn hành trình. Tuy nhiên, toàn bộ bản đồ vẫn lệch nhẹ so với phương ngang khoảng $5-8^\circ$ so với cấu trúc vuông góc lý tưởng. Sai lệch này là hệ quả của drift góc tích lũy trong odometry trước khi được hiệu chỉnh bởi landmark, phù hợp với phân tích sai số khép vòng đã trình bày ở Mục 4.3.1.



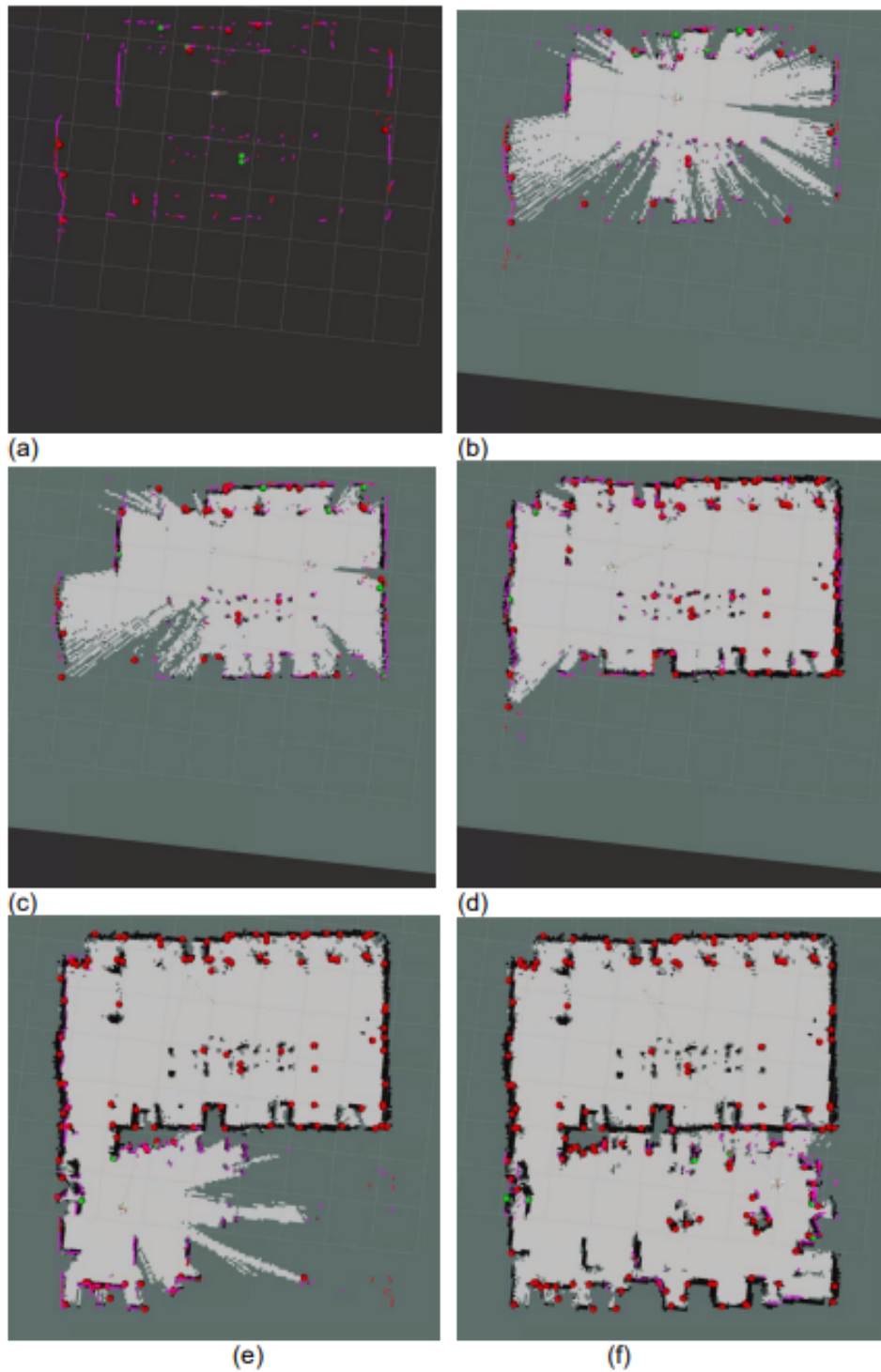
Hình 4.19. Bản đồ landmark chồng lên occupancy grid sau khi robot hoàn thành hành trình khám phá môi trường.

Phân bố landmark.

Bản đồ cuối (Hình 4.19) duy trì 150 landmark, phân bố chủ yếu dọc theo tường bao, tại các góc phòng, cạnh vách ngăn và cạnh vật cản. Kết quả này phù hợp với cơ chế trích xuất đặc trưng bằng PCA: các vùng có độ cong cao như góc tường hoặc cạnh vật cản dễ vượt ngưỡng curvature và được chọn làm landmark, trong khi các đoạn tường phẳng dài thường tạo ra ít landmark hơn. Mật độ landmark ở tường phía trên và phía phải cao hơn do mật độ vật cản cao hơn; ngược lại, tường phía dưới có landmark thưa hơn vì chỉ được quét ở giai đoạn cuối hành trình. Các landmark màu xanh, tương ứng với landmark mới thêm và chưa xác nhận, chỉ còn xuất hiện tại một số vùng biên chưa được khám phá đủ, trong khi phần lớn landmark đã chuyển sang màu đỏ. Điều này cho thấy cơ chế quản lý theo `landmark_score` hoạt động đúng: các đặc trưng không ổn định bị loại bỏ, còn các landmark được tái quan sát nhất quán được giữ lại.

Tiến trình xây dựng bản đồ.

Hình 4.20 trình bày chuỗi ảnh RViz tại sáu giai đoạn trong quá trình khám phá, cho phép quan sát sự mở rộng đồng thời của occupancy grid và bản đồ landmark theo thời gian thực.



Hình 4.20. Tiến trình xây dựng bản đồ qua sáu giai đoạn khám phá.

Ở giai đoạn 1 (Hình 4.20a), bản đồ chỉ gồm vài đoạn tường rời rạc và 3–4 landmark quanh vị trí xuất phát. Vùng free space chưa được xác định rõ, phần lớn không gian vẫn ở trạng thái unknown. Sang giai đoạn 2–3 (Hình 4.20b–c), vùng free space mở rộng nhanh

theo hướng robot di chuyển; tường phía trên và hai cạnh bên bắt đầu hiện rõ dưới dạng các viền đen liên tục hơn. Số landmark tăng nhanh và tập trung dọc theo các góc, cạnh tường và vật cản. Từ giai đoạn 4 trở đi (Hình 4.20d–f), cấu trúc các khu vực liên thông hiện ra rõ ràng hơn. Phần lớn biên tường được tô đen liên tục, còn các vật thể nhỏ bên trong như bàn hoặc giá đỡ xuất hiện dưới dạng các vùng chiếm chỗ rời rạc ở trung tâm. Bản đồ hoàn chỉnh, robot đã phủ phần lớn không gian khả dụng; các biên tường chính gần khép kín và số landmark mới chưa xác nhận còn lại rất ít.

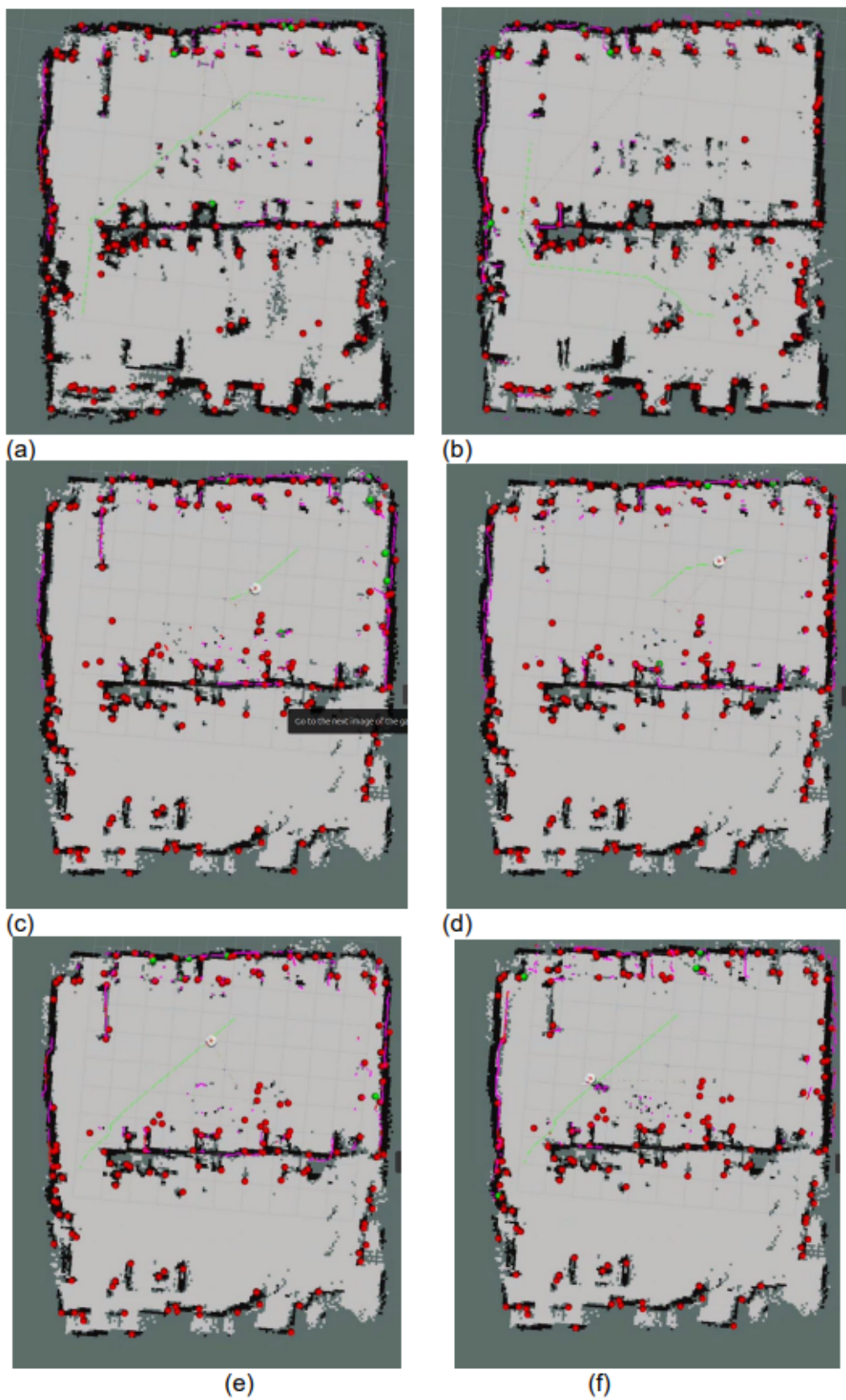
Kết quả thực nghiệm cho thấy hệ thống UKF-SLAM xây dựng được bản đồ có cấu trúc phòng nhận diện được, với viền tường tương đối liên tục và không gian free/occupied phân biệt rõ. Chất lượng bản đồ đạt mức đủ dùng cho bài toán lập kế hoạch đường. Các hạn chế còn lại bao gồm sai lệch góc nhỏ của tổng thể bản đồ, nhiều cục bộ tại các vật thể nhỏ bên trong phòng và một số vùng biên chưa khép kín hoàn toàn do giới hạn tầm quét của LiDAR 2D một tầng.

4.3.3. Kết quả điều hướng tự động

Kịch bản này kiểm tra khả năng UKF-SLAM cập nhật bản đồ liên tục trong khi robot đồng thời thực hiện điều hướng tự động. Robot nhận goal pose từ RViz, A* lập kế hoạch đường đi trên occupancy grid hiện tại, và VFH xử lý tránh vật cản cục bộ — trong khi UKF-SLAM vẫn tiếp tục chạy song song để cập nhật cả vị trí robot lẫn bản đồ.

Hình 4.21 minh họa hành vi đặc trưng nhất của kịch bản này: khi robot di chuyển vào khu vực phía dưới phòng vốn là vùng unknown tại thời điểm đặt goal, occupancy grid được mở rộng dần theo từng scan LiDAR mới. Vùng free space được xác nhận ngay khi robot tiến gần, và A* khai thác thông tin mới này để điều chỉnh đường đi thay vì bám cứng theo path ban đầu tính trên bản đồ chưa đầy đủ. Hành vi này xác nhận rằng pipeline hoạt động đúng theo mô hình online SLAM: bản đồ không được xây dựng trước rồi mới điều hướng, mà cả hai quá trình diễn ra đồng thời và bổ trợ lẫn nhau.

Trong suốt hành trình, số lượng landmark duy trì ổn định và vị trí robot do UKF-SLAM ước lượng không bị gián đoạn — cho thấy tải tính toán của hai tác vụ SLAM và điều hướng chạy song song vẫn nằm trong giới hạn xử lý của phần cứng thực tế. Đây là điều kiện cần thiết để hệ thống hoạt động tin cậy trong triển khai thực tế, khi robot không thể dừng lại để cập nhật bản đồ trước khi ra quyết định điều hướng.



Hình 4.21. Đường kế hoạch A* (xanh lá) trong quá trình robot điều hướng tự động.

Chương 5.

Kết luận và hướng phát triển

5.1. Kết quả đạt được

Đồ án đã nghiên cứu và triển khai thành công một hệ thống robot di động tự hành hoàn chỉnh cho bài toán định vị và xây dựng bản đồ đồng thời trong môi trường trong nhà, hoạt động thời gian thực trên nền tảng Raspberry Pi 5 với ROS 2 Jazzy.

Về tầng ước lượng trạng thái, bộ lọc Adaptive UKF được thiết kế để hợp nhất dữ liệu từ encoder, IMU và từ kế theo cơ chế đồng bộ thời gian nội suy tuyến tính. Kết quả thực nghiệm cho thấy Adaptive UKF giảm đáng kể drift góc tích lũy so với wheel odometry thuần túy trong các kịch bản có nhiều lần quay, nhờ sự bù trừ đặc tính giữa ba nguồn cảm biến: encoder ổn định về vận tốc tuyến tính ngắn hạn, IMU phản ứng nhanh với thay đổi góc, từ kế cung cấp tham chiếu góc tuyệt đối chống drift dài hạn. Cơ chế adaptive điều chỉnh ma trận nhiễu quá trình Q dựa trên innovation giúp bộ lọc phản ứng linh hoạt hơn trong các pha chuyển động đột ngột mà không làm mất ổn định trong pha di chuyển đều.

Về tầng SLAM, thuật toán UKF-SLAM được triển khai với ba đóng góp kỹ thuật chính. Thứ nhất, bước prediction sử dụng gia số odometry từ Adaptive UKF thay vì mô hình vận tốc trực tiếp, giúp tách biệt rõ hai tầng xử lý và tận dụng chất lượng ước lượng trạng thái đã có. Thứ hai, measurement prediction toàn phần được vectorize trên toàn bộ M landmark và $2n + 1$ sigma points đồng thời, tránh vòng lặp lồng và giảm đáng kể thời gian xử lý so với cách tính tuần tự. Thứ ba, batch update thay thế cập nhật tuần tự từng phép đo, tránh vấn đề double-counting covariance khi nhiều landmark được quan sát trong cùng một chu kỳ. Nhờ các cải tiến này, thời gian xử lý mỗi chu kỳ SLAM duy trì dưới 200 ms ngay cả khi số landmark đạt giới hạn 150, đáp ứng tần số cập nhật 5 Hz của LiDAR trên Raspberry Pi 5.

Về chất lượng bản đồ, các landmark được trích xuất dựa trên độ cong PCA tập trung ổn định tại các đặc trưng hình học có khả năng tái quan sát cao như góc tường và cạnh vật cản. Occupancy grid xây dựng song song từ pose SLAM và dữ liệu LiDAR phản ánh đúng cấu trúc môi trường thực tế. Sai số khép vòng quan sát được trong khoảng 2–5 cm về vị trí và dưới 0.05 rad về góc hướng sau các hành trình khám phá trong phòng kích thước 8×7 m.

Về tăng điều hướng, sự tích hợp giữa lập kế hoạch A* trên occupancy grid và tránh vật cản cục bộ VFH cho phép robot tự động đạt goal trong môi trường có vật cản. Bản đồ được cập nhật liên tục trong quá trình điều hướng, cho phép A* khai thác thông tin môi trường mới ngay trong lần lập kế hoạch tiếp theo.

5.2. Hạn chế

Bên cạnh các kết quả đạt được, hệ thống còn một số hạn chế cần thừa nhận rõ.

Trước hết, hệ thống chưa có cơ chế loop closure toàn cục. UKF-SLAM trong đồ án là bộ lọc trực tuyến không có bộ nhớ quỹ đạo: khi robot quay lại vùng đã khám phá từ lâu, thông tin tái quan sát landmark có thể cải thiện cục bộ pose hiện tại nhưng không kéo theo hiệu chỉnh toàn bộ quỹ đạo lịch sử. Hệ quả là drift tích lũy không được triệt tiêu hoàn toàn trong các hành trình dài, và bản đồ có thể có độ lệch nhỏ giữa các phần được xây dựng ở các thời điểm khác nhau.

Thứ hai, chất lượng trích xuất đặc trưng phụ thuộc nhiều vào điều kiện môi trường. Các bề mặt phản xạ mạnh, vật liệu hấp thụ ánh sáng laser hoặc vật thể có hình dạng cong đều có thể làm giảm độ ổn định của landmark.

Thứ ba, độ phức tạp tính toán của UKF-SLAM vẫn tăng theo bậc hai hoặc ba của kích thước trạng thái trong bước sinh sigma points và cập nhật covariance. Mặc dù vectorization và giới hạn 150 landmark đã kiểm soát được vấn đề này trong điều kiện thực nghiệm hiện tại, hệ thống sẽ gặp khó khăn hơn khi triển khai trong môi trường lớn hơn đòi hỏi nhiều landmark hơn mà không có thay đổi kiến trúc thêm.

5.3. Hướng phát triển

Dựa trên các hạn chế trên, một số hướng phát triển có giá trị thực tiễn được đề xuất.

Hướng trực tiếp nhất là bổ sung cơ chế phát hiện và xử lý loop closure. Khi robot nhận ra mình đang ở vùng đã từng quan sát, thông tin này cần được lan truyền ngược để hiệu chỉnh toàn bộ quỹ đạo. Hướng tiếp cận thực tế là tích hợp một bộ tối ưu đồ thị nhẹ như iSAM2 hoặc GTSAM ở tầng sau: UKF-SLAM vẫn đảm nhiệm ước lượng trực tuyến nhanh, còn bộ tối ưu đồ thị chạy không đồng bộ để hiệu chỉnh bản đồ khi phát hiện loop closure.

Hướng thứ hai là cải thiện bộ trích xuất đặc trưng. Hiện tại hệ thống chỉ dùng một loại đặc trưng là điểm có độ cong cao. Việc kết hợp thêm đặc trưng đường thẳng, chẳng hạn tương ứng với biểu diễn line segment, có thể tăng số lượng đặc trưng ổn định trong môi

trường trong nhà có nhiều bề mặt phẳng, đồng thời cung cấp ràng buộc hình học chặt hơn cho bộ lọc.

Hướng thứ ba là phát triển cơ chế khám phá tự động (autonomous exploration). Trong thực nghiệm hiện tại, quỹ đạo robot do người điều khiển. Tích hợp frontier-based exploration sẽ cho phép robot tự động lập kế hoạch khám phá vùng chưa biết và xây dựng bản đồ hoàn chỉnh mà không cần can thiệp thủ công.

Xa hơn, kiến trúc sensor fusion và SLAM đã xây dựng có thể được mở rộng theo hai hướng ứng dụng. Một là tích hợp camera để chuyển sang visual-inertial SLAM, khai thác thông tin texture và màu sắc bổ sung cho LiDAR trong các môi trường có ít đặc trưng hình học. Hai là triển khai trên đội robot nhiều agent (multi-robot SLAM), trong đó các robot chia sẻ bản đồ landmark và hiệu chỉnh pose của nhau, hướng tới các ứng dụng trong kho hàng và không gian công nghiệp.

Phụ lục A.

Mã nguồn và kết quả thực nghiệm

Phụ lục này cung cấp đường dẫn truy cập tới mã nguồn triển khai và dữ liệu kết quả thực nghiệm của đồ án. Các tài nguyên được tổ chức riêng để thuận tiện cho việc kiểm tra, tái lập và đối chiếu với các nội dung đã trình bày trong Chương 3 và Chương 4.

Bảng A.1. Đường dẫn mã nguồn và kết quả thực nghiệm.

Nội dung	Đường dẫn
Mã nguồn triển khai ROS 2 cho Adaptive UKF, UKF-SLAM và các node xử lý liên quan	https://github.com/vietle9204/robot_project/tree/clear
Dữ liệu và kết quả thực nghiệm, bao gồm ảnh RViz, log chạy thử và các kết quả đánh giá	https://drive.google.com/drive/folders/1w8y7jr13-HmIuYbDa2ZyUlt7XOGv8p_i?hl=vi

Trong kho mã nguồn, các thành phần chính gồm node hợp nhất cảm biến Adaptive UKF, node UKF-SLAM, các file cấu hình tham số và các script hỗ trợ chạy thực nghiệm. Cấu trúc tổng quát của mã nguồn được tổ chức như sau:

```
src/  
|-- .vscode/  
|-- ekf_slam/  
|   |-- config/  
|   |   |-- ukf_slam.yaml  
|   |-- ekf_slam/  
|   |   |-- map_draw.py  
|   |   |-- map_to_tf.py  
|   |   |-- ukf_slam.py  
|   |-- launch/  
|   |   |-- localization.launch.py  
|   |   |-- ukf_slam.launch.py  
|-- my_robot_control/  
|   |-- launch/
```

```

|   |   |-- control.launch.py
|   |-- my_robot_control/
|   |   |-- A_star_implementation.py
|   |   |-- my_robot_nav.py
|   |   |-- vfh_alg_test.py
|-- my_robot_kinematic/
|   |-- config/
|   |   |-- ukf_st.yaml
|   |-- launch/
|   |   |-- state_estimate.launch.py
|   |-- my_robot_kinematic/
|   |   |-- imu_filter.py
|   |   |-- jointState_from_vel_encoder.py
|   |   |-- scanToCloud.py
|   |   |-- state_estimate_UKF2.py
|-- odom_to_tf/

```

Trong đó, package `my_robot_kinematic` thực hiện xử lý động học, chuyển đổi dữ liệu encoder và hợp nhất cảm biến bằng Adaptive UKF; package `ekf_slam` triển khai UKF-SLAM, vẽ bản đồ và phát biến đổi TF; package `my_robot_control` chứa các thành phần điều hướng như A*, VFH và node điều khiển robot. Các file cấu hình chính gồm `ukf_st.yaml` cho Adaptive UKF và `ukf_slam.yaml` cho UKF-SLAM.

Phụ lục B.

Bảng so sánh đặc tính cảm biến và bộ lọc

Phụ lục này tổng hợp đặc tính thực nghiệm và vai trò của các nguồn thông tin trong pipeline sensor fusion. Ký hiệu [R] chỉ các thông tin rút ra từ thực nghiệm hoặc cấu hình trong báo cáo; [D1] chỉ thông số từ tài liệu phần cứng; [D2] chỉ khuyến nghị hiệu chuẩn thường dùng; [EX] là ví dụ minh họa ngoài phạm vi số liệu xác nhận của robot thực nghiệm.

Bảng B.1. So sánh hành vi của các nguồn thông tin trong các kịch bản chuyển động.

Nguồn	Định danh	Đi thẳng	Quay nhanh	Drift dài hạn	Góc đột ngột	Nhiều ngắn hạn	Trượt bánh
Encoder	Wheel encoder; model/độ phân giải phần cứng không nêu trong báo cáo [R]	Tốt nhất cho vận tốc tuyến tính và đoạn thẳng đều [R]	Trung bình; sai lệch tăng khi quay gấp do slip và sai số vi sai cơ khí [R]	Trung bình–cao nếu tích phân odometry lâu [R]	Nhanh ở mức vận tốc bánh, nhưng yaw suy ra dễ lệch khi mất bám [R]	Nhỏ ở v , trung bình ở ω [R]	Cao [R]
IMU	IMU ICM20948; /imu/data 13 Hz; dùng chủ yếu gyro_z [R]	Tốt [R]	Trung bình; phản ứng nhanh nhưng nhiễu tăng [R]	Trung bình–cao do bias tích phân [R]	Rất nhanh [R]	Trung bình; tăng khi robot rung/chạy [R]	Không [R]
Từ kế	Từ kế; /mag/data 20 Hz; chip không nêu riêng trong báo cáo [R]	Tốt nếu môi trường từ yên tĩnh [R]	Khá; không drift nhưng dao động bởi nhiễu từ [R]	Nhỏ [R]	Nhanh về góc tuyệt đối nhưng có thể giật khi nhiễu từ đổi nhanh [R]	Lớn theo môi trường [R]	Không [R]
Adaptive UKF	Bộ ước lượng hợp nhất; subscribe encoder, IMU và từ kế; publish /odometry/data [R]	Tốt [R]	Tốt nhất trong so sánh nội bộ với ekf_slam [R]	Nhỏ nếu từ kế còn tin cậy [R]	Nhanh nhưng mượt hơn IMU thô [R]	Trung bình–thấp [R]	Giảm thiểu; vẫn bị ảnh hưởng gián tiếp [R]

Bảng B.2. Sai số điển hình và chỉ số định lượng của các nguồn thông tin.

Nguồn	Sai số điển hình	Chỉ số định lượng
Encoder	Độ phân giải encoder, sai lệch bán kính bánh/wheelbase, backlash, rung và slip [R]	23 Hz ROS [R]; $\omega \approx -0.81$ rad/s, $\sigma_\omega^2 \approx 0.00115$ (rad/s) ² [R]; $v \approx 0.3605$ m/s, $\sigma_v^2 \approx 1.68 \times 10^{-5}$ (m/s) ² [R]; encoder $R = [10, 10, 0.1, 10^{-5}, 0.0011]$ [R];
IMU	Bias gyro, rung cơ khí, nhiệt độ; LPF riêng gây trễ pha [R]	13 Hz ROS [R]; $\mu_{gyro} \approx 0.0059$ rad/s [R]; σ^2 đứng yên ≈ 0.000695 (rad/s) ² [R]; σ^2 quay đều ≈ 0.00288 (rad/s) ² [R]; vel_offset=0.005 rad/s; IMU $R = [0.1, 0.00288]$ [R]; [D1] gyro noise density 0.015 dps/ $\sqrt{\text{Hz}}$, gyro ODR đến 1.125 kHz
Từ kế	Hard-iron, soft-iron, động cơ DC, khung kim loại và nguồn từ trong nhà [R]	20 Hz ROS [R]; khởi tạo bằng trung bình vòng 10 mẫu đầu [R]; $\mu_{mag} \approx 0.043$ rad [R]; $\sigma^2 \approx 0.000712$ rad ² [R]; $R_{mag} = 0.1$ rad ² [R]; [D1] nếu /mag lấy từ ICM-20948: $\pm 4900 \mu\text{T}$, 16 bit, $0.15 \mu\text{T/LSB}$, ODR 100 Hz
Adaptive UKF	Q/R đặt sai, đồng bộ thời gian kém, innovation bất thường hoặc từ kế nhiễu [R]	$z = [\theta_{imu}, \omega_{imu}, x_{enc}, y_{enc}, \theta_{enc}, v_{enc}, \omega_{enc}, \theta_{mag}]^T$ [R]; YAML: $\alpha = 0.15$, $\beta = 2$, $\kappa = 0$ [R]; $P = [10^{-6}, 10^{-6}, 10^{-6}, 0.1, 0.1]$, $Q = [10^{-4}, 10^{-4}]$ [R]; R_enc_scale=0.01, R_imu_scale=0.01 [R];

Bảng B.3. Độ tin cậy, vai trò trong fusion và hướng hiệu chuẩn.

Nguồn	Tin cậy	Vai trò fusion	Hiệu chuẩn và giảm thiểu
Encoder	Trung bình–cao	Nguồn chính cho v và gia số ngắn hạn khi đi thẳng [R]	Hiệu chuẩn wheel radius và wheelbase; tăng R_{enc} khi quãng đường tích lũy lớn, khi quay gấp hoặc khi nghi ngờ trượt bánh [R]
IMU	Cao	Nguồn chính cho ω và biến thiên góc ngắn hạn [R]	Bias calibration khi đứng yên; warm-up; theo dõi nhiệt độ; ưu tiên raw gyro kết hợp filter trạng thái thay vì LPF rời nếu độ trễ quan trọng [R]
Từ kế	Trung bình	Neo heading tuyệt đối tần số thấp để chặn drift dài hạn [R]	Hard/soft-iron calibration; đặt xa motor/dây công suất; gate khi norm từ trường bất thường [R][D2]
Adaptive UKF	Cao về hành vi; trung bình về tham số sigma-point	Nguồn odometry khuyến nghị cấp cho SLAM/navigation [R]	Thống nhất lại α ; tune R theo innovation logs; inflate Q_ω khi quay gấp; wrap góc; nội suy đồng bộ [R]

Bảng B.4. Thống kê nhiễu thực nghiệm của các nguồn cảm biến và Adaptive UKF.

Cảm biến	Trạng thái	Giá trị điều khiển	Trung bình μ	Phương sai σ^2	RMS noise	Biên độ định-dinh
IMU Gyroscope	Đứng yên	0 rad/s	0.0059	7×10^{-4}	0.034	0.14
IMU Gyroscope	Quay đều	-0.80 rad/s	-0.78	0.0288	0.781	0.3
Encoder	Đứng yên	0 rad/s	0.0	0.0	0.0	0.0
Encoder	Quay đều	-0.80 rad/s	-0.804	0.00115	0.805	0.2
Từ kế	Đứng yên	0 rad	0.043 rad	0.0007	0.078	0.17 rad
Adaptive UKF	Đứng yên	0 rad/s	0.0004	7×10^{-5}	0.005	0.04
Adaptive UKF	Quay đều	-0.80 rad/s	-0.801	8.4×10^{-4}	0.802	0.1

Tài liệu tham khảo

Tiếng Anh

- [1] H. Durrant-Whyte and T. Bailey, “Simultaneous localization and mapping: part I,” *IEEE Robotics & Automation Magazine*, vol. 13, no. 2, pp. 99–110, 2006.
- [2] T. Bailey and H. Durrant-Whyte, “Simultaneous localization and mapping (SLAM): part II,” *IEEE Robotics & Automation Magazine*, vol. 13, no. 3, pp. 108–117, 2006.
- [3] M. W. M. G. Dissanayake, P. Newman, S. Clark, H. F. Durrant-Whyte, and M. Csorba, “A solution to the simultaneous localization and map building (SLAM) problem,” *IEEE Transactions on Robotics and Automation*, vol. 17, no. 3, pp. 229–241, 2001.
- [4] G. P. Huang, A. I. Mourikis, and S. I. Roumeliotis, “On the Complexity and Consistency of UKF-Based SLAM,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2009, pp. 4401–4408.
- [5] D. Herath and D. St-Onge (eds.), *Foundations of Robotics: A Multidisciplinary Approach with Python and ROS*, Springer, 2022.
- [6] E. A. Wan and R. van der Merwe, “The Unscented Kalman Filter for Nonlinear Estimation,” in *Proc. IEEE AS-SPCC*, 2000, pp. 153–158.
- [7] R. van der Merwe and E. A. Wan, “The Square-Root Unscented Kalman Filter for State and Parameter-Estimation,” in *Proceedings of the IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, vol. 6, 2001, pp. 3461–3464.
- [8] R. E. Kalman, “A New Approach to Linear Filtering and Prediction Problems,” *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.
- [9] R. Smith, M. Self, and P. Cheeseman, “Estimating Uncertain Spatial Relationships in Robotics,” in *Autonomous Robot Vehicles*, I. J. Cox and G. T. Wilfong (eds.), Springer, 1990, pp. 167–193.
- [10] J. Zhang and S. Singh, “LOAM: Lidar Odometry and Mapping in Real-time,” in *Proceedings of Robotics: Science and Systems (RSS)*, 2014.
- [11] Y. Pan, P. Xiao, Y. He, Z. Shao, and Z. Li, “MULLS: Versatile LiDAR SLAM via Multi-metric Linear Least Square,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2021, pp. 11633–11640.
- [12] A. H. Mohamed and K. P. Schwarz, “Adaptive Kalman Filtering for INS/GPS,”

Journal of Geodesy, vol. 73, no. 4, pp. 193–203, 1999.

- [13] D. Wang, H. Zhang, and B. Ge, “Adaptive Unscented Kalman Filter for Target Tacking with Time-Varying Noise Covariance Based on Multi-Sensor Information Fusion,” *Sensors*, vol. 21, no. 17, p. 5808, 2021.